

DV Agent Harness L5 完整詳細使用與操作手冊

DV Agent Harness L5 -- 完整詳細使用與操作手冊

本文件彙整 DV Agent Harness L5 截至 2026-09-07 為止已建置完成的全部模組、機制與操作流程，內容取代並擴充先前的 docs/DV_Agent_Harness_L5_Detailed_User_Guide_TC.pdf（發行於 2026-08-27）——凡該份 PDF 涵蓋之內容，本文件皆已納入並補上自該日期之後新增的所有能力（AMBA 匯流排拓模分析、系統級驗證整合、Spec-to-VPlan 編譯管線、DE Command/Runtime 架構、智慧收案、模擬與 LSF Regression、Loop Engineering、驗證智慧、覆蓋率結案與治理、供應鏈風險、共用知識中心與第三方元件治理、Human Control Plane 與 Dashboard、完整 CLI 參考等），讀者日後應以本文件為準。

目錄

1. 導論與核心架構總覽 (Graph / Blackboard / Memory / ReAct / Autonomous Inference)
 2. VIP/UVM 驗證環境生成 (Subsystem-Mode / System-Level Mode, VIP 綁定與結構檢查)
 3. AMBA M x N 匯流排拓模與交易分析 (Fabric Discovery / Port Registry / Transaction / Route-Transform)
 4. AMBA 驗證進階議題 (功能覆蓋率 / 排序網域 / 仲裁 / 安全 / QoS / 效能)
 5. 系統級 (Subsystem-to-System) 驗證:拓模整合、資源仲裁、失效傳播與系統結案
 6. Spec-to-VPlan 編譯流程 (需求萃取、衝突偵測、風險評分、vPlan 完整性)
 7. DE Command / Runtime 架構(指令風格學習、分支所有權、事件登錄、Pattern IR)
 8. 智慧收案 (Intake) 與提問優先序
 9. 模擬、除錯與 LSF Regression
 10. Loop Engineering(收斂偵測、預算、斷路器、遙測)
 11. 驗證智慧 (跨專案挖掘、信心校準、驗證策略建議、影子驗證)
 12. 覆蓋率結案、Waiver、Signoff 與 Golden Scenario
 13. 治理、Git 保護與供應鏈風險
 14. 共用知識中心與第三方元件治理 (SyoSil / ATB)
 15. Human Control Plane、Dashboard 與 GUI
 16. 完整 CLI 指令參考與端到端使用範例
-

1. 導論與核心架構總覽 (Graph / Blackboard / Memory / ReAct / Autonomous Inference)

DV Agent Harness L5 是一套以「證據優先」為最高原則的驗證流程自動化框架。整個系統的行為並非交由 LLM 自由發揮，而是由五個彼此獨立、各司其職的機制共同組成一套可稽核的執行骨架：**Graph**（工作流程權威）、**Blackboard**（當前驗證真相）、**Memory**（歷史工程知識）、**ReAct 內迴圈**（單一 stage 內的反思式重試）、以及 **Autonomous Inference**（Hypothesis → Evidence → Confidence → Gap → Next-Best-Action 的量化推理）。CLAUDE.md 的 Core Operating Rules 開宗明義寫著：「Graph is the global workflow authority. Blackboard stores current verification truth. Verification Memory stores historical verified engineering knowledge... Memory is prior knowledge, not current evidence.」本章即以這五者為主軸，說明它們在 dv_harness/ 原始碼中的實際落地方式，以及各自明確揭露的邊界。

Graph：全域工作流程權威

dv_harness/models.py 定義了 Stage（如 ENV_CHECK、DISCOVERY、DE_BASELINE_REPRODUCTION、VPLAN、IMPLEMENT、BUILD、VERIFY、REGRESSION、FAILURE_RECOVERY、SIGNOFF 等，涵蓋從 intake 到簽約的完整驗證生命週期）與 Status（NOT_STARTED、RUNNING、PASS、FAIL、PARTIAL、BLOCKED、RETRY、WAIT_USER、CLOSED、ACCEPTED_RISK）。HarnessState / StageState 是持久化在 state.json 中的執行狀態，包含 current_stage、overall_status、每個 stage 的 attempts（受 policy.max_stage_retries 限制的重試計數，範圍侷限在單一 graph node，一旦換 stage 便歸零）。

真正決定「下一步該去哪」的是 dv_harness/policy.py 的 graph_next(current, status, root)：它讀取 .dv-harness/graph/main_graph.json 的邊定義來路由，只有在沒有 graph 檔案或找不到對應邊時才退回到 next_stage() 這個線性的 ORDER 陣列。engine.py 的 DVHarness.run_stage() / loop() 是實際驅動者：每次呼叫 run_stage() 執行一個 stage 一次，讀出 ss["status"]，若是 FAIL 且重試已耗盡則呼叫 graph_next 走向該 stage 的 FAIL 邊；policy.can_signoff(state, cfg, blackboard) 則是 SIGNOFF 之前的硬性關卡——即便所有 gate 都通過，只要 Blackboard 上記錄的 qualified_conclusion 標示 is_qualified: false，或 require_second_pass_audit 未滿足，signoff 仍會被 BLOCKED，且沒有自動改道，必須交由人類決策。

值得注意的揭露事項：planner.py、react.py、router.py、multi_agent.py、skill_resolver.py 這幾個模組雖然存在且經過 2026-08-28 架構稽核確認「正確」，但從未被任何實際執行路徑 import 過；已刪除的 graph_runtime.py 同樣是「真實但未接線」的殘留。engine.py 選擇直接在 run_stage() 內重用這些底層模組（GraphDefinition、Blackboard、RouteResolver、PlanStore、MultiAgentOrchestrator、ReactRecorder、SkillResolver），而非整包導入那套從未接線的 GraphRuntime——這是一個「模組真實存在但曾經未被呼叫」的典型案列，讀者在查找某個機制是否「真正生效」時，務必以 engine.py 的實際 import 與呼叫點為準，而非模組本身是否存在。

Blackboard：當前驗證真相

dv_harness/blackboard.py 的 Blackboard 類別是一個以 topic 為單位、寫入 .dv-harness/blackboard/<topic>.json 的鍵值儲存區，write() 自 2026-09-04 起改為透過 storage.atomic_replace() 做 atomic replace（先寫暫存檔再 os.replace()），read() 對 Windows 上常見的 PermissionError 與部分損毀的 JSON 具備重試機制——但若整個重試預算耗盡仍讀不到合法內容，會直接拋出例外，而非悄悄回傳 default，因為「回報一個無法讀取的 topic 為『不存在』，等同於把『沒有紀錄驗證真相』偽裝成事實，這正是 Blackboard 絕不能犯的錯誤」。

Blackboard 上有若干具名的結構化 topic，例如 findings（open/closed 缺陷登記表，engine.py 的 _sync_findings_state() 會把計數鏡射回 HarnessState）、debug_loop_history（每一輪完整的 Fix→Push→Build→Verify 迴圈紀錄，用來回答「這個失敗已經跑過幾輪除錯」這種跨 stage 節點的問題，因為單一 stage 的 attempts 欄位無法回答這個問題）、以及 qualified_conclusion（RE_AUDIT / RCA_JOIN 通過 gate 驗證後，由 engine._score_root_cause_confidence() 寫入的「結論品質」記錄）。

CLAUDE.md 的「Blackboard Topics Written Outside the Graph」章節特別揭露：env_manifest、open_questions_decisions、connectivity_gates 這三個 topic 原本只能透過人工呼叫的 CLI/runner（dv-harness env-manifest generate、just connectivity-check）才會刷新，2026-09-04 之後 engine.run_stage() 才新增 _refresh_declared_subsystem_topics()，依照 graph node 自己宣告的 blackboard_read 欄位（而非寫死的 stage 清單）在進入 stage 前主動刷新這三者，讓純自動化的 loop() 也能產生它們。這是一個很好的範例，說明本專案如何持續把「原本只在人工路徑上才會發生的機制」逐步接上自動化路徑。

Memory：五層歷史工程知識

dv_harness/memory.py 定義了 MEMORY_LEVELS = ["working", "job", "project", "engineering", "organizational"] 五個層級，每筆紀錄寫入前都會先經過 _guard_record_before_write()——先做 memory_security.redact_record() 秘密偵測與遮蔽，再做 memory_artifact_policy.enforce_record_artifact_policy() 大型內容截斷，且順序刻意固定為「先遮蔽、後截斷」，避免截斷把一段跨行的 SSH 私鑰切斷而讓遮蔽規則失效。memory_router.py 的 route_and_store() 依照紀錄的 kind 決定該落在哪一層；晉升規則是嚴格單向的：Engineering → Organizational 只能經由 promote_to_organizational()，且必須同時滿足三個條件——質性上 CLOSED/VERIFIED、inference.score_confidence() 給出 HIGH、以及 confirmation_count >= ORGANIZATIONAL_MIN_CONFIRMATIONS（硬編碼為 2，意即需要第二次獨立的運行重新推导出同一個 root_cause，而非同一次運行被計數兩次）。CLAUDE.md 明白寫著：「Memory is prior knowledge, not current evidence... Never promote an unverified hypothesis straight to Engineering or Organizational Memory.」

Memory 與 Blackboard 的分野在此清楚體現：Memory 是「先前知識」，任何從 Memory 檢索到的候選假說在使用前都必須用當前的 RTL/VIP/sim.log/波形證據重新驗證，絕不可直接當作已確認的 root cause——這正是 Evidence Truth Rule 對 Engineering Memory Policy 的具體套用。

ReAct 內迴圈：react_loop.py

dv_harness/react_loop.py 的 InnerReactLoop 是一個存在於單一 stage 呼叫內部的 Reason-Act-Observe-Reflect-RePlan 迴圈，介於「adapter 呼叫產生 result.text」與「賦值 ss['status']」之間，並不取代 engine.loop() 的外層重試機制——loop() 依然每個 outer attempt 呼叫一次 run_stage()，並照舊讀取 ss["status"]。是否啟用受兩個條件同時控制：graph node 自身的 react 布林欄位（預設 True）與 cfg["policy"]["enable_inner_react_loop"]（全域開關，預設 True）；只要有一個為否，該 stage 就完全不進入這個模組，走向原本的 PARTIAL + replan_stage() 路徑。

`build_menu()` 產生的每一個 `MenuOption` (動作種類為 `RETRY_TARGETED` 、 `REQUEST_EVIDENCE` 、 `REROUTE` 、 `CONVERGE_TERMINATE` 、 `CONVERGE_NO_NEW_INFORMATION` 之一) 都必須源自真實的 `registry/graph/gate` 資料，絕非 LLM 自行編造；其中 `REROUTE` 選項只有在真的存在對應的 `graph.GraphDefinition` 節點時才會被提供，一個幻覺出來的目標節點會被直接捨棄。

Autonomous Inference：`inference.py` 的量化推理

`dv_harness/inference.py` 把過去只存在於 skill 散文中的 Hypothesis → Evidence → Confidence → Gap → Next-Best-Action 流程，變成真正的可執行 Python 運算。`score_confidence(independent_sources_count, evidence_refs_verified, counter_evidence_count, multi_agent_consensus_count)` 依公式 $base = \min(sources, 3) * 2 + (2 \text{ if } verified \text{ else } 0) - counter * 3$ (若多代理共識 ≥ 2 再加 2) 換算成 HIGH / MEDIUM / LOW 三級，並設有安全下限：只要存在未處理的反證 (`counter_evidence_count > 0`)，即使分數達標也不得回報 HIGH，會被強制降為 MEDIUM 並標記 `capped_by_counter_evidence`。`identify_gap()` 是一個純粹的集合差集運算 (required 減去 supplied，大小寫敏感、不做模糊比對)。`next_best_action(protocol, gaps, root, gap_action_catalog=None)` 則對照 `protocol_builder_registry.json` 給出具體下一步建議，2026-09-04 起新增的 `gap_action_catalog` 參數讓非 DV 領域的呼叫者 (例如 `capability_evolution.py`、`golden_flow_readiness.py`) 可以重用同一套「Gap → Action」引擎邏輯，而不必各自重寫一份。

CLAUDE.md 特別強調此模組刻意不與另外兩個外觀相似的分級機制共用詞彙：`connectivity.classify_bind_tier()` (T1~T4，依證據「種類」而非「數量」排序，一個已存在的 bind 天生就贏過任何量的結構匹配) 與 `question_queue.classify_tier()` (SELF_RESOLVE/SAFE_TO_ASSUME/CANNOT_ASSUME，Tier 3 是「必須由人決定」的升級路徑，而非低分)。`score_confidence()` 計的是證據「數量」的加法排序，兩者語義根本不同，硬套會讓「一個已存在的 bind」與「結構性匹配」得出相同的 MEDIUM 分數，抹去了原本刻意保留的優先序差異——這是本專案「刻意不整合看似相似機制」的一個具代表性案例。

三層記憶查詢與人機邊界的收斂

除了上述五個核心機制，`dv_harness/question_queue.py` 的三級「詢問人類」協定 (SELF_RESOLVE / SAFE_TO_ASSUME / CANNOT_ASSUME) 與 `dv_harness/source_authority.py` 的九層「來源權威順序」(1. 實際模擬結果最高 9. VIP 文件最低) 共同構成了本系統遇到證據衝突或需要人類決策時的收斂路徑：`source_authority.resolve_conflict()` 只回答「兩者互相矛盾時該採用哪個值」，同層級衝突 (UNDECIDABLE_SAME_AUTHORITY) 或任何解出的衝突都會經由 `escalate_conflict()` 送入 question queue 成為一個 Tier-3 阻斷式問題，而非由系統自行仲裁孰是孰非。這正呼應了 Core Operating Rules 的最後一條精神：「Human Override is always valid」——上述所有機制 (Graph 路由、Blackboard 記錄、Memory 晉升、ReAct 動作選單、Inference 信心分級) 最終都設計為在遇到無法自動判定的節點時，明確地把決定權交還給人類，而不是用一個看似合理的猜測掩蓋這個邊界。

2. VIP/UVM 驗證環境生成 (Subsystem-Mode / System-Level Mode, VIP 綁定與結構檢查)

本章說明 DV Agent Harness L5 如何真正產生一份 UVM 驗證環境:從「先決定 SUBSYSTEM_MODE / SYSTEM_LEVEL_MODE、再產生」的入口分派,到產生後立刻對 產出的 SV 原始碼跑結構性 lint、對每一次 VIP API 呼叫做可證性檢查,以及 bind 位置與訊號連通性的四層規劃。所有描述均直接對照

dv_harness/uvm_generator/create_environment.py、protocol_env_generator.py、protocol_model_layer.py、uvm_structural_lint.py、connectivity.py、phy_boundary.py、vip_symbol_index.py、vip_api_card.py、vip_capability_extraction.py、protocol_capability.py 與 env_manifest.py 的原始碼與其於 CLAUDE.md 中對應章節的揭露文字。

2.1 CREATE ENVIRONMENT 之前:SUBSYSTEM_MODE / SYSTEM_LEVEL_MODE 分派

CLAUDE.md 的「Environment Generation Mode」明文規定:在下達 CREATE ENVIRONMENT 之前,必須先選擇 SUBSYSTEM_MODE(建立單一 protocol/subsystem 環境)或 SYSTEM_LEVEL_MODE(挑選/重用已完成的 subsystem 環境並組成 Full-SoC/System-Level 環境);若 SYSTEM_LEVEL_MODE 所需的某個 subsystem 尚未存在,規則要求「先用 SUBSYSTEM_MODE 建好它,再回頭組成」。

這個決策本來只是「算出來但沒人接線」的殘局:environment_mode_router.resolve_environment_mode() 每個 stage 都會被 engine.py 的 begin_stage() 呼叫並算出結果,但答案只落在 route_info["environment_mode_decision"]——也就是進了 prompt 與 ReAct 紀錄,從沒有任何程式碼真正依此分支。真正的產生入口 tools/generate_protocol_uvm_environment.py (所有十份 .claude/skills/PROTOCOL_BUILDERS/*/SKILL.md 都呼叫這一支腳本)過去無條件呼叫 ProtocolEnvGenerator,於是一個貨真價實的雙 subsystem 請求會默默只產出一個 subsystem 環境。

dv_harness/uvm_generator/create_environment.py 的 create_environment(root, request, out_dir=None) 就是補上的分派點,實際呼叫方式如下:

```
python tools/generate_protocol_uvm_environment.py \
--manifest manifest.json --out generated/usb_env --root .
```

其內部邏輯:

- 從 request 算出 _requested_subsystems()——若 manifest 明確帶 requested_subsystems 就用那份清單(SYSTEM_LEVEL_MODE 的組成形狀);否則單一 protocol 欄位本身就是唯一請求的 subsystem,這正是既有 PROTOCOL_BUILDERS manifest 不必改就能經由真正的 router 解析為 SUBSYSTEM_MODE、而不是繞過決策的原因。
- 呼叫 resolve_environment_mode(),並把 .dv-harness/soc-composer/subsystem_environment_registry.json 讀出的 read_registered_subsystem_entries() 結果一併帶入,決定 SUBSYSTEM_MODE 或 SYSTEM_LEVEL_MODE。
- SUBSYSTEM_MODE 原封不動呼叫 ProtocolEnvGenerator(out).generate(m)——單一 protocol manifest 產出的檔案與這個分派點加入前逐位元組相同。
- SYSTEM_LEVEL_MODE 呼叫 compose_soc_environment(),但只能組成「登記在真實 registry」的 subsystem——這份 registry 只有 engine.py 的 _persist_subsystem_registry_entry() 在真正 SIGNOFF PASS、且 subsystem_environment_registration_gate 驗證通過後才會寫入。呼叫端無法透過這條路徑把未登記的 subsystem 塞進組成。
- 若請求的 subsystem 有任何一個尚未登記,直接丟出 SubsystemModeRequiredError,並在 exit code 2 附上缺哪些 subsystem(SUBSYSTEM_MODE_REQUIRED_FIRST)——而不是「先組出登記過的子集」;若請求根本沒有指名任何 subsystem,則丟出 EnvironmentModeUnresolvedError (exit code 3),對應 environment_mode_policy.json 的 mode_must_be_explicit_before_generation 設定,拒絕默默套用預設值。

揭露的邊界:compose_soc_environment() 的 cross_subsystem_scenarios()/end_to_end_scoreboard()/system_coverage() 仍然刻意丟出 NotImplementedError——這些屬於 protocol-行為內容,依「No Golden-Reference Content Mining」原則必須來自真正的跨 subsystem topology(address/interrupt/DMA map),而目前 registry entry 只帶身分/qualification metadata,尚未有這份描述可用。此外,這個機制本身在此 harness repo 自己的專案裡從未真正觸發過——subsystem_environment_registry.json 目前合法地是空的,因為這個 repo 沒有自己的多 subsystem 專案;文件明言:機制已對「真實專案供應真實已登記 subsystem」證明可行,但沒有為了「讓它顯得曾經觸發過」而灌入虛構的 registry 紀錄。

2.2 SUBSYSTEM_MODE 的實際產出:目錄佈局與五個 Protocol Model

ProtocolEnvGenerator (dv_harness/uvm_generator/protocol_env_generator.py) 是取代 generator.py 舊版扁平輸出的目錄化產生器,把 UVMEnvironmentGenerator 既有的逐檔 emit 方法(config/vseq/base_vseq/ scoreboard/coverage/assertions/env/base_test/smoke_test/tb_top)不變地重用,再依 LAYOUT 表分配到符合真實 catalogued USB_UVM_Handoff 套件的樹狀結構:

- tb/env :pkg、config、scoreboard、coverage、assertions、env
- tb/seq :virtual_sequencer、base_vseq
- tb/tests :base_test、smoke_test
- tb/top :tb_top.sv
- filelist/dv_uvm_files.f
- tb/agents、tb/patterns/common :目前只建立空目錄(尚無逐 class 的 agent emitter,是尚未排入本階段的獨立工作)

generate(self, m: dict) 會依 manifest 的 protocol 欄位決定 sv_id 命名,並用 m.get("smoke_tests") 產生一或多支 smoke test。

真正讓「protocol 自己的行為模型」接上主產生路徑的是 dv_harness/uvm_generator/protocol_model_layer.py。在此之前,pcie_ltssm_generator.py、mipi_dphy_generator.py、canfd_arbitration_generator.py、emmc_cmdq_generator.py、amba_fabric_generator.py 這五支真實、有單元測試的 protocol model,唯一呼叫者都只是各自的 tools/generate_*.py 獨立腳本——create_environment.py 一個都沒 import,於是一個 protocol: "PCIe" 的 manifest 和一個 protocol: "Ethernet" 的 manifest 產出完全相同的 protocol-agnostic 骨架。protocol_model_layer.py 補上這條線,規則是:

- 哪個模組實作哪個 protocol,一律來自 protocol_capability.PROTOCOL_CAPABILITIES (唯一、已被 --check 做 drift-check 的答案),不在此另建第二張表。

- 如何呼叫,是該 entry 自己的 `generator_class`,經 `protocol_capability.resolve_generator_class()` 透過 import 系統解析。
- 餵給它什麼,是 manifest 的 `protocol_model_topology` ——PCIe 的 `lane_width/gen_speed/role`、D-PHY 的 `role/lane_count`、CAN-FD 的 `id_format/node_ids`、AMBA 的 `masters/slaves/addr_width` 等真實 DUT 事實,由各模型自己的型別驗證器判斷,此模組不重新驗證也不代填預設值。

三個刻意設計的性質:

1. 沒有模型、也沒有 topology 的 protocol,產出的 SV 依然逐位元組不變。USB 是唯一 DUT_PROVEN 的 protocol,設計上本來就沒有自己的 protocol-model 模組,走 NO_PROTOCOL_MODEL 分支,輸出完全不受影響。
2. 缺 topology 絕不用猜的——`lane_width/gen_speed/role` 是特定 DUT 的事實,為了讓 layer 生效而發明它們正是 CLAUDE.md Evidence Truth Rule 禁止的事。一個沒帶 `protocol_model_topology` 的 PCIe manifest 會在 自己的 `environment_manifest.json` 裡記下 `PROTOCOL_MODEL_TOPOLOGY_NOT_SUPPLIED`,並點名本應執行卻沒執行的 模組。
3. 模型會真正接進主環境,而不只是丟到 `protocol_model/` 子目錄:當 模型公開狀態機圖(目前只有 PCIe 的 `LTSSM_TRANSITIONS`)且 manifest 指名對應的真實 DUT 訊號時,會透過既有 `state_machine_checks` DSL 編譯成真正的 transition-legality SVA,寫入 `tb/env/<p>_assertions.sv`,並把模型的 package prepend 進環境 filelist 使其先於引用它的檔案編譯。

Scope 邊界(文件明言,避免被誤解為更多):這一段只解決「宣稱」與「接線」問題,不解決「能力」問題——沒有任何非 USB 的 protocol 因此被真正 DUT 證明過,layered 輸出從未真正被編譯或 bind 過,建議的第一個非 USB 試點 仍是 PCIe。

2.3 Per-Protocol Capability:兩個問題,不能只用一個標籤回答

`dv_harness/protocol_capability.py` 修正的是「這個 harness 能替哪些 protocol 產生」與「這個 harness 真的對哪些 protocol 證明過」被錯誤地合併成單一欄位的問題——過去 `.dv-harness/qualification/protocol_capability_registry.json` 對全部 11 個 protocol 都寫 "status": "REAL_GENERATION_READY" 與 "generation_capability": "REAL_CODE_GENERATOR_AVAILABLE",但 Ethernet、SD_SDIO、eDP、UCle 背後根本沒有 protocol-specific 的 Python 模組,只有每個 protocol 都有扁的扁平骨架,外加沒有任何程式碼讀取的 `builder_profile.json`。這個誇大不是無害的文字——`dashboard.py` 的 `_protocol_registry()` 會把 registry 直接渲染成專案的真實 protocol 就緒度, `_qualification_tier_reached()` 也由此推導 Qualification-Tiers 卡片。

修正後拆成三個各自可查核的事實:

- generic skeleton——每個 protocol 都有,來自 `uvm_generator/protocol_env_generator.py`,這半句原本就是真的,維持不變。
- protocol model generator——共五支,涵蓋八個 protocol key: `pcie_ltssm_generator` (PCIe)、`mipi_dphy_generator` (MIPI_CSI2/MIPI_DSI,只模擬 D-PHY 電氣層,兩個 packet 層都未建模)、`canfd_arbitration_generator` (CAN_FD)、`amba_fabric_generator` (AMBA4,唯一被 production-adjacent 程式碼——`address_map_verifier.py`——import 的一支)、`emmc_cmdq_generator` (eMMC/SD_SDIO,只模擬 protocol-agnostic 的 tag lifecycle)。每一筆都要真正能透過 import 系統解析、且對應的獨立工具檔案真的存在於磁碟上才會回報,否則一律回報 NONE。每個部分模型都在 `does_not_model` 裡點名自己沒建模的層(例如 AMBA 的 `ace_lite_coherency` / `axi_stream`,PCIe 的 `t1p_layer` / `config_space`)。
- DUT proof——`dut_proof` 路徑必須真的存在,目前只有 USB 有,USB 因此是唯一 DUT_PROVEN 的 protocol。

`capability_status` (`GENERIC_SKELETON_ONLY` / `PROTOCOL_MODEL_PARTIAL` / `PROTOCOL_MODEL_COMPLETE` / `DUT_PROVEN`,代表「產生程式碼存在什麼」)刻意與 `qualification.py` 的 8 階 ladder(代表「一個已產生的環境被證明到多深」)使用完全不共享 token 的獨立詞彙——PCIe 擁有全 repo 最深的 protocol model,卻仍停在 `BUILDER_AVAILABLE`,正是一個欄位裝不下兩件事的例子。

Registry 是生成的,不是手改的:

```
python -m dv_harness.protocol_capability --sync # 依程式碼重寫 registry
python -m dv_harness.protocol_capability --check # 有落差就 exit 2
```

`tools/universal_protocol/protocol_status.py` 會呼叫 `--check`,一旦偵測到 落差就直接拒絕印出任何結果,而不是印出一個程式碼撐不住的就緒度。

2.4 產生後立即執行:UVM Structural Lint

過去這個 repo 從沒有任何機制在「產生完的 UVM 原始碼」上做過結構檢查——`verible_parser.py` 只解析 RTL, `bind_verification_lint.py` 檢查的是 elaboration/simulation 的報表文字(log lines),不是 UVM 原始碼本身。一個產生出來的環境第一個真正的結構回饋就是 VCS compile,正是 CLAUDE.md 220 節「Structural lint runs before expensive simulation」想要搶在前面的那個昂貴路徑。

`dv_harness/uvm_structural_lint.py` 補上這一段。它重用既有的 verible 前端 (`verible_parser.run_export_json()` 及其現已公開的 tree-walk 輔助函式),不是第二支手刻的 SystemVerilog parser;每個呼叫點的邊界一律來自 verible 的語法樹,只有已被切出的 callee 識別字串(如 `uvm_config_db#(T)::set`、`phase.raise_objection`、`env.mon.ap.connect`)才會被當字串比對。實際檢查五類、也是「純解析、不需 elaboration 就能判斷」的子集:

- `FACTORY_REGISTRATION_*`:每個繼承鏈終於 UVM component/object root 的 class,是否帶對應的 `uvm_component_utils` / `uvm_object_utils` 家族巨集且命名自己。
- `PHASE_METHOD_*`:以真實 UVM phase 命名的方法是否用該 phase 的真實簽章(void function 還是 task,恰好一個 `uvm_phase` 參數)。
- `CONFIG_DB_*`:每個可靜態解析的 `uvm_config_db` field key, get 過的是否有對應 `set`,反之亦然。
- `TLM_PORT_NEVER_CONNECTED`:宣告的 TLM *_port 成員是否在環境中某處 `.connect()` 過。
- `OBJECTION_*`:每個方法內 `raise_objection` / `drop_objection` 是否平衡。

執行位置: `create_environment()` ——唯一真正的 CREATE ENVIRONMENT 入口——會對剛產生的環境跑這個 lint,把結果同時寫回呼叫端回傳值的 `structural_lint` 欄位,以及 `<out_dir>/uvm_structural_lint.json`。預設 非阻斷(不能因為新加了一個檢查就讓原本能跑的產生流程變成硬性失敗); manifest 若設 "strict_structural_lint": true,ERROR 等級的發現會丟出 `StructuralLintFailedError`, `tools/generate_protocol_uvm_environment.py` 以 exit code 5 呈現。也可以獨立呼叫:

```
dv-harness uvm-lint --env-dir generated/usb_env [--json] [--fail-on-error]
# 或針對個別檔案: dv-harness uvm-lint --file a.sv --file b.sv
```

揭露的邊界:這是 parser,不是 elaborator——`generate`/`ifdef` 條件不會被評估,參數不會被解析,一個繼承自這次解析從未見過的 base(例如 VIP class `svt_usb_agent`)會被記為 UNCLASSIFIED 且不觸發警告,因為未知的 base 無法證明缺少註冊。兩個 `config_db` 檢查一律是 WARNING 而非 ERROR,因為「找不到對應的另一半」這件事本身無法被這層分析證明(可能藏在 VIP 程式碼、專案自己的 top test,或執行期組出的 key 裡)。`.svh` 無法獨立解析時視為 WARNING(`bind_mechanism_generator.py` 本來就會合法產生 top-module-scope 的 `dv_uvm_hook.svh`),但`.sv` 無法解析則視為 ERROR。`verible` 無法執行時狀態是 NOT_AVAILABLE 並附真實原因,絕不是 PASS。220 節列出的其餘七項 (analysis-port 語意、sequencer/driver 連接、virtual interface binding、package/import 相依、重複定義、重複的 active driver、非法階層假設)尚未 實作,其中多項需要 elaboration-time 才拿得到的真相。

2.5 VIP API Card:每一次 VIP API 呼叫都要能被證明

Spec 第 187 節的 VIP API 流程以「If API cannot be proven: UNKNOWN / BLOCKED」收尾,但在 `vip_api_card.py` 出現之前,這條規則只存在於給 LLM 看的 prompt 文字裡(`.claude/skills/CORE/vip-scenario-branch/SKILL.md` 寫著「禁止憑猜測 invent VIP API/class/sequence」);全 repo 搜尋

`VIPApiCard / validate_vip_api_usage / UNPROVABLE` 找不到任何可執行程式碼——一段引用虛構方法 `svt_usb_agent.reconfigure()` 的產生序列,和引用真實方法的序列,離開產生器時完全一樣,第一個會發現問題的還是 VCS compile。

`dv_harness/vip_api_card.py` 重用 `vip_symbol_index.py` (見下節),而不是重建第二支 SystemVerilog 掃描器。它對每一次 VIP API 引用產生一筆 **VIPApiCard** 記錄:引用了哪個 VIP class/method、在產生程式碼的哪個位置引用、index 解析到的真實 `file:line` (或找不到)、繼承鏈上真正宣告它的 class,以及一個狀態。四個狀態中 **BLOCKED** 刻意做得很窄:必須同時滿足 (1) 接收者宣告的型別是 index 真的收錄的 class、(2) 該成員在整條已索引的繼承鏈中都不存在、(3) 該鏈中每個未被索引的 base 都是宣告過的 base-library class(預設 `uvm*`,即世界必須「封閉」)、(4) 該成員不在文件化的 SystemVerilog/UVM base-library 方法清單中。若繼承鏈離開 index、進入未知的非 library base,則回報 UNPROVABLE(對應 187 節的 UNKNOWN)——不是通過,也絕不會被靜默丟棄。

執行位置:manifest 只要指名 `vip_symbol_index: <path>`, `create_environment()` 就會對剛產生的程式碼做驗證,結果寫入回傳值的 `vip_api_validation` 與 `<out_dir>/vip_api_cards.json`;預設非阻斷, `"strict_vip_api": true` 讓 BLOCKED 引用丟出 `VipApiUnprovableError`。也可獨立執行:

```
dv-harness vip-api-check --source generated/usb_env/tb --index vip_symbol_index.json \
[--out-dir generated/usb_env] [--strict-unprovable] [--json]
# 或: python -m dv_harness.vip_api_card --source <dir> --index <index.json>
# (exit 0 PROVEN, 1 BLOCKED, 2 NOT_AVAILABLE, 3 UNPROVABLE)
```

揭露的邊界:只判斷方法呼叫、class 型別引用與 `Class::` 範圍引用, `Class::MEMBER` 只判斷 class 那一半(不建模 enum 常數/參數/typedef);純 property 存取 (`cfg.some_field`)完全不判斷,因為 index 只收錄受限的一組資料型別,欄位不在 index 裡不代表欄位不存在。接收者型別無法解析的呼叫不會產生任何卡片。唯一的假陽性風險是 `BASE_LIBRARY_METHODS` allowlist 不完整——但這種缺漏只會讓判斷「降級」,只可能產生假 BLOCKED,絕不會產生假 PROVEN,因此接入產生路徑時預設非阻斷。Index 必須是「環境實際綁定的那支 VIP」的 index,拿別支 VIP 的 index 驗證只會把每個真呼叫都判為 unprovable——因此這個接線是 manifest 明確 opt-in,不是自動探索的預設值。它只回報,不做任何決定:不觸發 build、job 或 approval。

2.6 VIP Symbol Index / VIP Capability Extraction:VIP 原始碼的導覽層

`dv_harness/vip_symbol_index.py` 是 VIP 原始碼的符號索引器,也是 `vip_ref/<protocol>.md` 濃縮文件的產生器。CLAUDE.md 的 context-budget 政策明言此 repo「沒有 VIP-source distiller」,VIP 原始碼(row 13)本應「不載入 context,只做符號索引(class/method 名稱 + file:line)」。這個模組唯一定義性的限制是:只讀取宣告,絕不保留實作內容——class 名稱、base class、method 簽章、config field 名稱與各自的 file:line 會被保留,method 內部的敘述與運算式會被比對後立即丟棄, `assert_no_bodies_retained()` 讓這個承諾成為一項可被測試檢查的性質。

刻意選擇 regex 而非 verible parser 的原因是:production VIP 樹經常包含 加密區塊 (`svp / vp` protected regions)或程式產生的檔案,verible 遇到會直接失敗;逐行宣告掃描能優雅降級——看不懂的檔案只是貢獻較少符號,而不是拋例外。代價是誠實而有界的:這份索引是**導覽工具**,不是語意模型,絕不能用來對 VIP 行為下正確性判斷——需要下判斷時,永遠回到針對該 `file:line` 的一次目標式閱讀。

在此之上, `dv_harness/vip_capability_extraction.py` 把索引出的宣告進一步分類成產生器/落差分析真正需要的五種能力形狀:VIPConfigIR、VIPTransactionIR、VIPScenarioPatternIR、VIPCheckerCapabilityIR、VIPCoverageCapabilityIR,並為每一筆標上 5 級可查核的信心標籤——

PROJECT_PROVEN (在專案原始碼真的被引用)、VIP_DOCUMENTED (在 `vip_user_guide_distill.py` 濃縮出的使用手冊章節裡真的出現)、VIP_EXAMPLE_MATCHED (在 VIP 自帶的 Examples/ 原始碼中真的被引用)、INFERRED_FROM_NAMING (僅命名/繼承啟發式判斷,沒有外部佐證時的誠實下限)、UNKNOWN (命名與繼承兩種啟發式互相矛盾,分類被保留而不是用猜的)。命令列用法:

```
python -m dv_harness.vip_capability_extraction --index vip_symbol_index.json \
[--project-source <dir>] [--example-source <dir>] \
[--user-guide-reference-md <path>] [--out-dir <dir>] [--json]
```

2.7 Bind-Location Rules:PHY 邊界先決定 mount 層,再談 hierarchy path

CLAUDE.md 的「Bind-Location Rules」對每一個 bind 陳述式定下五條硬性規則(不是建議,也不是逐案判斷),分別由 `connectivity.py` (4 層 bind-confidence 分類、既有 bind 的 grep、3-gate connectivity pipeline)、`phy_boundary.py` (mount 層決策,Rule 5)與

`uvm_generator/bind_mechanism_generator.py` (證據門控的 bind/hook-skeleton 產生)共同支撐:

- 綁 bare module name 會套用到該 module 的**每一個** instance,在 IP-level(單一 DUT instance)通常沒問題,但在 SoC-level 若該 module 被實例化超過一次就會默默全部綁上——SoC-level 一律必須用完整 instance path(`chip.core.subsys0.usb0`, 不能是 `usb3_subsystem`)。
- 每個 bind 陳述式都集中放在 `tb/` 下的一份 `*_bind.sv` 檔案裡;RTL 檔案唯讀,絕不可直接在 RTL 裡插入 bind。
- clock/reset 必須透過 bind instance 自己的 port list 明確傳入,不可用跨層級的 hierarchical reference(XMR)去抓——後者在 gate-level netlist 或任何 wrapper 替換後都會斷掉。
- bind target path 裡不可含 generate-block 或 for-loop 構造,必須展開成明確的字面索引(`chip.core.phy_array[0].u_phy`、`[1].u_phy`.....),不接受任何 wildcard/loop-based 目標。
- PHY model 先決定可以 mount 的「層」,之後才問「層裡的哪個 path」。這是排序規則:先辨認出熟悉的 module/signal 名稱再回頭問中間有沒有 PHY,是相反的順序,而且不是無害的順序問題——一個綁在 line-rate 序列通道上的 protocol VIP monitor,沒有 PHY model 就解不出任何東西,卻能通

過 Gate 1(elaborates)、Gate 2(clock 有 toggle、reset 有 release),只在 Gate 3 才會現形為一個「安靜的」monitor——這是整條 pipeline 裡發現 mount 層選錯代價最高的位置。這條規則源自一次真實糾正:agent 曾以為 USB3 的 PIPE4 介面取代了 PHY 而整顆停用 PHY instance,被使用者糾正「USB3 也是要經過 PHY,不是 PIPE 介面」。

Rule 5 已在程式碼中硬性強制(2026-09-04): phy_boundary.py 從 env_manifest.py 產生的同一份 verible-parsed port table 推導序列/並列邊界, bind_mechanism_generator.assert_phy_boundary_decided_first() 會在 validate_bind_entries() / emit_bind_sv() 內、先於 tier gate 執行這項決策——邊界若判為 SERIAL-only 或 UNDECIDABLE,直接丟出 PhyBoundaryValidationError,不寫入任何檔案。實際呼叫:

```
python tools/generate_bind_mechanism.py --phy-boundary phy_boundary.json \
--require-phy-boundary ...
```

揭露的殘留: require_phy_boundary 預設為 False——完全沒有提供邊界文件的呼叫端仍然會產生輸出,這是刻意的(IP-level 的 DUT 若根本沒有 PHY 子區塊,就真的沒有邊界可判斷),意即 Rule 5 只在「邊界確實在範圍內且已知」時才硬性生效,並非普遍強制。要強制契約請加 --require-phy-boundary。

Rules 1–4(路徑規則)目前仍靠人工 review 把關,尚未有獨立 lint gate 硬性擋下; connectivity.py 的 T1–T4 bind-confidence 分類器與 grep_existing_binds() /matrix 輸出會讓邊規在 connectivity matrix 的 bind_target 欄位中直接現形供人 review,但不會自動擋下產生。

與 Rule 5「同樣硬性擋下」但獨立於路徑規則的,是 **bind-CONFIDENCE tier** 本身: validate_bind_entries() 在 Rule 5 之後、寫入任何檔案之前呼叫 connectivity.enforce_bind_tier_policy(),對以下情形丟出硬性 BindTierError:

- **T4**(T4_BIND_MUST_GO_TO_QUESTION_QUEUE)——永遠不可產生,只能經 build_t4_question_queue_entry() 進入 question queue。
- **T3 且沒有真正人工確認**(T3_BIND_REQUIRES_HUMAN_CONFIRMATION)——entry 必須帶 human_confirmation: {source, confirmed_by, basis},其 source 必須是 question queue 的 HUMAN_DECISION_SOURCE。
- 無法辨識的 tier 字串,以及在 --require-tier / require_tier=True 下完全沒有 tier 的 entry。

揭露的殘留: require_tier 預設為 False,一筆完全沒帶 tier 的 entry 依然可以產生,維持既有 3 欄位 entry 契約(target_instance / ports / reason)不被破壞;真正硬性擋下的是危險情境——已被 pipeline 判定為 unconfirmed 或 undecidable 的 entry 卻仍被產生。要強制契約請加 --require-tier。

connectivity.py 的三個機器 gate(elaboration / static zero-time connectivity / transaction activity)是每次 IP_UVM_DV_Gen build 都必須真正「跑過」的檢查點,而不只是存在即可——一個 build 從第一次成功 compile/elaboration 起就必須跑過 Gate 1(run_gate1_elaboration_check()) 與 Gate 2(evaluate_zero_time_connectivity() / run_gate2_against_live_simv()),誠實回報 NOT_AVAILABLE(環境中沒有 slang / vcs /trace)也算滿足這個檢查點,但從不呼叫兩者才是不滿足。Gate 3(transaction activity)在真實 pattern 完成前無法 PASS/FAIL,必須明確追蹤為 PENDING(evaluate_transaction_activity_status (pattern_completed=False)), GateStatus 這個 enum 專門加了 NOT_YET_RUN / PENDING,避免這個狀態被和 FAILED 或「不適用」混淆。

2.8 這一整套檢查如何在產生流程中串起來

create_environment() 回傳值的形狀,反映了本章描述的每一段檢查其實都在同一次呼叫裡真的跑過: environment_mode、完整 router_decision、out_dir、generated_files;SUBSYSTEM_MODE 額外帶 protocol_model / protocol_model_files,SYSTEM_LEVEL_MODE 額外帶 composed_subsystems;兩種模式都一律帶 structural_lint(產生後立即對剛產生的 UVM 碼跑 uvm_structural_lint)與 vip_api_validation(對 manifest 指名的 VIP index 驗證每一次 API 引用)。換句話說,「產生環境」這個動作本身,已經把 mode 分派、protocol model 分層、結構 lint、VIP API 可證性檢查串成同一條不可繞過的路徑,而 bind-location 與 bind-confidence 的把關則發生在 bind_mechanism_generator.py 產生 *_bind.sv 的路徑上,兩條路徑各自有各自明確揭露、尚未硬性擋下的殘留範圍。

3. AMBA M x N 匯流排拓模與交易分析 (Fabric Discovery / Port Registry / Transaction / Route-Transform)

3.1 為什麼是八個模組而非一個

DV Agent Harness L5 的 AMBA 家族刻意拆成多個小模組，而不是塞進既有的 `connectivity.py`。 `connectivity.py` 負責「單一模組邊界」層級的分類——AMBA-4 的協定判定 (`classify_amba_protocol()`)、AMBA-5 的雙視角色判定 (`determine_fabric_interface_roles()`)、以及 T1-T4 的 bind 信心分級 (`classify_bind_tier()`)。而 AMBA-7 之後要回答的是結構性完全不同的問題：「這個 fabric port 的另一端，跨過幾層 wrapper 與一個 arbiter 之後，究竟接到誰？」這需要一張 port-level 的連通性圖 (graph) 與圖上的走訪 (traversal)，而不是單點判斷。

因此本章介紹的八個模組全部是「組合 (compose) 既有原語，而非重造」： `amba_fabric_discovery.py` 每一跳都重新呼叫 `classify_amba_protocol()` (這正是偵測到 bridge crossing 的方式)、每一個候選 bind 都重新跑 `classify_bind_tier()`、且不建立第二套 AMBA signal 詞彙表——一律沿用 `connectivity.amba_signal_tokens()` / `ALL_AMBA_SIGNAL_NAMES`。這個「唯一事實來源」原則會一路貫穿到 port registry、fabric analysis、transaction IR 與 route/transform predictor。

3.2 Fabric Discovery：把每個 fabric port 追到底 (`dv_harness/amba_fabric_discovery.py`)

用途。這是 AMBA-7 至 AMBA-14 的實作：從 verible 解析出的 RTL 模組/實例/連續指定 (`ModuleDef` / `InstanceDef` / `AssignEdge`) 建出一張扁平的階層式網表 (`FabricNetlist`，由 `build_fabric_netlist(parse_results, top_module)` 建立)，再對每一個 fabric port 執行 `trace_fabric_port()`，往「遠離 fabric」的方向走，穿過透明 wrapper (因為模組邊界的 port 與外部接線在 SystemVerilog 語意上就是同一個等電位網，wrapper 本身不會截斷這張網)，並在恰好十種可回報狀態之一終止 (`TraceTerminationStatus`)： `SOURCE_FOUND` / `DESTINATION_FOUND` / `MULTIPLE_SOURCE` / `MULTIPLE_DESTINATION` / `PROTOCOL_BRIDGE_FOUND` / `INTERNAL_ONLY` / `TRACE_BLOCKED` / `SOURCE_NOT_FOUND` / `DESTINATION_NOT_FOUND` / `AMBIGUOUS`。其中四種 (`TRACE_BLOCKED`、`SOURCE_NOT_FOUND`、`DESTINATION_NOT_FOUND`、`AMBIGUOUS`) 屬於「懸而未決」，`assert_unresolved_states_explained()` 會強制每一筆都必須附上非空的 `reason` 與 `missing_evidence`，不允許留白讓審查者自己猜。

如何被使用。模組沒有 `dv-harness` CLI verb，是給生成器/agent 呼叫的函式庫，典型流程：

```
from dv_harness.amba_fabric_discovery import (
    build_fabric_netlist, trace_all_fabric_ports,
    build_fabric_vip_bind_matrix, build_vip_instance_plan,
)

netlist = build_fabric_netlist(parse_results, top_module="chip_top")
traces = trace_all_fabric_ports(netlist, fabric_path=("chip_top", "u_fabric"))
matrix = build_fabric_vip_bind_matrix(netlist, traces) # AMBA-16 matrix
plan = build_vip_instance_plan(netlist, traces, matrix) # AMBA-20 VIP 實例規劃
```

保證與邊界。每個候選 bind 都會依走訪距離分到 P1 (真正的 IP/AMBA 邊界) 到 P4 (bus fabric port) 四個優先序，而且結構上保證 P1-P3 絕不會落在協定轉換之後——因為 trace 一旦跨過協定改變就終止 (`StructuralRole.PROTOCOL_BRIDGE`)，不是事後加規則檢查。這個模組只做「規劃 (planning)」，不做「產生 (emission)」： `assert_no_bind_statement()` 會對本模組自己算繪的報告文字執行斷言，確保裡面沒有出現任何真正的 SystemVerilog bind 陳述句——真正的 emission 留給 `uvm_generator/bind_mechanism_generator.py`，且必須經過強制人工審查 (人工確認的 gate) 才能落地。子角色 (例如 `REGISTER_SLICE_OR_PIPELINE`、`WIDTH_CONVERTER`) 只是根據實例名稱給出的 T3 命名啟發式提示，`requires_human_confirmation=True`，絕不會改變結構角色或 tier。

3.3 Port Registry：一列一個 fabric port，十九個欄位全部是「聯集」而非「重算」 (`dv_harness/amba_port_registry.py`)

用途。這是 AMBA-22 的 `AMBA_PORT_REGISTRY`： `AMBA_PORT_REGISTRY_FIELDS` 恰好十九欄—— `port_id`、`fabric_port`、`protocol`、`fabric_role`、`endpoint_role`、`endpoint_hierarchy`、`vip_bind_hierarchy`、`vip_mode`、`clock`、`reset`、`address_width`、`data_width`、`id_width`、`user_widths`、`scoreboard_channel`、`trace_status`、`readiness`、`confidence`、`source_evidence`。這些欄位分別讀自 AMBA-16 的 bind matrix (`build_fabric_vip_bind_matrix()`)、AMBA-15 的 bind 位置驗證 (`validate_vip_bind_location()`)、AMBA-20 的 VIP 規劃 (`build_vip_instance_plan()`)、以及 AMBA-21 的 scoreboard channel 對應。每一欄都是從已生成的產物「讀出」，絕不第二次重新推導，這正是「registry 不會與人已審過的 matrix/checklist/VIP 規劃互相矛盾」的原因所在。

如何被使用：

```
from dv_harness.amba_port_registry import (
    build_amba_port_registry, project_to_fabric_topology,
)

rows = build_amba_port_registry(netlist, traces, plan=plan, ingress_mapping=None)
topology_doc = project_to_fabric_topology(rows) # masters/slaves/scoreboard_matrix/address_map
```

`project_to_fabric_topology()` 把 registry 往下投影成 `tools/verification_flow/fabric_topology_completeness_gate.py` 既有驗證的那個更小的頂層 shape (`masters/slaves/scoreboard_matrix/address_map`)，刻意只往一個方向丟失資訊——registry 本身是那份文件的真超集 (多了每個 port 的協定、時脈、reset、位寬、readiness、confidence 與證據)，不會引入第二套拓模 schema。

邊界。`vip_bind_hierarchy` 是提議給人審核的位置字串，不是已生成的 bind；同樣受 `assert_no_bind_statement()` 保護。`assert_no_generic_port_ids()` 會拒絕沒有專案理由就用 `master0/slave1` 這種通用命名的 port id——AMBA-22 要求 port id 必須來自真實的階層路徑，才能追溯回 RTL。

3.4 三個 Fabric-Level 分析：AMBA-23/24/25 (`dv_harness/amba_fabric_analysis.py`)

這個檔案回答的是「已發現拓模之上」的三個不同問題，因此也不是 discovery 的延伸：

- **AMBA-23 位址地圖交叉檢查** (`cross_check_fabric_address_map()`)：協調多個獨立的位址地圖證據來源，並與實際走訪到的拓模互相比對，透過 `source_authority.resolve_conflict()` (唯一的衝突解決順序) 決定。這裡有一個刻意設計的**不對稱**：規格文件永遠只能當作輔助證據，一個從未被實際 trace 到的位址區域，即使文件宣稱存在，也會被記錄、回報，然後拒絕作為 bind 規劃的輸入 (狀態為 `NO_PHYSICAL_CONNECTIVITY`) —— `assert_address_map_does_not_replace_topology()` 就是把「文件只能佐證，物理連通性才是強制」這句話變成程式碼斷言，而非留在文件裡的一句話。
- **AMBA-24 時脈 / reset domain 分析**：沿著時脈與 reset 網走訪一份 FabricNetlist (`analyze_clock_frequency()` 、 `analyze_reset_polarity()` 、 `analyze_bind_point_domains()`) ，判斷兩個 bind 點是否共用同一個 domain，並偵測跨 clock-domain (CDC) 情形。
- **AMBA-25 Scoreboard 與 Port Scaling**：純粹的 M×N 配對算術 (`build_fabric_scaling_plan()`) ，重用 `uvvm_generator/amba_fabric_generator.build_scoreboard_matrix()` 、 `compute_address_regions()` 、 `compute_id_width()` ， `assert_every_port_has_its_own_channel()` 保證每個 port 都分到獨立通道，不會共用。

3.5 Transaction IR 與 Route/Transform Predictor：規劃，而不是實測

`amba_transaction_ir.py` (SYOSCB-9/10) 是一份「共用內部 AMBA 交易表示法」，但它並非無中生有——七成欄位直接來自 `AMBA_PORT_REGISTRY_FIELDS` (`IR_FIELD_ORIGIN` 有斷言強制檢查)，其餘十五個是逐筆交易的執行期數值，欄位對某協定「不適用」時 (例如 APB 沒有 burst) 明確標記 `IR_FIELD_APPLICABILITY_UNKNOWN` 或不適用，而不是硬塞一個預設值——這個判斷本身也不是憑空猜的，而是從 `connectivity.py` 既有的 AMBA signal 集合推導 (`ir_field_applicability()`) 。

`amba_route_transform_predictor.py` (SYOSCB-12/13) 只回答「結構上」的一半問題：給定已發現的拓模，路由是否存在、是否合法、拓模是否隱含了某種轉換 (ID 擴充、位寬轉換、burst 拆分/合併、協定橋接、回應重新編碼、順序 domain)。它從不猜測「這一筆 AWADDR 在 slave port 會變成哪個 ADDR」這種逐筆交易數值，因為那需要即時流量與尚未讀取過的 fabric 組態；SYOSCB-12 的「不要猜路由」原則直接反映在三種結果值上：`TRANSFORM_PREDICTED_FROM_TOPOLOGY` (拓模已確定的結構性結論)、`TRANSFORM_NOT IMPLIED` (拓模未隱含此轉換)、`TRANSFORM_RUNTIME_PHASE_2` (形狀已定，實際數值要等 Phase-2)。`detect_width_conversion()`、`detect_burst_split_merge()`、`detect_id_remap()`、`detect_bridge_behavior()`、`detect_ordering_domain()` 都只重用既有原語 (`compute_id_width()`、`compute_address_regions()`、`ir_field_applicability()`)，沒有第二套算法。SYOSCB-13 的位址地圖證據交叉檢查 (`cross_check_address_map_evidence()`) 同樣重用 `address_map_verifier.verify_address_map()`；`assert_address_map_does_not_replace_topology()` 再次保證文件證據永遠不會取代已走訪的拓模。

3.6 Master/Slave Constraint IR：三層絕不合併 (`dv_harness/amba_master_slave_constraint_ir.py`)

這是三個結構上刻意分離的物件：`ProtocolLegalConstraintIR` (AMBA-4 協定「一般」允許什麼，永不涉及特定 DUT)、`DUTCapabilityConstraintIR` (這顆 DUT「實際」實作了什麼，只能來自真實 RTL/spec 證據)、`ScenarioConstraintIR` (一個測試情境「合法」可以送什麼，由前兩者導出，DUT 層未確認的欄位一律標記 `REQUIRES_HUMAN_CONFIRMATION`)。

本模組存在的唯一目的，就是強制執行這條規則：**絕不可單憑 VIP 能力就推論出 DUT 能力**。`assert_no_vip_sourced_dut_capability()` 會在建構 `build_dut_capability_constraint_ir()` 前對每一筆 DUT 證據執行，只要 `source_kind` 帶有 VIP 來源標記就直接拋出例外——沒有降級路徑；VIP user manual 或範例證明的是「VIP 能驅動什麼」，永遠不是「DUT 實際實作了什麼」。`assert_layers_structurally_separate()` 則保證 `build_amba_master_slave_constraint_model()` 回傳的物件恰好只有三個頂層鍵，任何試圖把某個維度拉平到頂層的模型都會被拒絕。

3.7 Fabric Graph IR：型態化多節點拓模與多路徑列舉 (`dv_harness/amba_fabric_graph_ir.py`)

`AMBAFabricGraphIR` 回答的是與上面三層模型完全不同的問題：fabric 內部由**哪些節點組成** (crossbar、arbiter、decoder、bridge、位寬/ID/時脈轉換器、暫存器切片、firewall、位址翻譯器、coherent node、記憶體控制器——`NODE_KINDS` 是封閉且固定的詞彙表，未知種類一律硬性拋出例外，絕不悄悄用最接近的名稱)，以及當呼叫端提供的真實拓模證據顯示某一對 (master, slave) 之間存在不只一條物理路徑時，`AMBAPathIR` 會把每一條都完整保留，絕不合併成單一「那條」路徑。「可重組態 (reconfigurable)」這個高風險宣稱被特別拒絕自由心證：`assert_no_ungrounded_reconfigurable_claim()` 會在建圖前逐節點檢查，一個節點宣告 `reconfigurable: True` 卻沒有引用真實的暫存器欄位證據，就會被拒絕——絕不能只憑協定或 fabric 家族推論。

3.8 AMBA command.txt 擴充：四個面向的語意編譯 (`dv_harness/amba_command_txt_extension.py`)

這是對一行既有 DE command.txt 陳述句做的 AMBA 專屬語意編譯，回答四個獨立問題：哪個 AMBA master 發出了這筆交易 (`master`)、位址落在哪個位址區域 (`region`)、這是 WRITE、READ 還是都不是 (`operation`)，以及這行是否在 `fork / join` 平行群組內執行 (`parallel_group`)。輸入刻意採 duck-typed (純 dict 或任何有屬性的物件)，因為它與同批次的 `de_command_style_learning.py`、`pattern_ir_assembly.py` 完全解耦，不互相 import。`master` 與 `region` 的判定只能依賴呼叫端提供的 `region_map / master_registry` (真實事實，例如把 fabric discovery/port registry 產物簡化成純 dict 後傳入)，本模組本身從不宣稱哪些 master 或 region 存在，只是依據那份資料去編譯每一行命令。`operation` 也只在真的偵測到巨集形狀的 WRITE/READ token (或明確欄位) 時才回報，不會從命令名稱猜測語意；`parallel_group` 是真的追蹤 `fork / join` 巢狀堆疊，不是靠命名慣例猜的。

3.9 讀者可信賴什麼、不可以要求什麼

以上八個模組的共同保證，可以濃縮成三句話：(1) **每一個結論都可以追溯到一個真實原語**——連通性走訪、verible 解析出的 RTL 結構、或既有已審核過的產物；(2) **不確定就明講不確定**，用固定、封閉的狀態詞彙 (十種 trace 終止狀態、`TRANSFORM_*` 三態、`UNKNOWN / REQUIRES_HUMAN_CONFIRMATION`) 取代預設猜測；(3) **所有輸出都是規劃，不是產出**——沒有一個模組會真的寫出 SystemVerilog bind 陳述句，或未經人工確認就把 DUT 能力當作事實。

CLAUDE.md 另有九個以此為基礎的複合 readiness 閘門 (AMBA Readiness Gates 一節：`L3_REFERENCE_READY`、`AMBA_PORT_REGISTRY_READY`、`AMBA_CONSTRAINT_READY`、`AMBA_CONNECTIVITY_READY`、`AMBA_VIP_BIND_READY`、`AMBA_SCOREBOARD_READY`、`AMBA_COVERAGE_READY`、`AMBA_TEST_GENERATION_READY`、`AMBA_SIGNOFF_READY`)，每一個都是逐條 AND 條件、單一未達成即整體 `NOT_READY`、證據不足一律 `INCOMPLETE_EVIDENCE`，絕不平均分數蓋過一個真正的阻塞點——這也是使用者在判讀本章任何一個模組輸出時，應該套用的同一種紀律。

4. AMBA 驗證進階議題 (功能覆蓋率 / 排序網域 / 仲裁 / 安全 / QoS / 效能)

DV Agent Harness L5 針對 AMBA 系列 (AXI/AHB/APB) fabric 驗證，在既有的 `amba_fabric_discovery.py`、`amba_port_registry.py`、`amba_transaction_ir.py`、`amba_route_transform_predictor.py` 之上，逐步補上了六個進階議題各自獨立的 IR (Intermediate Representation) 模組。這些模組共同的設計哲學與 Evidence Truth Rule 一致：每一個分類結果都必須來自呼叫者提供、附有明確證據引用 (evidence citation) 的事實，而不是從元件名稱、協定家族或拓模形狀去猜測。本章逐一介紹這批模組實際能做什麼、如何呼叫，以及各自明確揭露 (disclosed) 的能力邊界。

4.1 功能覆蓋率： `amba_functional_coverage_ir.py`

`AMBAFunctionalCoverageIR` 把連通性 (connectivity)、記憶體位址映射 (memory-map)、路由 (routing) 與排序 (ordering) 等既有事實，轉成功能覆蓋點 (coverpoint bins)，並以 7 值可達性分類 (`REACHABILITY_VALUES`) 取代傳統的 covered/uncovered 二元判斷： `COVERED_OBSERVED`、`REACHABLE_NOT_YET_HIT`、`PARTIALLY_REACHABLE_CONDITIONAL`、`UNREACHABLE_NO_LEGAL_PATH`、`UNREACHABLE_STRUCTURALLY_EXCLUDED`、`REACHABILITY_CONTRADICTED`、`REACHABILITY_UNKNOWN_INSUFFICIENT_EVIDENCE`。這七個值的用意是：「沒有連通合法路徑的 bin」與「合法但尚未被打到的 bin」與「因缺乏證據而無法判定的 bin」與「兩份證據互相矛盾的 bin」是四種本質不同的事實，全部塌陷成 uncovered 會抹去覆蓋率收斂審查最需要的區別。

模組提供四種 coverpoint 建構器： `build_connectivity_coverpoints()` (連通合法邊)、`build_memory_map_coverpoints()` (位址區域擁有權)、`build_routing_coverpoints()` (路由路徑)、`build_ordering_coverpoints()` (排序情境，如 outstanding 深度分桶、亂序完成情境)。「只追蹤有意義的 cross coverage」 (meaningful crosses only) 是本模組獨立重新實作、而非引用同一批次中 `pattern_coverage_contribution.py` 已有的同名邏輯——兩者刻意各自實作同一個小判斷：一個 cross bin 的兩個軸若都已各自 100% covered，則該 cross 被歸類 `FULLY_EXPLAINED_BY_AXES` 並直接略過，不建立任何組合 bin；若任一軸的覆蓋率無法判定，則誠實回報 `UNKNOWN_AXIS_COVERAGE`，絕不視為已充分解釋。

本模組不負責判定連通合法性、位址擁有權、路由/轉換行為或排序語意——這些都必須由呼叫者的真實管線 (連通性分析、位址映射交叉檢查、`amba_route_transform_predictor.py`) 提供。它也不寫入任何證據庫、不產生 memory/blackboard 記錄，且刻意沒有 stage gate。校對更正：2026-09-06 的批次 CLI 接線已把它掛成正式的 `dv-harness` 子指令 (且該次接線把模組原本 `python -m` 介面的 `build` 子動詞拉平，`--facts` 變成直接旗標)：

```
dv-harness amba-functional-coverage-ir --facts <file.json> [--json]
# 舊的 python -m 介面 (仍保留 build 子動詞) 依然可用：
python -m dv_harness.amba_functional_coverage_ir build --facts <file.json> [--json]
```

4.2 排序網域： `ordering_domain_graph.py`

`amba_route_transform_predictor.detect_ordering_domain()` 只回答一個較窄的問題：單一 (master, slave) 路由本身的交易流是 in-order 還是 out-of-order。`OrderingDomainGraph` 補上更高一層的問題——排序網域 (ordering domain) 作為一個具備真實成員關係的一級物件，以及兩個網域之間是否存在已宣告的排序保序關係。

三個資料形狀都強制要求證據引用，缺引用即以 `OrderingDomainGraphError` 拒絕：`OrderingDomain` (`domain_id + evidence`)、節點成員關係 (`node_id/domain_id + evidence`)、以及有向的保序邊 (`from_domain / to_domain / preservation` $\in$ { `PRESERVED`, `NOT_PRESERVED`, `UNKNOWN` } + `evidence`)。若某網域宣告自己是「可重組態 / 動態」 (reconfigurable)，必須額外引用真實的暫存器欄位或文件化的模式切換證據，否則直接拒絕——這正是本模組對「絕不能只憑協定家族猜測可重組態性」這條專案級鐵律的具體實作。

`query_domain_preservation()` 的核心規則是絕不猜測：它只回答「A 到 B 這一個方向是否有直接宣告的保序邊」，從不從正向關係反推逆向保證 (橋接元件常常一個方向保序、另一個方向不保序)，也從不計算三個以上網域的遞移閉包 (A 保序進 B、B 保序進 C，不代表 A 保序進 C，因為 B、C 之間的合併/比較階段是另一件事，沒有人宣告過)。兩個方向相同的邊若宣告不一致的保序值，會回報為 `CONTRADICTORY_EDGE` 發現，解析後的邊讀作 `UNKNOWN` 並同時引用雙方證據——本模組從不裁決哪一方正確。一個節點同時屬於多個網域也不是錯誤 (橋接元件確實可能同時處於兩個排序網域)，而是回報 `MULTI_DOMAIN_MEMBERSHIP` 供人工檢視。

本模組完全不匯入 `arbitration_policy_ir.py` 或本批次其他新模組，僅重用 `connectivity.render_markdown_table()` 做選擇性的 markdown 渲染。它不發現拓模、不計算 reorder-window 深度或 outstanding 上限，且刻意沒有 stage gate 與 `dv-harness` CLI 動詞：

```
python -m dv_harness.ordering_domain_graph --facts-file <file.json> [--json]
```

4.3 仲裁： `arbitration_policy_ir.py`

工程紀律規則早已明文要求「跨 block / branch_a* 等分支、共用同一資源的並行 APB/AXI 交易，必須有明確、以 RTL 證據為基礎的仲裁政策」，但此前沒有任何機制真正分類那個政策究竟是什麼。`classify_arbitration_scheme()` 從呼叫者提供的證據文字 (RTL 註解、仲裁器模組標頭、規格書段落) 分類出五種真實仲裁機制之一：`FIXED_PRIORITY`、`ROUND_ROBIN`、`WEIGHTED_ROUND_ROBIN`、`AGE_BASED`、`QOS_BASED`，或誠實的 `UNKNOWN`。

分類是逐字片語比對，不是關鍵字或名稱啟發式：每種機制只靠一組固定、大小寫不敏感的字面片語辨識 (例如 "weighted round robin arbitration"、"fixed priority arbitration")。這一點有專門的負向測試證明——一個字面上叫做 `round_robin_arbiter` 的元件，若其證據文字實際描述的是 `FIXED_PRIORITY` 行為，分類結果就是 `FIXED_PRIORITY`，絕不受名稱影響；`component_name / fabric_name` 參數純粹只是結果的標籤。若證據文字同時命中兩種真正不同的機制 (且無包含關係)，則誠實回報 `AMBIGUOUS`，並列出所有命中的機制，而非任意選一個。有一條刻意的去重規則：`WEIGHTED_ROUND_ROBIN` 的片語必然包含 `ROUND_ROBIN` 片語的子字串，這屬於「包含」而非「歧義」，因此更具體的 `WEIGHTED_ROUND_ROBIN` 勝出。

飢餓風險偵測 (starvation-risk detection) 從不臆造公平性界限。`extract_fairness_bound()` 只在同一段證據文字裡尋找已聲明的服務窗/公平性界限 (例如 "maximum wait of N cycles")；若找不到，且分類出的機制是 `FIXED_PRIORITY`，同時呼叫者宣告的請求模式中有一個持續活躍的高優先權請求者與一個同時存在的低優先權請求者，則依據一般 (非 RTL 特定的) 仲裁理論回報 `POTENTIAL_STARVATION`；否則誠實回報 `UNKNOWN`。若真的找到界限，會拿去與呼叫

者宣告的 `max_grants_between_service` (真實模擬量測、形式證明或文件化最壞情況分析的結果，本模組本身不做任何這類推導) 比較，得出 `BOUNDED` 或 `POTENTIAL_STARVATION` 。

本模組不讀取 RTL 或規格文件本身、不模擬仲裁器的逐週期授權序列，也不判斷公平性界限本身是否「正確」，僅判斷宣告的請求模式是否落在宣告的界限內。**校對更正**：2026-09-06 的批次 CLI 接線已把它掛成正式的 `dv-harness` 子指令，旗標形狀不變：

```
dv-harness arbitration-policy-ir --evidence-file <file> [--request-pattern-file <file>] [--json]
```

4.4 安全：security_policy_ir.py

`SecurityPolicyIR` 建立一個 `secure/non-secure × privileged/unprivileged` 的存取矩陣，每一筆存取規則 (`master`、`region` 及其萬用子集組合) 都必須帶有非空的 `evidence` 引用 (規格章節、RTL file:line、暫存器程式設計指南) ；缺引用的規則會被 `SecurityPolicyIRError` 直接拒絕，因為一個沒有引用來源的安全存取決策，正是 `Evidence Truth Rule` 所禁止的「自信猜測」。

沒有任何規則覆蓋到的存取組合分類為 `UNKNOWN`——絕不預設為 `ALLOWED` (`fail-open` 的預設允許猜測正是本專案禁止的靜默預設)，也絕不預設為 `DENIED` (那會捏造一個沒人宣告過的安全決策)。呼叫者可以宣告一個明確且附引用的 `default_decision` (例如「未宣告的區域/master 依規格 X 節預設 `DENY`」)，該預設值只在沒有特定規則命中時套用，且在結果中永遠與「命中特定規則」的決策可區分。兩條同等 `specificity`、針對同一存取組合卻給出不同決策的規則，會構成真正的 `CONFLICT`——本模組從不選出勝方 (沒有在此宣告任何來源權威順序；若需要仲裁，應由呼叫者另行調用如 `source_authority.py` 這類真正的衝突解決機制)。

模組另外提供 `verify_denial_observed()` 這個負向測試驗證輔助函式：給定一個矩陣判定應為 `DENIED` 的存取，它檢查呼叫者提供的 `test_result` 是否真正證明該拒絕被觸發並被觀察到——絕不因為某個總體 `verdict` 欄位寫 `PASS` 就假設負向測試通過。三件事都必須成立且各自有證據：該存取確實依矩陣分類為 `DENIED`；`test_result` 宣告 `access_attempted: true`；`test_result` 宣告一個真實、附引用的 `observed_outcome` (`ACCESS_DENIED_OBSERVED` / `ACCESS_ALLOWED_OBSERVED` / `NO_RESPONSE_OBSERVED` / `UNKNOWN`)。只有 `ACCESS_DENIED_OBSERVED` 且附真實引用才確認拒絕 (`DENIAL_CONFIRMED`)；`ACCESS_ALLOWED_OBSERVED` 是這個輔助函式存在的核心目的——矩陣宣告應拒絕的存取，實測卻被觀察到成功，這是安全關鍵發現，回報為 `DENIAL_NOT_CONFIRMED` 並明確點名該發現；無回應/逾時同樣是 `DENIAL_NOT_CONFIRMED`，因為沒有回應不等於拒絕的證明。

校對更正 (兩處)：(1) 第二個子動詞的真實名稱是 `verify-negative-test`，不是 `verify`；(2) 2026-09-06 的批次 CLI 接線已把它掛成正式的 `dv-harness` 子指令：

```
dv-harness security-policy-ir classify --policy <policy.json> --master <m> --region <r> --secure {true,false} --privileged {true,false} [--json]
dv-harness security-policy-ir verify-negative-test --policy <policy.json> --master <m> --region <r> --secure {true,false} --privileged {true,false} --test-result <result.json> [--json]
```

4.5 QoS：qos_policy_ir.py

`QoSPolicyIR` 是一個逐 `master` 的 `QoS-level/priority/weight` 對照表，同樣建立在「只轉錄，不創作」的原則上：任何帶有 `master_id` 卻沒有 `evidence` 的條目都會被拒絕；一個確實查無 `QoS` 事實可報告的 `master` 不是錯誤，而是記錄 `NO_QOS_FACTS_DECLARED` 這個誠實的「無」，與格式錯誤的紀錄明確區分。

`derive_priority_ordering()` 處理「數字越大代表優先權越高，還是越低？」這個沒有唯一答案的協定慣例問題——AXI 自身的 `QoS` 欄位與手工仲裁優先權暫存器對此經常互相矛盾，猜錯會靜默反轉下游所有的排序判定。因此本函式要求呼叫者明確宣告 `priority_convention` (`HIGHER_IS_HIGHER_PRIORITY` 或 `LOWER_IS_HIGHER_PRIORITY`)；若沒有宣告，或帶有真實 `priority` 值的 `master` 少於兩個，就誠實回報 `ORDERING_NOT_AVAILABLE`，絕不猜測排序。

`verify_qos_contention()` 是一個純序數 (`ordinal`) 檢查，明確地不是效能檢查——本批次任務明文將效能驗證 (任何數值化的延遲/頻寬/吞吐量目標) 排除在此範圍之外。它只回答：在同一 `window_id` 內真正互相競爭同一資源的交易中，優先權較高的 `master` 交易，其 (呼叫者提供的序數) `position` 是否不晚於優先權較低者。`position` 是授權順序或 `scoreboard` 序號一類的純序數索引，絕不是時間戳記或週期數，本模組本身也完全不計算或宣稱任何速度數字。三個誠實判定 `VERIFIED` / `VIOLATED` / `UNKNOWN` 從不塌陷為兩個。

4.6 效能：amba_performance_calculator.py / _requirement_checker.py / _classification.py / _readiness_gates.py

效能是本批次風險最高的領域，因為此 `harness` 本身沒有可運行的模擬器或形式化工具——所有數字都必須來自呼叫者已經產生的證據 (`fsdb_report.py` 輸出、`sim.log`、或呼叫者自行整理的量測紀錄)。四個模組共同守住三條被寫進程式碼、而非只寫在文件裡的鐵律：

1. 數值門檻/目標從不臆造——`evaluate_against_target()` 在沒有呼叫者宣告 `target_value` 時一律回報 `NOT_APPLICABLE`，絕不生成一個假的通過/失敗。
2. 無法證明的峰值/基準/指標一律回報 `UNKNOWN`，絕不輸出一個「看起來像算出來的」數字——`bandwidth_utilization()` 是最鮮明的例子：沒有呼叫者提供、已證實的峰值頻寬，它必須回報 `UNKNOWN`，絕不拿一個自行倒推或猜測出的天花板去除。
3. 功能正確性永遠凌駕效能 `PASS`——`decide_overall_verdict()` 是一個硬性優先順序分支 (重用 `dv_harness.models.Status`，而非重新拼一套詞彙)：功能面 `FAIL` 就是整體 `FAIL`，不論效能數字多漂亮；效能 `PASS` 也絕不能把一個功能不正確的結果拉抬成整體 `PASS`。

`amba_performance_calculator.py` 提供純算術函式：`compute_bandwidth` / `compute_throughput` (位元組或交易數 / 時間)、`compute_latency_percentiles` (`p50/p90/p95/p99`，需強制宣告 `latency_definition` 以避免不同呼叫者對「延遲」定義不一致)、`compute_outstanding_stats`、`compute_stall_ratio` / `compute_utilization`。`amba_performance_requirement_checker.py` 在此之上加一層需求比對：`PerformanceRequirementIR` 是永遠必須來自呼叫者/規格宣告的門檻物件，`check_against_requirement()` 重用 (而非重新實作) `evaluate_against_target()` 的比對邏輯，並把規則 3 的優先順序落實為程式碼——一旦 `functional_verdict` 是真實的 `Status.FAIL`，回傳的整體狀態就被無條件覆寫為 `FAIL`。`amba_performance_classification.py` 再往上一層，做飽和度 (`saturation`)、瓶頸候選、異常偵測與迴歸差量四種分類，且每一種都要求至少兩個相互印證的指標，單一指標永遠回報 `UNKNOWN`，兩個指標互相矛盾則回報 `INDETERMINATE`；`identify_bottleneck_candidate()` 更直接拒絕只有一筆證據支撐的假說物件。最後，`amba_performance_readiness_gates.py` 提供兩個複合就緒閘門 `BUS_PERFORMANCE_READY` 與 `BUS_PERFORMANCE_SIGNOFF_READY`，遵循本專案一貫的「worst-wins、絕不平均」折疊規則：

校對更正：四個效能模組中，只有 `amba_performance_readiness_gates.py` 在 2026-09-06 的批次 CLI 接線中被掛成正式的 `dv-harness` 子指令；另外三個（`amba_performance_calculator.py` / `_requirement_checker.py` / `_classification.py`）截至校對時仍維持 `python -m` 前門，未被納入這批接線：

```
dv-harness amba-performance-readiness-gates gates [--json]
dv-harness amba-performance-readiness-gates conditions --gate BUS_PERFORMANCE_READY [--json]
dv-harness amba-performance-readiness-gates evaluate --conditions <file.json> [--json]
```

這四個效能模組彼此之間也刻意保持隔離——`amba_performance_readiness_gates.py` 不匯入另外三個計算/分類模組，只接受呼叫者已整理好的 `{condition_name, status}` 紀錄，因此即使其中某個效能子模組尚未完成或未執行，就緒閘門仍可運作而不會假設它已跑過。

4.7 小結：共通的邊界揭露

以上六大議題的模組有一個共同、反覆出現的揭露模式：它們都只做分類與比對，不做拓樸/連通性/RTL/波形的原始發現（那是 `amba_fabric_discovery.py`、`connectivity.py` 等既有模組的責任）；都沒有 stage gate，也沒有註冊 `dv-harness` CLI 動詞（`cli.py` / `gates.py` 在這批任務中刻意保持不動）；前門一律是各自的 `python -m dv_harness.<module>` 介面。任何「無法判定」的情況一律誠實回報為 `NOT_AVAILABLE` / `UNKNOWN` / `AMBIGUOUS` 之一，絕不與「已確認為負」或「已確認為正」混淆——這正是本章開頭所述 Evidence Truth Rule 在 AMBA 進階驗證領域的具體落實。

5. 系統級 (Subsystem-to-System) 驗證:拓樸整合、資源仲裁、失效傳播與系統結案

當多個 subsystem-mode 驗證環境要被組合成一個 SoC 等級的系統時,「這個系統可以整合了嗎」不再是單一環境的問題,而是牽涉到位址地圖是否衝突、時脈/重置是否相容、多個 subsystem 是否搶著驅動同一個實體介面、命令 是否互相碰撞、以及一旦某個 subsystem 出錯,故障會不會沿著真實的拓樸關係擴散到其他 subsystem。DV Agent Harness L5 把這一整條 System-Level Verification Integration workflow (spec 內部代號 SYS-1 到 SYS-40) 拆成一系列職責單一、彼此唯讀組合的模組。這一章要講的,正是這條 workflow 裡「系統級」的那半段:拓樸分析 (SYS-28..30)、資源盤點與衝突仲裁 (SYS-9..17)、命令規劃 (SYS-18..22)、排程規劃 (SYS-23..27)、系統 建置證明 (spec 206)、失效傳播與失效/檢查器分類法,以及最後把一切捲成一個結案判斷的 Verification Contract 與 Closure Aggregator。

貫穿整章最重要的一條紀律是:這些模組幾乎全部只做分類、比對、彙總,不做生成或仲裁。凡是會真的 產出 System-Level UVM 原始碼、System command.txt、共享 sequencer/driver、或替兩個衝突的 ACTIVE driver 選出贏家的動作,一律屬於 SYS-40,而 SYS-40 被 SYS-39 這個「明確人類核准」關卡擋住,尚未被本專案 自動實作。這個邊界在每個模組自己的 docstring 裡都寫明,不是文件杜撰出來的美化說法。

5.1 SYS-28..30:跨 subsystem 拓樸分析 (system_topology_analysis.py)

這個模組做三件事,而且明講「這不是什麼」:它從不搬移、重新配置或保留任何位址範圍,也從不產生、合併或重新 命名任何時脈/重置訊號,更不輸出任何 System command.txt 或 scenario 內容。

- **SYS-28 位址地圖分析**:分類兩個 subsystem 之間的位址關係 (ADDRESS_OVERLAP_VALID / ADDRESS_OVERLAP_CONFLICT / SHARED_MEMORY / UNKNOWN),真正的衝突只會 被填進真實的 question queue 裡等人回答,絕不會自己決定該怎麼重新配置。
- **SYS-29 時脈/重置整合比對**:比較兩個 subsystem 各自宣告的時脈來源/頻率與重置來源/極性/順序,包含 CDC (clock-domain-crossing) 邊界的偵測。它輸出的「ownership」欄位永遠是一個 PLANNED_NOT_IMPLEMENTED 的建議,而不是既成事實。
- **SYS-30 系統級 scenario 形狀規劃**:描述一個多 subsystem scenario「長什麼樣子」——哪些 subsystem 會參與、各自哪些既有命令該落在 parallel block、哪些落在 sequential block——但每一個 scenario 的 scenario_body_status 都被釘死在 NOT_GENERATED_SYS40_REQUIRES_HUMAN_APPROVAL ,不吐出一行 命令文字。

assert_no_emitted_artifacts() 這個函式在執行期真的驗證上述三條邊界,測試也直接斷言它,而不是只信任 docstring 裡的文字。CLI 進入點是:

```
dv-harness system-topology-analysis --select <SUBSYSTEM> [--select <SUBSYSTEM> ...] \
  [--command-inventory <CSV>] [--escalate] [--json]
```

--select 是必填且可重複,因為 SYS-1 的「明確選擇」拒絕條件不允許被這個指令繞過; --escalate 會把每一個 真實的跨 subsystem 位址衝突透過 source_authority.escalate_conflict() 寫進真正的 question queue (同一組未改變的位址地圖重跑會重新產生相同的 Q-ID,具鼻等性)。

5.2 SYS-9..17:跨 subsystem 資源盤點與主動驅動衝突 (system_resource_inventory.py)

connectivity.py 既有的 verify_matrix_self_check_identity() 是一個封閉世界的身份檢查:它只在「一個」subsystem 的 connectivity matrix 裡驗證「已驗證介面數 == VIP 實例數 + 豁免數」,而 matrix 欄位 本身不帶任何 subsystem 識別欄位。這代表如果有人把兩個 subsystem 的 rows 串接起來比對,同一個「CPU AXI Master」在 PCIe 環境出現一次、在 USB 環境又出現一次,兩邊各自的自我檢查都會通過,重複的資源 永遠不會被揭露——因為這個函式從來沒有同時拿到兩份 matrix。system_resource_inventory.py 正是為了補上 這個結構性缺口而生的新模組,而不是延伸既有檔案:它同時消化 N 個 subsystem 的證據,對每個 subsystem 先各自跑一次 connectivity.verify_matrix_self_check_identity(),再做跨 subsystem 的重複資源偵測、關係分類、以及主動驅動衝突規則——也就是 SYS-11/SYS-12 的核心:一旦偵測到兩個 subsystem 各自的 VIP agent 同時 ACTIVE 驅動同一個實體介面,就回報 REL_DRIVER_CONFLICT ,並用 INTEGRATION_STOPPED 這個結果 token 停止自動整合,只把 SYS12_PREFERRED_MODEL 這段建議文字留給 人類決定——它從不替兩個 ACTIVE driver 選出贏家。

CLI:

```
dv-harness system-resource-inventory --select <SUBSYSTEM> [--select <SUBSYSTEM> ...] \
  [--knowledge-center] [--command-inventory <CSV>] [--json]
```

同一份 workflow 裡緊接著的 SYS-15..17 產出 SYSTEM_RESOURCE_REGISTRY (每個系統級資源一列,帶 owner 與 consumer_subsystems 欄位,是與 subsystem-granularity 的 registry 及 connectivity matrix 都不同的第三種粒度),對應的 CLI 是 dv-harness system-integration-plan 。

5.3 SYS-18..22:系統命令規劃與碰撞偵測 (system_command_plan.py)

這個模組同樣先講清楚「不是什麼」:它不會輸出 System command.txt、System Command Parser/Router、System Scenario Planner、System Virtual Sequencer、command adapter,或任何 System-Level UVM 原始碼。SYS-18 的架構樹只是一份「建議結構」的描述;SYS-19 的路由規劃只是一張「哪個既有 subsystem parser 會接手哪個系統級命令名稱」的表;SYS-20 的 adapter 規劃只是命名一個尚不存在的 adapter。真正 把這些東西建出來屬於 SYS-40,一樣被 SYS-39 擋住。

值得特別記住的是它跟 subsystem_command_contract.py (SYS-8) 的分野:SubsystemCommandContract 是「單一 subsystem 自己的 command.txt 怎麼寫」的記錄,粒度太細,不能直接拿來做跨 subsystem 的命令碰撞 偵測,所以本模組是新開的、更粗粒度的彙總層。CLI:

```
dv-harness system-command-plan --select <SUBSYSTEM> [--select <SUBSYSTEM> ...] [--json]
```

5.4 SYS-23..27:排程規劃、平行化模型與跨 subsystem 檢查規劃 (system_scheduling_plan.py)

同一條「不生成、只規劃」的紀律延伸到排程層:這裡不產出共享 sequencer、共享 driver、共享佇列、arbiter、System Scoreboard、Correlation Layer、跨 subsystem checker,或任何 scenario 內容。五個子步驟各自的 邊界:

- **SYS-23 初始化去重分類**: 只分類命令, 從不移除任何一條——`SYS23_DEDUP_ACTIONS` 裡刻意沒有 `REMOVE` 這個值, 該步驟自己的結語就是「沒有證據就不要移除命令」, Phase-1 報告最多只能建議人類去 決定。
- **SYS-24 共享資源排程規劃**: 只命名每一個共享實體 agent 應該經過的單一存取點, `access_point_status` 永遠是 `PLANNED_NOT_IMPLEMENTED` ——因為 sequencer/driver/queue 是程式碼, 程式碼屬於 SYS-40。
- **SYS-25 平行化模型**: 分類命令對之間的關係, 不排程任何東西。
- **SYS-26 Scoreboard 整合重用規劃**: 記錄每個被選中 subsystem 的既有 scoreboard 會被原封不動重用, 並描述上層應該存在的 correlation layer (含其尚不存在的前置條件), 不寫任何 scoreboard 內容。
- **SYS-27 跨 subsystem 檢查規劃** 同樣是規劃而非實作。

CLI: `dv-harness system-scheduling-plan --select <SUBSYSTEM> [...] [--json]`。

5.5 spec 206: 系統建置與煙霧證明 (`system_build_proof.py`)

在這批模組加入前, 對 spec section 206 那條「Build → Elaborate → Boot/Reset/Init → Shared-Resource-Access → One-Subsystem → Two-Subsystem-Interaction → One-End-to-End-Scenario → WAVE=1/fsdbreport → Scoreboard/Assertion → SYSTEM_READY」的煙霧證明梯子, repo 裡完全沒有任何程式碼 真的跑過它 (用 `grep -rn "smoke_proof|SMOKE_PROOF|system_smoke"` 驗證過, 一個匹配都沒有)。`system_readiness.derive_system_readiness()` 自己的 docstring 就承認它只是「靜態 metadata rollup」, 「Phase 1 的 build integration 是關於輸入而非建置本身的問題: 根本沒有 System-Level filelist 可編譯」; `uvm_structural_lint.py` 也只 lint 單一環境, 同名的 class 或 package 出現在兩個不同 subsystem 環境裡, 對逐環境的 lint 而言天生不可見。

`system_build_proof.py` 把兩件事都做成真的:

- `analyze_system_merge()` 是靜態、免模擬器的合併衝突檢查。它用 `uvm_structural_lint.parse_uvm_file()` 同一套 verible 前端 (整個 repo 沒有第二套 SystemVerilog parser) 解析合併後的來源集合, 偵測 section 206 點名的五種缺陷: `DUPLICATE_PACKAGE_DECLARATION`、`DUPLICATE_TYPE_DEFINITION` (class/interface/module)、`FACTORY_TYPE_NAME_COLLISION` (UVM factory 是用註冊字串當 key, 兩個不同名的 class 註冊同一個字串就會衝突, 即使沒有任何重複定義)、`CONFIG_DB_SET_SCOPE_COLLISION`、`VIRTUAL_INTERFACE_CONFLICT`。它刻意的邊界: `config_db` 衝突只在兩邊的 `set()` 都是全域 scope (`null / uvm_root::get() / uvm_top`) 且 `inst_name` glob 重疊時才回報——`set(this, ...)` 的實際 scope 要看實例化位置, parser 看不出來, 所以永不回報 ERROR, ERROR 只保留給來源真的能證明的情況; 跑時期才決定的 scope 會列成 INFO, 說明「已排除」, 絕不悄悄丟掉; 同一個 subsystem 內部的兩個衝突 `set` 不回報 (那個環境本來就能單獨跑, 這個檢查關心的是「合併打壞了什麼」); 重複的 VIP 與位址衝突不在這裡重做, 那是 SYS-9.14 / SYS-28 的地盤。
- `run_system_smoke_proof()` 這條梯子完全呼叫既有的真實機制, 絕不自己重新發明: ELABORATE 用 `connectivity.run_gate1_elaboration_check()`; BOOT_RESET_INIT 用 `connectivity.evaluate_zero_time_connectivity()` (或誠實回報 NOT_AVAILABLE 的 `run_gate2_against_live_simv()`); SHARED_RESOURCE_ACCESS 就是 `system_resource_inventory.real_cross_subsystem_findings()` 本身; ONE_SUBSYSTEM / TWO_SUBSYSTEM_INTERACTION 用 `connectivity.evaluate_transaction_activity_status()` (在真正的 pattern 完成前是 PENDING, 絕不因「沒證據」就判 FAIL); WAVE_FSDBREPORT 用 `fsdb_report.run_fsdbreport()` + `parse_fsdbreport_output()`, 前提是呼叫者已經有一份 fsdb (它從不主動啟用 dump); SCOREBOARD_ASSERTION 讀真實 `evidence_db` 的 `normalized_evidence` 列並用 `golden_scenario.PASS_VERDICTS` 判斷, 不另立第二套「乾淨」的定義; END_TO_END_SCENARIO 預設是 NOT_AVAILABLE, 因為 `cross_subsystem_scenarios()` 是刻意 `raise NotImplementedError` (不做 Golden-Reference 內容探勘), 而 SYS-40 停在人類核准前。

SYSTEM_READY 要求每一階都 PASS; 任何一階 NOT_AVAILABLE/PENDING 就是 `SMOKE_NOT_PROVEN` (UNKNOWN 永遠不會自動變成 READY); 一旦某階 FAIL, 梯子立刻 `SMOKE_FAIL` 並整個停住, 後面的階都標成 NOT_YET_RUN。CLI:

```
dv-harness system-smoke-proof [--merge-only] [--json]
python -m dv_harness.system_build_proof
```

exit code: 0=SYSTEM_READY、1=SMOKE_FAIL、2=SMOKE_NOT_PROVEN。這個模組不生成任何東西、也不仲裁任何東西——它不呼叫任何 composer 的生成入口, 不寫任何檔案, 不送出任何 job, 也不替兩個 ACTIVE driver 選贏家: 一旦有 DRIVER_CONFLICT, 那一階就 FAIL, 並帶著 `human_arbitration_required` 與 SYS-12 偏好模型的文字, 留給人類決定。實測時它真的對本專案自己產生出來的兩個範例環境 (`examples/generated_pcie_uvm_env`、`examples/generated_usb_real_evidence_v12`) 跑過, 發現兩者都輸出了同名的 `module tb_top`, 是一個真實的系統合併缺陷。

5.6 失效傳播 IR: 追蹤真實的跨 subsystem 波及範圍 (`system_error_propagation.py`)

SYS-28/SYS-29 的機制已經算出這個 repo 裡唯一真實的跨 subsystem 「關係」: 哪些位址區域被兩個 subsystem 實體共享或衝突、哪些中斷線名稱被多個 subsystem 的證據同時命名、哪些時脈/重置名稱被兩個 subsystem 共享 (或跨越一條共享位址路徑)。但過去沒有任何東西把這些關係變成一張「傳播圖」, 回答「如果 subsystem X 發生某個故障條件, 拓樸證據實際顯示哪些其他 subsystem 可能受影響, 而這些受影響的 subsystem 是否都申報了真實的復原/回應動作?」`system_error_propagation.py` 正是這條追蹤, 而且只做這件事。

為了遵守本批次自己的檔案安全範圍, 它不 import `system_topology_analysis.py` 或 `system_resource_inventory.py`, 而是接受一個 duck-typed 的 `topology` 參數, 結構要像 `build_system_topology_analysis()` 的真實輸出 (`address_map_reconciliation.overlaps`、`interrupt_map_reconciliation.lines`、`clock_reset_comparison.clock_comparisons / reset_comparisons`), 用純字典存取讀取, 絕不重新推導拓樸分析。

關鍵規則: 一條傳播邊只會從拓樸分析自己已經判為「證實耦合」的判斷產生, 絕不會從它自己誠實的「無法判斷」判斷產生。位址方面只有 `SHARED_MEMORY / ADDRESS_OVERLAP_VALID / ADDRESS_OVERLAP_CONFLICT` 算數, SYS-28 自己的 UNKNOWN (單一 subsystem 內部證據品質缺陷) 被排除, 否則等於用一個 subsystem 內部的不一致去捏造一個跨 subsystem 的發現。傳播本身是 `bfs_reachable()` 跑的真實廣度優先搜尋, 只沿著跟宣告的失效條件種類相關的邊走 (`CONDITION_TO_EDGE_KINDS`: `ADDRESS_DECODE_FAULT / BUS_ERROR / DMA_CORRUPTION` 只走共享位址邊; `INTERRUPT_STORM` 只走共享中斷線邊; `RESET_ASSERTION` 只走共享重置域邊; `CLOCK_LOSS` 只走共享時脈域邊; `CDC_VIOLATION` 走 CDC 邊加共享時脈/重置邊; `GENERIC` 或任何本模組不認識的條件種類則走全部邊——取最寬的集合, 絕不猜一個較窄的)。拓樸沒有真的連到的 subsystem, 絕不會被回報成受影響。

至於「每個 subsystem 是否申報了復原動作」, 這個 repo 目前沒有任何模組真的產出 `recovery_action / expected_response / error_handler` 欄位, 所以這是誠實的呼叫端提供輸入。`resolve_declared_response()` 把每個受影響 subsystem 分成四種狀態: `RESPONSE_DECLARED` (真實、非佔位的復原文字)、`NO_RESPONSE_DECLARED` (完全沒有對應紀錄或文字為空)、`RESPONSE_PLACEHOLDER_ONLY` (像 TBD / N/A / unknown / ? 這類佔位值)、

RESPONSE_EXPLICITLY_DECLARED_NONE (明確聲明 declares_response: false)——只有 RESPONSE_DECLARED 才算數。RECOVERY_CHAIN_COMPLETE 只有在每一個受影響 subsystem 都是 RESPONSE_DECLARED 時才成立,任何一個缺漏、佔位或明確無回應都讓整條鏈變成 RECOVERY_CHAIN_INCOMPLETE,並點名是哪個 subsystem。零個受影響 subsystem 是誠實地區分出來的 NO_PROPAGATION_DETECTED (代表拓模顯示故障被侷限在原點,而不是「已檢查過且乾淨」的 COMPLETE)。拓模裡完全沒提到的來源,或空拓模,回報 NOT_AVAILABLE。

沒有 dv-harness CLI 動詞(這批次的檔案安全範圍沒有動到 cli.py),前門是:

```
python -m dv_harness.system_error_propagation trace --origin <SUBSYSTEM> --condition <KIND> \
--topology <FILE> [--declared-responses <FILE>] [--json]
```

exit code:0= RECOVERY_CHAIN_COMPLETE、1= RECOVERY_CHAIN_INCOMPLETE、2= NOT_AVAILABLE。它從不仲裁誰的復原動作才「正確」,也從不判斷傳播路徑該不該在架構上被切斷。

5.7 系統級失效分類法與檢查器分類法

system_failure_taxonomy.py 在同一個模組裡放了三個回答「同一個粒度問題」的機制,並且明確跟兩個最容易混淆的鄰居劃清界線: command_error_taxonomy.py 的十一類回答「這一次命令派送為什麼失敗」, loop_budget.FailureType 的十類回答「這個 stage 的重試為什麼耗盡」,而本模組回答的是「這是哪一種系統/多 subsystem 組合層級的失敗」——一個更粗的粒度。三個機制是:

1. 14 值系統整合失效分類法(SUBSYSTEM_FAILURE、INTEGRATION_FAILURE、ROUTING_FAILURE、RESOURCE_CONTENTION_FAILURE、ADDRESS_MAP_FAILURE、CLOCK_RESET_FAILURE、COMMAND_COMPATIBILITY_FAILURE、BUILD_COMPOSITION_FAILURE、SCOREBOARD_COMPOSITION_FAILURE、VIP_DEDUP_FAILURE、ERROR_PROPAGATION_FAILURE、TIMING_FAILURE、CONFIGURATION_FAILURE、RECOVERY_FAILURE,加上誠實的 UNCLASSIFIED 後備值)。classify_system_integration_failure(text) 是純規則式的正則分類器,吃現成的失效文字(例如 system_build_proof 產生的合併報告行、跨 subsystem gate 的拒絕文字、組合環境的 sim.log 片段),不讀檔、不跑 subprocess,不匹配的文字誠實回 UNCLASSIFIED,絕不硬塞進十四類之一。TIMING_FAILURE 只分類「已經被別的工具回報出來的」時序/競爭文字,本身不做任何時序分析。
2. 根因邊界定位 localize_failure_boundary(subsystem_io_map, connections=None): 給定每個 subsystem 自報的輸入/輸出正確性(CORRECT / INCORRECT / UNKNOWN),只有「恰好一個 subsystem 輸入確認 CORRECT 且輸出確認 INCORRECT」這種情況才會收斂成 status: "NARROWED", 點名那個 subsystem 為 boundary。零個候選、多個候選,或跨 subsystem 連線出現真正矛盾,都回報 status: "UNDETERMINED"、boundary: None,絕不用猜的方式指認單一根因。
3. 6 值系統覆蓋率缺口分類法(SUBSYSTEM_GAP、INTEGRATION_GAP、RESOURCE_GAP、SCENARIO_GAP、ERROR_PATH_GAP、COVERAGE_MODEL_GAP,加上 UNCLASSIFIED_COVERAGE_HOLE),跟 coverage_analysis.classify_coverage_hole() 的四類「單一 bin 為何沒打中」是刻意不同的更粗粒度,兩套分類法各自的詞彙不共用任何 token。

system_checker_taxonomy.py 則是一個只做分類的 9 值 SYSTEM-scope checker 類型分類器(DATA_FLOW_CHECKER、RESOURCE_ARBITRATION_CHECKER、ADDRESS_ROUTING_CHECKER、CLOCK_RESET_SEQUENCING_CHECKER、COMMAND_COMPATIBILITY_CHECKER、BUILD_INTEGRITY_CHECKER、SCOREBOARD_COMPOSITION_CHECKER、RECOVERY_CHECKER、ERROR_PROPAGATION_CHECKER,加上 UNCLASSIFIED_SYSTEM_CHECKER)。它跟 verification_architecture.py's CheckerIR 是刻意不同、更高一層的分類法——CheckerIR 問的是「這個 checker 在自己 subsystem 裡有沒有正確綁定」,本模組問的是「這個 checker 驗證的是哪一種系統級關注點」。classify_system_checker(description) 吃字串或 dict/物件描述,有明確的 declared_checker_type 一定優先,否則走固定的 CLASSIFICATION_ORDER 關鍵字比對,匹配不到就誠實回 UNCLASSIFIED_SYSTEM_CHECKER。兩者都不讀檔、不跑模擬,沒有 dv-harness CLI 動詞,前門是 python -m dv_harness.system_failure_taxonomy / python -m dv_harness.system_checker_taxonomy {types|classify}。

5.8 系統級就緒關卡 (system_readiness_gates.py)

system_readiness.derive_system_readiness() 已經把 SYS-37 的十項輸入(subsystem readiness、shared-resource conflicts、command compatibility、scoreboard compatibility、address map、clock/reset、VIP dedup resolution、build integration、scenario availability、regression evidence)折成一個 READY/PARTIAL/BLOCKED/UNKNOWN 總判斷,但這對「到底哪一項才是卡住的關鍵」而言太粗。system_readiness_gates.py 是純粹讀取層,對這十項輸入(加上 SYS-35 的 build_composition_version_pin() 之 restorable 欄位與 SYS-33 迴歸計畫的 entry_count)重新分組成八個具名的複合關卡: SUBSYSTEM_SELECTION_READY、RESOURCE_RECONCILIATION_READY、COMMAND_COMPATIBILITY_READY、SCOREBOARD_COMPOSITION_READY、BUILD_COMPOSITION_READY、REGRESSION_PLAN_READY、ERROR_HANDLING_READY、SYSTEM_SIGNOFF_READY (最後一項要求前七項全部 READY 且 SYS-37 本身的總判斷也是 READY)。它套用跟 subsystem_maturity_gate.py、functional_coverage_signoff.py 相同的worst-wins、絕不平均折算規則:一個關卡裡只要有一項 BLOCKED/CONCERN,整個關卡就是 NOT_READY,兩項乾淨一項卡住不算「大致就緒」;真正缺證據(沒被提供的輸入)則是第三種誠實狀態 INCOMPLETE_EVIDENCE,絕不悄悄變成 READY。這個模組不衍生任何新事實,只是唯讀地重組 SYS-37 已有的判斷,沒有 dv-harness CLI 動詞,前門是 system_readiness_gates.derive_system_readiness_gates()。一個 SYSTEM_SIGNOFF_READY 判斷跟 SYS-37 本身的 READY 一樣「不具行動力」——它是 SYS-39 人類核准關卡的輸入,絕不是替代品。

5.9 三層結案彙總:Subsystem Contract → System Verification Contract → System Closure Aggregator

這批模組的收尾是三層彙總,粒度由小到大:

subsystem_contract.py (單一 subsystem 範圍)把 spec/DUT/TB 身份(signoff_export. capture_baseline())、protocols/interfaces/vPlan 對應(env_manifest.load_env_manifest())、requirements(requirement_contract.execute_verb())、regression(regression_reporter. load_jobs())、signoff(signoff_export.read_signoff_stage_status() 加上凍結基準線)、evidence 與 reproducibility capsule(golden_scenario.open_store())、waivers(waiver_store. status_report())彙整成一筆記錄,unknowns 是誠實面,completeness 分 COMPLETE/PARTIAL/ NOT_AVAILABLE。CLI: python -m dv_harness.subsystem_contract assemble|snapshot [--subsystem ...] [--json] (assemble 只讀, snapshot 才會真的寫入 .dv-harness/subsystem_contract.json)。

system_verification_contract.py 是它的「手足彙總」——把 N 份 subsystem contract,加上 system_topology_analysis.py、system_resource_inventory.py、system_command_plan.py 已經算好的跨 subsystem 事實,組成一筆系統級記錄。它是純彙總,完全不 import 這四個被彙總的模組本身,而是接受它們輸出形狀的 duck-typed 參數,並用一支 AST 測試斷言原始碼裡真的沒有這四個模組名稱出現在 import 陳述式裡。unknowns 針對固定

四項(subsystem_contracts、system_topology、system_resource_registry、system_command_registry)計分, completeness 同樣是 COMPLETE/PARTIAL/NOT_AVAILABLE 三態。CLI:

```
python -m dv_harness.system_verification_contract assemble|snapshot \
  [--subsystem-contracts <file>] [--topology <file>] \
  [--resource-registry <file>] [--command-registry <file>] \
  [--system-name ...] [--json]
```

揭露的邊界:這是純彙總模組,自己完全不去讀 env.manifest.json、evidence database 或 subsystem registry——四項輸入都必須由呼叫端(Python API 傳 dict,或 CLI 傳 JSON 檔)已經組好才行;它不做任何跨 subsystem 仲裁,一個 ACTIVE_DRIVER_CONFLICT 的 preferred_model 一樣只是原封不動帶過去給人類看的文字。目前沒有 dv-harness CLI 動詞、沒有 dashboard card、沒有 graph node——這是一個「已可觸及但尚未接線」(REACHED, not WIRED)的能力。

system_closure_aggregator.py 再往上一層,把十二個固定的結案維度(functional_coverage、protocol_coverage、requirement_closure、waiver_status、regression_status、evidence_integrity、error_propagation、build_composition、performance_closure、security_closure、arbitration_closure、change_impact_closure)捲成一個 SYSTEM_CLOSURE_STATUS。它同樣完全不 import 任何一個真正決定這些維度狀態的模組(functional_coverage_signoff.py、waiver_store.py、system_error_propagation.py 等等),每個維度只接受呼叫端提供的 {dimension_name, status} 這種 duck-typed 記錄。核心規則是 strict worst-wins,絕不平均:任何一個維度回報「未達成(UNMET)」就讓整體結果變 NOT_CLOSED,就算另外十一項全部乾淨,十一比十二仍然是 NOT_CLOSED;只有在沒有任何 UNMET 時,一個 UNKNOWN/NOT_AVAILABLE 或完全沒被提供的維度才讓整體變成第三種狀態 INCOMPLETE_EVIDENCE,跟 CLOSED、NOT_CLOSED 都不同。NOT_APPLICABLE 是唯一「不算缺證據」就能清除的狀態——代表呼叫端真的宣告「這個維度在此不適用」,不是沒人去看。十二個維度永遠逐一列出,絕不折成一個裸的通過/失敗 計數。CLI:

```
python -m dv_harness.system_closure_aggregator --dimensions <file.json> [--markdown]
```

exit code:0= CLOSED、1= NOT_CLOSED、2= INCOMPLETE_EVIDENCE。同樣沒有 dv-harness CLI 動詞,它決定的一切都只是給人類做結案/簽核審查的輸入,不是開卡本身。

5.10 單一 subsystem 內的擁有權與匯流排仲裁檢查

最後兩個模組不是跨 subsystem,而是單一 subsystem/單一環境內部更細的資源安全檢查,容易被誤認為 system_resource_inventory.py 的重複功能,因此必須分清楚:

- ip_ownership_conflict.py 回答「這一個 subsystem 自己的環境裡,是否有一個真實 VIP agent 和一個手寫的 legacy BFM/driver 同時 ACTIVE 掛在同一個介面上?」這比 SYS-11/SYS-12 的跨 subsystem ACTIVE_DRIVER_CONFLICT 窄:SYS-11 需要「第二個 subsystem 的資源」才有東西可以比對,而這個模組在 SoC 組合根本還不存在、純粹 IP-level intake 階段就能查。它重用 system_resource_inventory.REL_DRIVER_CONFLICT 與 INTEGRATION_STOPPED / SYS12_PREFERRED_MODEL 原封不動,不另立第二套詞彙。報告狀態是 CONFLICT/CLEAR/ NOT_APPLICABLE/UNKNOWN 四態,前門是:

```
python -m dv_harness.ip_ownership_conflict --env-manifest <FILE> \
  [--legacy-bfm <FILE>] [--connectivity-rows <FILE>] [--json]
```

exit code:0=CLEAR、1=CONFLICT、2=NOT_APPLICABLE/UNKNOWN。

- shared_bus_resource_registry.py 則是另一條軸線:同一個環境執行期間,branch_fw 的 per-port 事件服務迴圈與某個 sibling port 的 branch_a{i} 初始化,是否可能同時對同一個共享 匯流排資源(共享的 PHY config block、共享 reset controller、被仲裁的 AXI/APB slave)發出暫存器存取——這是 pattern-architecture skill 3.1 節講的「兩個獨立 task group 用不同名的 lock/semaphore 碰同一個實體匯流排 sequencer,可能被仲裁層用作者都無法控制的順序交錯,悄悄覆寫對方的寫入」這種併發競爭類別。system_resource_inventory.py 和 ip_ownership_conflict.py 都不建模「命名鎖」,也都不問「一個 pattern 內兩個 task group 能否同時搆到同一資源」——這正是本模組要補的缺口。由於整個 dv_harness/ 裡沒有任何 SystemVerilog pattern-body parser,「哪個 task group 在哪個命名鎖下寫哪個資源」誠實地是呼叫端宣告的事實(resource_declarations),每一筆宣告都必須附帶真實的 evidence 引用(pattern 檔案的 file:line,或 RTL/PHY 文件行號),絕不用猜的。報告狀態是 CONTENTION/CLEAR/NOT_APPLICABLE/ UNKNOWN,衝突種類目前只有一種具名值 FW_A_RACE。前門:

```
python -m dv_harness.shared_bus_resource_registry \
  [--resource-declarations <FILE>] [--connectivity-rows <FILE>] [--json]
```

exit code:0=CLEAR、1=CONTENTION、2=NOT_APPLICABLE/UNKNOWN。

5.11 這一層整體的邊界

把這一整章的模組放在一起看,共同的邊界很清楚:它們合力把「這個系統整合了嗎、可以簽核了嗎」這個問題的每一個子面向都變成可查詢、可重現的真實資料,但沒有一個模組會替你按下建置、模擬、送 job、或簽核的按鈕。真正會生成 System-Level UVM 原始碼、System command.txt、共享 sequencer/driver,或替 ACTIVE driver 衝突選出贏家的動作,全部落在 SYS-40,而 SYS-40 本身被 SYS-39 這個明確的人類核准關卡擋住。使用者若在 dv-harness system-phase1-report 這個涵蓋 SYS-33..39 的指令下取得一份 READY 的 Phase-1 報告,它代表的是「以現有真實證據來看,整合看起來沒有已知阻塞」,而不是「系統已經整合完成」——下一步永遠是人,而不是這個 harness 自己。

6. Spec-to-VPlan 編譯流程 (需求萃取、衝突偵測、風險評分、vPlan 完整性)

「Spec-to-VPlan」在本專案中不是單一模組,而是一條由多個各司其職、彼此以真實共用詞彙串接的模組所組成的管線:從一份規格文件(spec/datasheet)的結構索引出發,經過原子需求(atomic requirement)的契約化與風險評分,到跨來源證據比對、語意橋接、vPlan 完整性分析、語意差異(delta)偵測、結構性缺口偵測,最後匯總成一個 SPEC_VPLAN_READY 複合就緒判定。這條管線刻意遵守本專案的 Evidence Truth Rule:凡是「讀懂一份規格文件、把散文轉成需求」這種本質上屬於 LLM 閱讀理解的行為,程式碼從不假裝自己能「計算」出來;程式碼只做兩件事——建立文件的機械式結構索引,以及對任何 agent(或未來的萃取器)所宣稱的結果做真實的驗證/再推導(re-derivation)。以下依管線順序介紹每個環節。

6.1 文件結構索引:spec_doc_map 與 SpecMap

`dv_harness/spec_doc_map.py` 是針對「非 VIP 的 DUT 文件」(規格書、datasheet、programming guide)的離線結構萃取器,與既有的 `vip_user_guide_distill.py` 走同一套紀律,但刻意分屬不同模組、不共用私有函式,只產出結構、絕不輸出全文:章節標題索引(heading/number/level/page)、Table N.M 表格標號位置、暫存器章節的頁碼範圍。輸出的 `<stem>.structure_map.json` / `.structure_map.md` 兩份產物中,永遠不會出現文件本文的任何一句話。

```
python -m dv_harness.spec_doc_map extract --source <spec.pdf|spec.txt> --out-dir <dir> [--title] [--doc-kind dut_spec] [--json]
```

`dv_harness/spec_intelligence.py` 則在此之上疊加一層 SpecMap:它不重新做 PDF/文字解析,而是呼叫 `build_spec_map()` 讀回 `vip_user_guide_distill.py` 自己算好的 `.reference.md` 章節表,再加兩個新的機械掃描(表格標號位置、暫存器章節關鍵字判斷)。SpecMap 因此只有位置資訊(章節、表格、暫存器章節的頁碼/字元偏移),永遠不含本文。ad hoc 前門是 `python -m dv_harness.spec_intelligence spec-map|analyze`,目前尚未接上 `dv-harness` CLI 動詞或任何 stage gate。

6.2 Canonical Requirement Contract:十五欄位契約與五值狀態詞彙

`dv_harness/requirement_contract.py` 是 spec section 184 的「Canonical Requirement Contract」實作,是這條管線的核心地基。它把舊有的五欄位品質閘(`spec_ref / feature / expected_behavior / verification_method / coverage_goal`)擴充為十五個欄位——多出的八個是 Protocol、Configuration、Precondition、Observability、Checker、Coverage Intent、Priority、Criticality,並附上五值狀態詞彙 COMPLETE / PARTIAL / AMBIGUOUS / CONTRADICTION / UNKNOWN。

關鍵在於: `status` 是 agent 對自己萃取工作的自我宣稱,依照 Evidence Truth Rule 屬於「判斷」而非「證據」,所以 `derive_status()` 會從紀錄本身的内容重新推導狀態, `analyze_requirement_contract()` 再檢查宣稱的狀態是否被内容撐得住。兩條規則承擔主要重量:

- **STATUS_OVERCLAIMED**——宣稱 COMPLETE,但某欄位仍未解(空白、UNKNOWN / TBD / N/A 等佔位字)。
- **UNRESOLVED_BLOCKER_HIDDEN**——紀錄本身已填了未解的 ambiguity 或 contradiction,卻宣稱一個既非 AMBIGUOUS 也非 CONTRADICTION 的狀態,等於把衝突悄悄掩過去。

宣稱比實際内容「更保守」的狀態永遠合法(只回報 WARNING),但仲裁不在這裡發生:一條 CONTRADICTION 的需求就停在 CONTRADICTION,模組只點名互相矛盾的來源,絕不替人類決定哪一方對。只有 COMPLETE 且零 ERROR 的紀錄才能被 `downstream_consumable()` 判定為可供下游生成器使用。

```
dv-harness requirement-contract --requirements <file> [--json] [--fail-on-error]
python -m dv_harness.requirement_contract
```

(exit 0 乾淨、1 有 ERROR、2 NOT_AVAILABLE——本契約下沒有「什麼都沒查」卻算乾淨過關這回事。)

揭露的邊界:這個模組只讀取一筆需求紀錄本身,不解析規格文件、不從散文萃取需求、不對照 RTL/暫存器映射/模擬結果檢查需求。且目前本專案沒有任何生成器真的產出契約形狀的紀錄——閘門是「已就位但尚未被餵資料」的狀態。

6.3 Requirement Risk IR:六因子風險評分,誠實區分「量測」與「宣稱」

`dv_harness/requirement_risk_ir.py` 為單一需求算出六因子風險分數(complexity、change_frequency、bug_history、customer_impact、observability_difficulty、protocol_criticality),並刻意把六個因子分成三種誠實等級:

- **MEASURED**——只有 `change_frequency` 有真實機械來源:對需求對應的原始檔跑 `git log --oneline`,若專案不是 git repo、檔案未知或 git 逾時,一律降級為 NOT_AVAILABLE,絕不猜一個分數。
- **DECLARED**——complexity、customer_impact、observability_difficulty、protocol_criticality 在本專案中沒有任何機械產生器,因此接受呼叫端明確宣告的 1-5 值,並標記狀態為 DECLARED,與 MEASURED 明顯區分開來。
- **NOT_AVAILABLE**——bug_history 在本專案中完全沒有缺陷追蹤系統可查,永遠回報 NOT_AVAILABLE,不接受宣告、也不編造。

複合分數是「可用因子」的平均值,並且一定同時回報 `available_factor_count` 與 `missing_factors`,讓呼叫端不可能把部分分數誤當成完整分數。此模組本身不指派優先權、不做任何閘門判定,也不寫入任何儲存區。

6.4 跨來源證據比對:dut_evidence_correlation 與 design_knowledge_correlation

`dv_harness/dut_evidence_correlation.py` 把需求宣稱的 DUT 事實(中斷、時脈/重置訊號、模式、功能)與 `env.manifest.json` 的 `dut_facts` 層(RTL ports/signals/parameters、暫存器欄位、時脈重置、位址映射)做名稱比對,產出五值判定:RTL_CONFIRMED(精確比對且屬性一致)、RTL_CONTRADICTS_SPEC(精確比對但屬性——如 active_level、frequency_mhz、access——不符,只回報矛盾、不裁決誰對)、RTL_PARTIAL(僅子字串比對,未證實的可能對應)、RTL_NOT_FOUND(真的查過但找不到)、NOT_AVAILABLE(該層根本無法查)。它只做名稱比對(精確或大小寫不敏感的子字串),沒有語意模糊匹配,也沒有同義詞表;呼叫端必須自己把契約裡的散文欄位轉成候選名稱。

`dv_harness/design_knowledge_correlation.py` 則是更一般化的引擎:接受任意數量、任意角色(SPEC_DECLARATION、IMPLEMENTATION_EVIDENCE 等)的「知識來源」,找出 CONFLICT(多來源對同一 `fact_key` 給出不一致的值)、GAP(呼叫端預期某來源該涵蓋卻無人涵蓋)、DOCUMENTED_VS_IMPLEMENTED(規格宣告卻無 RTL 證據,或反之)。它的比對鍵是呼叫端提供的已規範化 `fact_key`,刻意不做模糊比對——避免製造出兩個本不該視為同一事實的假衝突。

6.5 Verification Intent IR:需求到生成器之間的語意橋接

`dv_harness/verification_intent_ir.py` 把一筆 COMPLETE 的需求契約轉成生成器(vPlan 撰寫器、情境規劃器、checker/coverage 產生器)真正需要的「詮釋性」形狀:測試目標是什麼、應驅動什麼刺激、應由什麼檢查、應涵蓋什麼覆蓋率,並跨七個結構性領域(state_machine、register_csr、interrupt、reset_clock、low_power、error_recovery、performance)分別解讀。其中五個領域真的接上既有的真實證據產生器(protocol_capability、sys_regmap、interrupt_dma_clock_reset_extraction、power_intent),error_recovery 與 performance 則因本專案完全沒有對應的產生器,永遠誠實回報 ERROR_TARGET_UNKNOWN / PERFORMANCE_TARGET_UNKNOWN,即使其他領域都已提供真實證據也不會被連帶「腦補」出目標值。

由於把需求解讀成「該驅動什麼、該檢查什麼」本質上是詮釋而非量測,每一筆 IR 紀錄都固定標記 `evidence_provenance.AGENT_SELF_ATTESTED`,這是建構子強制寫死的,呼叫端無法覆寫。

```
python -m dv_harness.verification_intent_ir --requirements <file> [--source-paths ...] [--sys-regmap ...] [--upf ...] [--json]
```

6.6 vPlan Artifact:九維度完整性,永不折算成單一分數

`dv_harness/vplan_artifact.py` 把 vPlan(章節/功能容器 + 章節點驗證項目的階層)變成可檢查的產物: `build_vplan_hierarchy()` 偵測重複 id、孤兒父節點參照、循環, `analyze_vplan_completeness()` 則對九個獨立維度分別評分——HIERARCHY_INTEGRITY、REQUIREMENT_COVERAGE、VERIFICATION_METHOD_ASSIGNMENT、COVERAGE_MODEL_LINKAGE、CHECKER_LINKAGE、TEST_STIMULUS_LINKAGE、OWNERSHIP_ASSIGNMENT、PRIORITY_ASSIGNMENT、INTENT_CROSS_CONSISTENCY,每一維各自 READY/PARTIAL/BLOCKED/UNKNOWN,絕不平均或加權折算成一個總分。十五值缺口分類(`GAP_TAXONOMY`)各自對應到一個維度與一個嚴重度(BLOCKED 為結構性/虛假宣稱,PARTIAL 為誠實的缺項),並透過 `inference.next_best_action()` 給出下一步建議動作——這裡沒有重造 Gap→Next-Best-Action 引擎,只新增一張查找表 `VPLAN_GAP_ACTION_CATALOG`。純 FORMAL 驗證方法的章節點,coverage/checker/test-stimulus 三個維度會誠實回報 UNKNOWN、零適用列,而非硬湊一個 READY。此模組只讀取資料,不寫任何檔案、不執行任何 stage、也沒有 `STAGE_GATES` 條目。

6.7 Spec/vPlan Semantic Delta:逐需求的 ADDED/MODIFIED/REMOVED/REVALIDATION_REQUIRED

`dv_harness/spec_vplan_delta.py` 比較兩份需求 IR 快照(`before / after`),判斷每一筆需求是 ADDED、REMOVED、MODIFIED(行為內容欄位有變)、REVALIDATION_REQUIRED(內容位元組相同,但來源/信心/優先權等佐證欄位變了,行為沒人改寫但佐證本身動了),或 UNCHANGED。這與 `change_impact.py` 的檔案級 git diff 是完全不同的軸線——規格文件可能活在本 repo 的 git 歷史之外,一次規格改版可以改寫需求行為卻在本專案留下零 git diff, `change_impact.py` 對此結構性地看不見。若呼叫端提供 `root`,每筆非 UNCHANGED 需求還會對照真實的 `.dv-harness/requirements.csv` 追溯登錄檔,標出 LINKED / NO_REGISTRY_ROW / NOT_REQUESTED 三種誠實結果。

```
python -m dv_harness.spec_vplan_delta --before <baseline.json> --after <current.json> [--root <dir>] [--json]
```

(exit 0 NO_DELTA、1 DELTA_FOUND、2 NOT_AVAILABLE——兩份快照都是空的絕不算乾淨過關。)

6.8 Potential Spec-Gap Detector:五種結構性缺失樣式

`dv_harness/potential_spec_gap_detector.py` 處理一個前面模組都沒問過的問題:規格書逐句都通順,卻可能整體結構不完整——定義了傳輸完成卻沒定義出錯、定義了 enable 卻沒定義 disable、定義了中斷觸發卻沒定義如何清除、定義了重置卻沒交代進行中操作怎麼辦、定義了錯誤條件卻沒定義復原路徑。它對需求集合做關鍵字掃描分類(九個固定詞彙旗標,每個都帶精確比對到的字詞作為證據),再用「同一主體」的 Jaccard 重疊度測試配對——刻意保守,避免把僅共享常見規格散文詞彙的兩條無關需求誤配成假缺口。每一個結果都結構性地被鎖死為 `POTENTIAL_SPEC_GAP` 這個唯一字串(`SpecGapFinding.__post_init__` 拒絕任何其他狀態),絕不會被誤讀成已證實的缺失或自動晉升為核准需求。

```
python -m dv_harness.potential_spec_gap_detector <requirements.json> [--json]
```

6.9 SPEC_VPLAN_READY:Spec-to-vPlan 階段的複合就緒判定

`dv_harness/spec_vplan_readiness_gate.py` 是專屬於「spec-to-vplan 這一個管線階段」的複合就緒閘,與專案級的 `INTAKE_READY`、子系統級的 9.0/9.5/10.0 成熟度階梯是三個獨立、範圍不同的機制,彼此互不匯入。它採用嚴格的兩層 worst-wins 摺疊:條件狀態詞彙 MET/UNMET/UNKNOWN/NOT_AVAILABLE,任何一個 UNMET 就讓整體判為 NOT_QUALIFIED(即使另外二十個條件都乾淨),只有在沒有 UNMET 時,UNKNOWN/NOT_AVAILABLE 才會把結果降為 INCOMPLETE_EVIDENCE——絕不平均、絕不折衷。條件集合完全由呼叫端命名與提供,模組本身不硬編任何一條「spec-to-vplan 完成」該檢查什麼。

```
python -m dv_harness.spec_vplan_readiness_gate statuses|verdicts|evaluate --conditions <file> [--json]
```

(exit 0 QUALIFIED、1 NOT_QUALIFIED、2 INCOMPLETE_EVIDENCE 或輸入錯誤。)

6.10 這條管線刻意不做事

整條管線有一致的邊界紀律,值得在使用前明確認知:(1) 沒有任何模組真的「讀懂」一份規格文件或從散文萃取需求——那一步永遠是 agent 或人類的自我宣稱工作,程式碼只驗證其結果是否自洽;(2) 沒有任何一步做仲裁——CONTRADICTORY 的需求、CONFLICTS_WITH 的關係、RTL_CONTRADICTS_SPEC 的比對結果,一律停在「已命名衝突」,由人類決定誰對;(3) 除 `requirement-contract` (已有 `dv-harness CLI 動詞`)外,本章介紹的其餘模組(spec_intelligence、verification_intent_ir、vplan_artifact、spec_vplan_delta、potential_spec_gap_detector、spec_vplan_readiness_gate、dut_evidence_correlation、

spec_doc_map)目前都只是「已可達(REACHED)但尚未接線(WIRED)」的能力——沒有 `dv-harness` 子命令、沒有 `run_stage()` / `advance()` 呼叫點、也沒有 `STAGE_GATES` 條目,前門一律是各自的 `python -m dv_harness.<module>` ;(4) 任何一份完整性/就緒報告都只是輸入給人類審查 vPlan 或核准 spec-to-vplan 階段的決策依據,絕非替代該決策本身——`ControlPlane.approve()`、`policy.can_signoff()`、`assert_human_approval()` 等治理機制在這整條管線中完全未被觸碰。

7. DE Command / Runtime 架構(指令風格學習、分支所有權、事件登錄、Pattern IR)

本章涵蓋 `dv_harness/` 底下一整組圍繞著 DE(Design/Verification Engineer)手寫 `command.txt` / `pattern` 檔案而生的模組群。這些模組彼此之間刻意保持「窄範圍、可疊加、絕不互相假設對方的欄位形狀」的設計原則——每一個模組的 `CLAUDE.md` 說明段落與原始檔頭 `docstring` 都會明確寫出「這個模組不是誰、不會 import 誰」,這是刻意的架構決策,不是遺漏。閱讀本章時請記住這批模組全部建立在 `.claude/skills/CORE/pattern-architecture/SKILL.md` 與 `.claude/skills/CORE/branch-mapper/SKILL.md` 已經定義好的四層 **task-composition 詞彙**之上——`block` (SoC 全域、一次性、non-per-port 的 bring-up)、`branch_a*` (每個 port 一個的 DUT+PHY 初始化分支)、`branch_fw` (每個 port 一個、只啟動一次、永不返回的 FW 事件服務迴圈)、`branch_b*` (每個 port 一個、由 VIP 驅動的實際測試本體,在 `pattern` 頂層 fork/join)。本章所有模組都是「操作化」(operationalize)這套既有詞彙,而不是發明新詞彙。

7.1 DE Command-Style Learning:學會一份 `command.txt` 自己的寫作風格

`dv_harness/de_command_style_learning.py` 回答一個 `reference_pattern_audit.py` 的 SYS-7 層從未問過的問題:不是「這一行陳述式在做什麼」(那是 SYS-7 `extract_command_statements()` / `classify_wait()` 的工作,本模組直接 reuse 而不重新推導),而是兩個新問題:

- CommandStyleIR** —— 這份具體、真實的 `command.txt` 檔案本身,實際使用的是什麼「格式慣例」?涵蓋 `separator_convention` (分號結尾單行 vs. 多陳述式/多行)、`argument_format` (括號短號呼叫風格、hex 常數是否加底線分組)、`comment_format` (行尾註解 vs. 獨立區塊註解)、`phase_markers` (真正的 `PHASE: / STAGE: / STEP: / SECTION:` 標記,而非僅僅包含 "stage" 子字串的普通單字)、`ordering_rules` (真正的 `fork / join` 關鍵字與註解層級的排序語言)。每個面向都是從檔案*自己的文字*偵測出來,絕不假設沿用其他專案的慣例,也絕不硬編碼成 SYS-7 當初驗證時所根據的 USB 巨集慣用語。任何面向若找不到證據,就誠實回報 `NOT_FOUND / NOT_AVAILABLE`,並附上實際執行過的搜尋動作。
- DECommandRegistryIR** —— 針對檔案中每一個「相異」的指令(依 `(kind, name)` 分組,確保兩個不同巨集永遠不會被誤併成一筆,而無法分類的行也只會跟逐字相同的另一行歸為一組)建立一筆 `DECommandEntry`,內含 best-effort 的 `semantic_operation`、`arguments`、一個 `branch_owner` 猜測值 (`GLOBAL / DUT / FW / VIP / UNKNOWN`,對應 `block / branch_a* / branch_fw / branch_b*` 四層),以及一個 `status` (`KNOWN / PARTIAL / AMBIGUOUS / UNSUPPORTED / DEPRECATED / UNKNOWN`)。 `KNOWN` 需要真正引用得到的文字特徵(例如 `HOSTWRITE* / CPUMWRITE*` 前綴,或是 `INTERRUPT_NAME_TOKENS` 命中);註解中出現真正的 `deprecation` 關鍵字(`deprecated/obsolete/do not use`)會讓整筆指令標記為 `DEPRECATED`,並引用實際命中的註解與關鍵字——這個判斷優先於一般的「跨出現次數分類不一致 → 降級為 `AMBIGUOUS`」規則,因為明確的棄用聲明是比單純分類分歧更強的證據。

呼叫方式是純函式庫 API: `build_de_command_registry(path, statements=None)`、`learn_command_style(path)`、`analyze_de_command_file(path)`、`format_de_command_analysis(analysis)`。這個模組沒有掛上 `dv-harness` CLI 動詞,也沒有自己的 `python -m` 前門——它被 `de_command_review_package.py` 等下游模組以函式庫方式呼叫的第二層分析引擎。

7.2 Branch Ownership Resolver:把「哪個分支該做這件事」變成可檢查的規則

`CLAUDE.md` 的 Engineering Discipline Rules 早已用文字描述過「Concurrent bus arbitration」與「Architecture-conformance audit」規則,但從未有程式碼真正檢查過。`dv_harness/branch_ownership_resolver.py` 補上這一塊,提供兩個函式:

`classify_operation_ownership(operation_kind, per_port=, driven_by=, arbitration_policy=)` 把一個「打算執行的動作」分類為 `GLOBAL / DUT / FW / VIP`,依據五種固定分層的 operation kind(`SOC_GLOBAL_ONE_SHOT_INIT`、`DUT_PHY_PORT_BRINGUP`、`FW_EVENT_SERVICE_LOOP`、`VIP_DRIVEN_TEST_BODY`、`VIP_DRIVEN_DATA_TRANSFER`,每一個都直接引用 `pattern-architecture SKILL.md` 第 1 節)以及兩種「情境相依」的 kind(`RAW_DUT_REGISTER_WRITE`、`SHARED_RESOURCE_ARBITRATED_ACCESS`),後者的分層取決於呼叫方宣告的 `per_port / driven_by / arbitration_policy` 事實;缺漏或矛盾時,結果一律是 `AMBIGUOUS` (並具名指出缺了哪個事實),絕不猜測一個分層。分類範圍以外的 operation kind 回報 `UNKNOWN`。

`validate_branch_assignment(branch_label, operation_kind, ...)` 則驗證一個既有的分支指派:先檢查 `branch_label` 是否符合 `branch_topology_gate.py` 已經建立的正典命名(`block / branch_a{i}` / `branch_fw / branch_b{i}` ,底線分隔、0-index),不合法就直接判 `INVALID` (`ARCH_CONFORMANCE_NAMING_VIOLATION`),與該操作實際內容無關;接著比對操作分類的分層與分支標籤暗示的分層,不一致時回報 `INVALID` 並具名規則(如 `VIP_DRIVEN_WORK_ASSIGNED_TO_BRANCH_A`、`RAW_DUT_OPERATION_ASSIGNED_TO_BRANCH_B`、`FW_SERVICE_LOOP_DUPLICATED_IN_BRANCH_B`),一致則 `VALID`,操作本身無法判定則 `AMBIGUOUS`。

刻意受限之處:本模組不讀 RTL、不讀 VIP index、不讀 `command.txt` 檔案本身——一個操作「究竟是不是 VIP 驅動」、「是不是 per-port」、「誰發出的」全部是呼叫方從自己真實證據宣告出的輸入,絕非本模組自行推導(依 Evidence Truth Rule,本專案沒有任何來源能單從一個裸暫存器位址或巨集名稱判斷出是哪個 task group 發出的寫入)。它只回報,不做任何 build/job/approval,也刻意沒有 stage gate;也不檢查分支「集合」是否完整(那仍是 `branch_topology_gate.py` 的工作),也不驗證仲裁政策本身是否真的對應到 RTL(`SHARED_RESOURCE_ARBITRATED_ACCESS` 只要政策被宣告就會被解析,是否正確對應真實仲裁器 RTL 不在此檢查範圍)。

```
python -m dv_harness.branch_ownership_resolver classify|validate --payload <json>
dv-harness branch-ownership-resolver --payload <json> classify|validate
```

`classify` 的 exit code:0=RESOLVED,1=AMBIGUOUS,2=UNKNOWN 或用法錯誤; `validate` :0=VALID,1=INVALID,2=AMBIGUOUS。

校對更正:草稿原先寫「沒有掛上 `dv-harness` CLI 動詞」,這在撰寫當時是正確的,但 `CLAUDE.md` 記錄的「CLI Wiring: 111 Modules Connected to dv-harness」批次(2026-09-06)已把本模組接上 `dv-harness branch-ownership-resolver` 動詞(以 `python -m dv_harness.cli branch-ownership-resolver --help` 現場驗證確認), `python -m` 前門仍然可用、行為不變。

7.3 Runtime Event Registry:具依賴圖的「一旦上游失敗就停止」事件登錄

生成 `pattern` 的 task composition(`block / branch_a* / branch_fw / branch_b*`)與其中斷驅動服務迴圈(ARM/WAIT/WAKE/DECODE/CLEAR)都會產生與消費「具名的 runtime 事件」——全域 bring-up 完成、某 port 的 DUT+PHY 初始化完成、中斷被看到後被服務、某 `branch_b*` VIP 場景完成檢查。過去

`blackboard.py` 只存自由格式的 named topic,不是有 producer/consumer/timeout/scope 的事件; `loop_contract.py` / `loop_budget.py` 追蹤的是 harness 自己的迴圈狀態,不是生成環境的 runtime 事件流。 `dv_harness/runtime_event_registry.py` 補上這塊。

記錄結構固定為 {event_name, producer, consumer, payload, timeout, scope, status}, 事件集合由呼叫方宣告

(GLOBAL_READY / DUT_READY / VIP_STARTED / IRQ_SEEN / IRQ_SERVICED / CHECK_DONE 僅為範例,絕非硬編碼)。四種關係: REQUIRES / WAITS_FOR 是「依賴方 → 前置條件」方向(B REQUIRES A 表示 B 依賴 A 先發生); TRIGGERS / UNBLOCKS 是「原因 → 結果」方向。 REQUIRES 是硬依賴, UNBLOCKS 是復原覆寫, WAITS_FOR / TRIGGERS 僅供參考(產生 AT_RISK_WAITS_FOR_FAILED_UPSTREAM / ORPHANED_TRIGGER 提示,不改變任何狀態)。

`propagate()` 是對 REQUIRES 子圖做定點運算(在建構時就驗證過必須是無環圖,否則直接拒絕);一個事件會從 PENDING 翻轉成 BLOCKED_BY_DEPENDENCY,條件是:自身原始狀態仍為 PENDING、至少一個 REQUIRES 前置條件已經(遞移地) FAILED / TIMEOUT / BLOCKED_BY_DEPENDENCY,且沒有任何 UNBLOCKS 復原事件已 FIRED。真實觀測到的狀態永遠不會被計算出的狀態覆寫。Evidence Truth Rule 在此以結構方式強制:任何非 PENDING 的觀測(FIRED / FAILED / TIMEOUT)都必須附帶非空的 evidence 引用(sim.log 行、evidence_db 記錄 id、波形位移); BLOCKED_BY_DEPENDENCY 永遠不可由呼叫方直接宣告,只能是 `propagate()` 自己算出的結論。

```
python -m dv_harness.runtime_event_registry graph|status --registry <file.json> [--json]
dv-harness runtime-events --registry <file.json> [--out OUT] [--json] graph|status
```

`graph` 只印出宣告的事件/關係; `status` 執行 `propagate()` 並印出 effective 狀態與 finding 表格。Exit 0=沒有任何事件 FAILED/TIMEOUT/BLOCKED_BY_DEPENDENCY,1=至少一個是,2=NOT_AVAILABLE 或宣告錯誤。

校對更正:CLAUDE.md 的「CLI Wiring: 111 Modules Connected to dv-harness」批次(2026-09-06)已把本模組接上 `dv-harness` CLI 動詞——但要特別注意動詞名稱是 `runtime-events`,不是模組名稱去底線化後的 `runtime-event-registry` (以 `python -m dv_harness.cli --help` 現場驗證確認),這不是單純的「舊寫法還沒補上動詞」,而是動詞命名本身就與模組名不同,容易猜錯。

刻意受限:本模組不觀測任何東西——事件「是否真的發生」由呼叫方從真實 sim.log/波形/evidence-db 證據決定後再呼叫; UNBLOCKS 是唯一建模的復原機制,沒有 any-of/all-of REQUIRES 的區分;它只回報,不做 gate、不做 build/job/approval。

7.4 Command Precondition Gate:在派送前問「這個指令現在可以跑嗎」

`runtime_event_registry.py` 回答「這個事件現在狀態如何」,但沒有回答一個 dispatcher 在真正派送 `block / branch_a* / branch_fw / branch_b*` 任務之前必須問的鄰接問題:「這個指令自己宣告的 precondition 集合,現在是否成立?」 `dv_harness/command_precondition_gate.py` 正是這一塊的橋接,而且刻意 reuse 而非重造:每個指令宣告自己的 precondition 名稱集合(GLOBAL_READY / DUT_READY / FW_READY / VIP_READY / MODE_VALID / RESET_DEASSERTED / PHY_READY 僅為範例),各自對照一次 `RuntimeEventRegistry.propagate()` 算出的 EFFECTIVE(propagation 之後)狀態。

詞彙固定為 READY / BLOCKED / UNKNOWN_PRECONDITION (與 `EventStatus`、`command_error_taxonomy` 的十一類詞彙皆刻意不相交,並在 import 時以 `assert_no_dispatch_status_vocabulary_collision()` 檢查)。每個指令的狀態依「最差優先」折疊:任何 precondition 指向的事件已知且為 FAILED / TIMEOUT / BLOCKED_BY_DEPENDENCY → BLOCKED;否則任何 precondition 指向的事件根本不在登錄中 → UNKNOWN_PRECONDITION (絕不默認為已滿足);否則任何已知事件仍 PENDING → BLOCKED (經典的「還在等一個尚未發生的條件」語意);否則(每個宣告的 precondition 都已 FIRED,或根本沒宣告任何 precondition)→ READY。每個判斷都附帶完整的逐條 precondition 證據(known / effective_status / raw_status / reason)。

```
python -m dv_harness.command_precondition_gate list|check --commands <commands.json> [--registry <events.json>] [--out <path>] [--json]
dv-harness command-precondition-gate --commands <commands.json> [--registry <events.json>] [--out <path>] [--json] list|check
```

`list` 只需要指令宣告; `check` 對整個登錄執行一次 `propagate()`,再對每個宣告的指令評估同一份狀態。Exit 0=全部 READY,1=至少一個 BLOCKED/UNKNOWN_PRECONDITION,2=NOT_AVAILABLE 或用法/宣告錯誤。

校對更正:CLAUDE.md 的「CLI Wiring: 111 Modules Connected to dv-harness」批次(2026-09-06)已把本模組接上 `dv-harness command-precondition-gate` 動詞(以 `python -m dv_harness.cli command-precondition-gate --help` 現場驗證確認)。

刻意受限:precondition 名稱是精確字串比對,沒有模糊或前綴比對、沒有別名表;它評估的是「某一時間點」的登錄狀態,不是訂閱或輪詢器,也不保留歷史。

7.5 DE Command Runtime Readiness Gate:把三個真實來源合併成一個判決

`de_command_runtime_readiness_gate.py` 是一個薄的聚合層,把三個各自獨立、各自真實的來源合而為一個 DE_COMMAND_RUNTIME_READY 判決:(1) `RuntimeEventRegistry.propagate()` 的結果(在此模組中只呼叫一次,絕不對每個指令重跑一次);(2) `command_precondition_gate.evaluate_command_preconditions()` 用同一份 propagation 報告評估出的每指令 `CommandDispatchStatus`; (3) 呼叫方提供、duck-typed 的 `branch_grammar_results` (例如 `branch_ownership_resolver.validate_branch_assignment()` 的 VALID/INVALID/AMBIGUOUS,或 `command_generation_gate.py` 的 PASS/BLOCKED 結果)——本模組刻意不 import 這批來源的任何模組,因為那是同一場工作中「可能還沒跑完」的另一個並行批次。整體判決規則:三個來源中任何一個回報真正的 blocker,整體就是 BLOCKED;只有當登錄無阻塞事件、每個宣告的指令都 READY、且每個提供的 branch/grammar 結果都是可辨識的 pass 狀態時才是 PASS。揭露的殘留邊界:本模組尚未在 `gates.py` 的 STAGE_GATES 中掛上條目,整合是後續步驟;它只聚合、不授權任何事情,也不引用任何 approval/governance 機制。

校對更正:CLAUDE.md 的「CLI Wiring: 111 Modules Connected to dv-harness」批次(2026-09-06)已把本模組接上 `dv-harness de-command-runtime-readiness-gate` 動詞(參數為 `--registry / --commands / --branch-grammar / --out / --json`,以 `python -m dv_harness.cli de-command-runtime-readiness-gate --help` 現場驗證確認)——但如上一段所述,這僅代表本模組現在有一個 CLI 前門, `gates.py` 整合(讓它真正參與某個 stage 的判決)仍是尚未完成的後續步驟,兩者不同層次的事。

7.6 Pattern Runtime Execution State Machine:單一 pattern 檔案自己的執行生命週期

`dv_harness/pattern_runtime_state_machine.py` 是一個「每個 pattern」的執行狀態機: CREATED -> PARSED -> VALIDATED -> READY -> RUNNING -> WAITING -> CHECKING -> PASS/FAIL/TIMEOUT/BLOCKED/CANCELLED,由 `assert_legal_transition()` / `advance_pattern_state()` 強制合法轉換(任何不在 LEGAL_TRANSITIONS 表

中的跳躍,例如 CREATED 直接跳到 PASS,一律拒絕並丟出 `IllegalPatternTransitionError`,終端狀態也不允許再轉換)。它追蹤的正是 `pattern-architecture/SKILL.md` 描述的顆粒度: `block -> branch_a* -> branch_fw -> branch_b*` 的 `fork/join -> verdict/FINAL_CHECK`。 `RUNNING` 合法允許跳過 `WAITING` 直接到 `CHECKING` (因為是否 `fork branch_b*` 依測試意圖而定), `WAITING` 永遠不會回到 `RUNNING`。

這是刻意與 `loop_contract.LoopState` 完全分離、沒有橋接的詞彙: `LoopState` 回答「整個驗證收斂流程(跨越許多 stage/pattern/regression 循環)現在處於哪個控制狀態」, `PatternRuntimeState` 回答「這個 pattern 現在執行到哪裡」——兩者連 `PASS/FAIL` 這種看似相同的字都不代表同一件事,因此沒有、也不打算有任何 `STATUS_TO_LOOP_STATE` 式的橋接。

`derive_observed_terminal_verdict(log_text)` 是本模組最重要的誠實性設計:它從一份真實 `sim.log` 讀出「這份 log 本身的證據支持什麼」——有真正的 `FINAL CHECK` epilogue `VERDICT: PASSED|FAILED` 行就用它;沒有的話,真正的 `fatal/error/scoreboard-mismatch/assertion` 標記回報 `FAIL`,真正的 `timeout/deadlock` 標記回報 `TIMEOUT`;一份乾淨結束、零錯誤、卻完全沒有陳述任何判決的 log,回報 `SILENT_FAILURE_SUSPECTED` (絕不默認為 `PASS`)——這正是 `pattern-architecture SKILL.md` 第 2 節所講的 `join/join_any` 靜默提早通過陷阱("run 乾淨結束、零錯誤,只因為根本沒有東西有機會失敗")在 `runtime` 觀測層的具體體現;空白 log 文字回報 `NOT_AVAILABLE`。 `apply_observed_verdict()` 把這個觀測與強制轉換層組合起來:沒有足夠證據支持任何終端判決時拒絕猜測(回傳未套用、記錄不變),而且即使有真實的終端證據,仍然強制檢查 `LEGAL_TRANSITIONS` ——擁有「結局」的證據,不代表可以跳過紀錄中的「中段」。

前門: `python -m dv_harness.pattern_runtime_state_machine {states|show|list|observe}`,沒有掛上 `dv-harness CLI` 動詞,也沒有任何 `run_stage()/advance()` 呼叫點上它。**刻意受限**:一個到達終端狀態的 pattern,重跑時是建立一筆全新的記錄(`create_pattern_record()`),而非原地恢復——不像持久化的 `loop session`,單一 pattern 的 `runtime` 狀態沒有值得跨重啟保留的部分進度。

7.7 Runtime CONTROL-Command 封閉詞彙:落實「command.txt 中不得有不受限的腳本行為」

`command.txt/pattern` 檔案有一條明文規則 —— 「不得在 `command.txt` 中建立不受限的腳本行為」—— 過去完全沒有程式碼強制執行。

`dv_harness/runtime_control_commands.py` 補上這一塊:任何被分類為 `CONTROL` 指令的陳述式,必須屬於封閉詞彙

`WAIT / POLL / REPEAT / BOUNDED_LOOP / SYNC / BARRIER`,否則回報 `ILLEGAL_CONTROL_COMMAND`;而 `REPEAT / BOUNDED_LOOP` 這兩個明確帶有迭代次數語意的詞彙,必須宣告一個真正的次數上限引數,否則回報 `ILLEGAL_UNBOUNDED_LOOP` ——即使指令本身的名字就叫 `BOUNDED_LOOP`,標籤本身也不被信任,只看它自己的引數。

剖析完全委派給既有的 `reference_pattern_audit.extract_command_statements()`,本模組不重新剖析 `command.txt` 文字。一個陳述式被分類為 `CONTROL`,途徑有二:(a) 無條件地,當其 `.category` 已經是 `C_CONTROL_FLOW` 或 `C_SYNCHRONIZATION` (原生

`repeat / while / for / forever / if / fork / join / disable / wait(...)/@(...)/#...` 直接使用,這正是規則所禁止的「不受限腳本行為」);(b) 對於反引號巨集/`model-task` 呼叫,透過對 `CONTROL_INTENT_NAME_TOKENS` 做具名證據分類(沿用 `reference_pattern_audit.classify_wait()` 已建立的慣例——一個有引用依據的啟發式,而非證明),因為原始解析器對裸巨集呼叫刻意保留 `C_UNCLASSIFIED`。

```
python -m dv_harness.runtime_control_commands --command-file <file> [--json]
```

`Exit 0=CLEAN,1=有真實違規,2=NOT_AVAILABLE`。 **Evidence Truth Rule**:未被辨識為 `control-shaped` 的陳述式,直接從報告中缺席,絕不默默視為「通過」;檔案讀不到/剖析不了回報 `NOT_AVAILABLE`,絕不偽造成 `CLEAN`。「宣告上限」的檢查只看次數位置是否存在在非空引數 token——它無法把 `define` 出來的常數解析成有限值,本模組也不發明「這個字面值代表無限」的慣例,因為本專案的真實證據中從未觀察過這種慣例。它只回報,不做 `build/job/approval`,也刻意沒有 `stage gate`。

7.8 Command Error Taxonomy:十一種派送層級錯誤分類,與 loop_budget.FailureType 刻意分離

`dv_harness/command_error_taxonomy.py` 把「一次指令/任務派送」產生的真實失敗文字(例外訊息、`sim.log` 中失敗任務周圍的節錄)分類成十一種細粒度類別之一,並附上比對到的證據,或在完全比對不上時回報 `UNCLASSIFIED`。依 **Evidence Truth Rule**,無法比對的訊息永遠不會被硬塞進某個具名類別 —— `UNCLASSIFIED` 是誠實的答案,不是功能缺失。

任務層級詞彙是「讀來的,不是發明的」:卡在 `branch_fw` (每 port 的 FW/事件服務迴圈)裡的派送是 `FW_TIMEOUT`;卡在 `branch_b*` (每 port 的 VIP 驅動測試本體,或名稱帶有 `Synopsys svt_` 前綴的 VIP 元件——與 `loop_budget.FailureType.VIP` 已記載的同一個前綴慣例)是 `VIP_TIMEOUT`;卡在 `branch_a*` (每 port 的 DUT+PHY 初始化任務,或泛泛提及 DUT/PHY 但沒有任何分支標籤)是 `DUT_TIMEOUT`。 `BRANCH_OWNERSHIP_ERROR` 把 `pattern-architecture 3.1` 節命名的陷阱類別——「兩個獨立的 task group 可能持有同一個共享匯流排定序器上互相衝突的鎖」——變成可從派送失敗文字檢查的規則:兩個不同的任務層家族與一個鎖/所有權/仲裁關鍵字同時出現在文字中。

`CHECK_FAILURE` 與部分 `TASK_ERROR` 是直接呼叫既有、真實的 `sim_log_analysis.parse_sim_log()/classify_signatures()` 分診引擎決定,本模組絕不重新推導一份自己的 `scoreboard/assertion/UVM_FATAL` 關鍵字表。這是與 `loop_budget.FailureType` (回答「為何一個 STAGE 的重試耗盡」,粗粒度、跨多次派送)刻意不同、也絕不合併的更細粒度詞彙, `assert_disjoint_from_loop_budget_failure_type()` 與 `assert_disjoint_from_verification_verdict_vocabulary()` 分別在 `import` 時檢查兩者及與 `models.Status` 都不相交。本模組只分類,不重試、不停止迴圈、不消耗預算、不決定 stage 的 `PASS/FAIL`,是一個純函式,沒有自己的檔案 I/O(`sim_log_analysis` 內部已有的除外),沒有 `subprocess`。

7.9 DE Command Review Package:只做渲染,絕不做回復推論

`dv_harness/de_command_review_package.py` 產生給人審閱的 `DE Command Review Package`:一張 `markdown` 表格,把從舊版 `command.txt-style pattern` 檔案中回復出的每一個既有指令 token,對照本專案自己回復證據對它的說法並列呈現,讓真人 `DE` 在這份回復意義變成下游機器信任的依據之前先確認或修正它。

這個模組只是渲染器。它從不回復任何指令的意義、從不推論分支所有權、從不發明相容性判決或時序數值——它接受一份 `generic`、`duck-typed` 的 `dict-like row` 清單(由這批工作中的其他上游回復步驟產生,或人工產生),只驗證一份審閱表不能誠實省略的兩個欄位,並把表格排版/樣式完全交給 `connectivity.render_markdown_table()` (本專案唯一參數化的表格渲染器)。欄位固定順序:Existing Command(來源檔案的逐字 token)、Recovered Meaning、Branch Owner(必須是 `block / branch_a{i} / branch_fw / branch_b{i}` 四層詞彙之一,詞彙以外的值會被大聲拒絕而非渲染出來,因為錯誤的 owner 會讓審閱送到沒有人看的地方)、Task、Preconditions、Expected Effect、Compatibility(逐字沿用上游回復結果)、Open Ambiguity(輸入 row 真的沒有內容時才留白,絕不預設填成捏造的「none」)。任何 row 缺少 `existing_command` 身分,或 `branch_owner` 字串在四層詞彙之外,都是硬性驗證錯誤 (`DECommandReviewPackageError`),絕不默默丟棄或強制轉型。**揭露的殘留邊界**:它只渲染;不回復、不推論、不核准任何東西,也不引用任何 `approval/governance` 機制。

7.10 command.txt Change-Impact:兩份 DE 指令登錄快照之間的語意差異

`dv_harness/command_txt_change_impact.py` 回答一個比既有 `change_impact.py` (git-diff 驅動的檔案存在/回歸選擇機制)更窄的問題:給定兩份實際的 DE 指令登錄快照(某次 generator/schema/command.txt 語料編輯前後各一份),個別指令的定義本身變了什麼、怎麼變的。它接受 `DECommandRegistryIR`-shaped 的 duck-typed dict/list,但刻意不 import `de_command_style_learning.py` (該形狀真正的產生者,屬於另一個並行批次)。

詞彙: UNCHANGED / ARGUMENT_CHANGE / SEMANTIC_CHANGE / NEW_COMMAND / REMOVED_COMMAND / DEPRECATED / AMBIGUOUS,與 `models.Status`、`loop_budget.FailureType` 皆刻意不相交。判定優先順序(由重到輕):明確的 deprecated 訊號優先;其次任何語意層面的差異(包含從 deprecated 狀態恢復);其次位置性參數清單的差異(參數順序對 command.txt 巨集/task 呼叫是有意義的,因此絕不按名稱排序後比對);其次任何一側無法解析的層面 → AMBIGUOUS,絕不默默當作 UNCHANGED;最後才是 UNCHANGED。絕不捏造改名:舊快照消失的指令 id,加上新快照中拼法不同的替代指令,永遠回報成一筆 REMOVED_COMMAND 加一筆 NEW_COMMAND——用猜測的相似度去配對兩者,正是 Evidence Truth Rule 禁止的捏造關聯。

```
python -m dv_harness.command_txt_change_impact --old <old.json> --new <new.json> [--json]
```

Exit 0=沒有值得關注的變化,1=有值得關注的判決,2=無法讀取/剖析。沒有 `dv-harness` CLI 動詞。

7.11 Existing-Command Reuse Score:排序既有指令,絕不強塞一個猜測答案

`.claude/skills/CORE/command-inventory/SKILL.md` 早已把既有的 DE command.txt 當成「可重用能力基準」,要求 `dv-workflow/command_inventory.csv` 這份庫存,但從未有程式碼真的針對一個新的 vPlan 需求把這份庫存排序過。`dv_harness/existing_command_reuse_score.py` 是那個排序器, `evaluate_reuse(need, existing_commands, ...)` 是主要進入點。

四個排序維度:(1) 語意名稱比對——確定性的 LEXICAL Jaccard token 重疊(`semantic_name_match()`),明確不是 embedding/ML 模型(本專案沒有這種東西,發明一個正是 Evidence Truth Rule 禁止的不可驗證機制);(2) 引數形狀相容性(`argument_shape_compatibility()`)——當引數是結構化的 {position, role} 記錄時做位置對齊的角色比較,只有扁平 PARAMETERS 字串/清單時降級為原始 token 比較,兩種形式在報告中保持可區分;(3) 分支所有權相容性(`branch_compatibility()`)——沿用 `block / branch_a* / branch_fw / branch_b*` 正典詞彙,分支家族不匹配會讓候選完全被排除而非只是低分,非正典命名(如 `Branch0`)仍會被解析成所屬家族但標記 LEGACY_NON_CANONICAL_NAMING;(4) 真實歷史 PASS 證據——唯讀取自 `evidence_db.py` 的 `regression_verdict_history` 表(`query_command_history()`),依序嘗試候選宣告的 `pattern / command_name / source_command_file` 的檔名主幹,誰有真實紀錄就用誰,完全沒有紀錄的候選回報 NO_RECORDED_HISTORY,絕不捏造 0 或 1 的通過率。

NO_REUSE_CANDIDATE,絕不強塞一個低信心選擇。一個候選必須同時跨過兩道門檻: `composite_score >= MIN_PLAUSIBLE_SCORE` (四個加權維度的 0.20),以及在兩個「可觀測」維度(語意名稱重疊或引數形狀重疊)中至少一個有真實、非零的證據——單靠分支家族一致,或「沒有歷史紀錄」貢獻的零分,永遠無法自己湊出一個看似合理的匹配。當沒有候選能同時跨過兩道門檻, `evaluate_reuse()` 回報 NO_REUSE_CANDIDATE 並附具名理由 (`NO_EXISTING_COMMANDS_SUPPLIED / INSUFFICIENT_NEED_DESCRIPTION / NO_PLAUSIBLE_MATCH`),絕不把矮子裡拔將軍的結果當成一個真正的發現回傳。

```
python -m dv_harness.existing_command_reuse_score --need-file ... --commands-file ... [--root ...] [--json]
```

它只評分、不決定、不核准、不建置、不執行——沒有 stage gate,候選自我宣告的 `STATUS / CONFIDENCE` 欄位絕不被當成 PASS 證據(逐字保留成 `declared_status / declared_confidence` 僅供人類參考,唯一真正的證據是一筆 `regression_verdict_history` row)。

7.12 Pattern IR Assembly 與 Pattern Fragment IR:從 ScenarioIR 到 PatternIR、從整份 pattern 到可複用片段

`dv_harness/pattern_ir_assembly.py` 的 `assemble_pattern_ir(scenario_ir, *, global_commands=None, dut_commands=None, ...)` 從一個 generic、duck-typed 的 ScenarioIR-shaped 輸入,組裝出一個 PatternIR——`global / dut / fw_policy / vip / check` 這五條指令清單,一一對應到 pattern-architecture SKILL.md 的真實詞彙: `global = block, dut = branch_a*, fw_policy = branch_fw, vip = branch_b*, check = verdict / FINAL_CHECK`。它刻意不 import `verification_intent_ir.py` 或 `vplan_artifact.py` (屬於另一個並行批次),而是把任何具備「objective 加上 stimulus/checker/coverage-intent 文字」形狀的 dict/list 都當作合法輸入,欄位一律 duck-typed 讀取(`.get` 或屬性存取,依呼叫方物件支援哪一種),絕不對某個沒有 import 授權的類別做 `isinstance` 檢查。`validate_layer_ordering()` 則驗證這五層的排列順序是否符合 pattern-architecture 已知安全的慣例。

`dv_harness/pattern_fragment_ir.py` 回答一個更小的問題:給定一個既有 pattern 的指令清單,以及呼叫方宣告的「一段」範圍(索引範圍,或以 marker 分隔的片段——絕非預設整份), `extract_pattern_fragment()` 把這一段抽取成一個自我描述的 `PatternFragmentIR`:它碰觸哪些資源(`resources_used`)、產生/消費哪些事件、以及 preconditions/postconditions,讓後續的系統層組合步驟有具體東西可以推理,而不必重新讀一次原始 pattern 文字。這個模組刻意比另外兩個容易混淆的模組更窄: `pattern_ir_assembly.py` 組裝的是整個場景的完整五層 PatternIR; `example_composition.py` 組合的是多個已經合格的完整 VIP 範例(host example 加 device example 等),受 7 條相容性條件把關。`pattern_fragment_ir.py` 只處理「既有一份 pattern、抽出其中一部分」這個更小的問題。`check_fragment_chain_readiness(fragments, *, order=None)` 進一步檢查多個片段依序串接時,彼此宣告的 preconditions/postconditions 與 produced/consumed events 是否鏈得起來。

這兩個模組都是純函式庫模組:沒有 `__main__` 前門、沒有掛上 `dv-harness` CLI 動詞,只能以 Python API 的方式從呼叫端 import 使用。

7.13 小結:這批模組共同的邊界

本章涵蓋的十餘個模組有一個共同、反覆出現的紀律:每一個都只回答自己那一個窄問題,絕不 import 兄弟模組去搶答對方的問題,絕不在證據不足時猜測一個看似合理的答案,而是誠實回報 `NOT_AVAILABLE / UNKNOWN / AMBIGUOUS / NO_REUSE_CANDIDATE` 等具名狀態。多數模組完全沒有掛上 `dv-harness` 的正式 CLI 動詞(`cli.py / gates.py` 明確被排除在這些批次的檔案安全範圍之外),前門只是各自的 `python -m dv_harness.<module>` ad hoc 介面,或甚至純粹是函式庫 API——這是刻意保留的整合缺口,不是遺漏:它們是被設計成隨時可以被下一步的 wiring 工作接上 `STAGE_GATES`、`run_stage() / advance()` 或真正的 CLI 動詞,但目前為止,仍然是「REACHED,而非 WIRED」的獨立分析層。

8. 智慧收案 (Intake) 與提問優先序

DV Agent Harness 在真正進入 UVM 產生或簽核之前，必須先確認一件事：這個專案的「收案」(intake) 到底做到哪裡、哪些事實已經確認、哪些還懸而未決。這件事橫跨好幾個各自獨立的子領域——`env_manifest.py` 的三層事實 manifest、`question_queue.py` 的 ask/decide 生命週期、`connectivity.py` 的 bind-tier 閘門、`requirement_contract.py` 的逐條需求狀態、`golden_scenario.py` 的測試新鮮度、`waiver_store.py` 的豁免狀態——每個模組把自己那塊管得很好，但沒有任何一個站在上面回答人類真正要問的問題：「整個專案的 intake 現在到底準備好了沒？」本章要介紹的，就是這一整組收案機制：如何向人類提問、如何決定先問哪一個、如何彙整成一張總表，以及最後怎麼把它們凍結成一份可稽核的 baseline。

3-Tier Ask-a-Human 協定：`question_queue.py` 與 `dv-harness question-queue`

真正「向人類提問」的權責模組是 `dv_harness/question_queue.py`，它實作 spec Part B 的三層決策：self-resolve (Tier 1，manifest 已經有答案)、safe-to-assume (Tier 2，可以先假設並記錄，繼續往下跑)、cannot-assume (Tier 3，一個寫死、非模糊的判斷條件觸發，必須等人類回答)。

`classify_tier()` 會先檢查 Tier-3 的硬性條件，永遠優先於任何 Tier-1/Tier-2 的捷徑判斷。問題與決策分別持久化在 `.dv-`

`harness/question_queue/decisions.json` (機器可讀) 與自動產生的 `decisions.md` (人類可讀)，而且會即時鏡射到 Blackboard 的

`open_questions_decisions` 主題，讓後續 stage 不必打開這個私有 store 也能知道「這件事已經問過、答過了」。這是本章唯一一個真正被 wire 進 `dv-harness` CLI 的子命令：

```
dv-harness question-queue add --domain vip \
--question "usb3_link_ctrl 的 reset polarity 為何?" \
--context-path rtl/usb3_link_ctrl.v \
--option "active_high" --option "active_low" \
--recommendation "active_low" \
--assumption-if-unanswered "active_low"

dv-harness question-queue answer Q-xxxx \
--answer "active_low" --basis "designer email 2026-09-05"

dv-harness question-queue digest --trigger stage_boundary --stage GATE1_ELABORATION
dv-harness question-queue status
```

一旦某個 `question_key` 被人類真正回答過，這個問題就「永遠不會再被問一次」——這是 `question_queue.py` 對外的核心保證。`digest` 不是即時推播，而是按日或 stage 邊界批次彙整；`status` 印出 self-resolve rate、blocking-questions/week、repeat-question-rate、assumption-overturnd-rate 四項追蹤指標。`revoke` 提供撤回一個錯誤 Tier-2 假設的合法途徑，決策紀錄只會移入 `revoked` 清單，絕不直接刪除。

提問優先序：Gating Table、Next-Best-Question 排序與 Batching

當 Tier-3 問題累積得比人類能回答的速度更快時，`dv_harness/intake_question_priority.py` 決定三件事——這個模組刻意不 import `question_queue.py`，待處理問題以一般 dict 清單傳入，讓它可以被任何「待問清單」重用。

第一，**Confidence × Criticality 的 16 格判斷表** (`GATING_TABLE`)，四個信心等級 (LOW/MEDIUM/HIGH/UNKNOWN) 交叉四個嚴重度等級 (MINOR/MAJOR/BLOCKER/UNKNOWN)，每一格的 ASK/DO_NOT_ASK 判斷都手寫列出，而非用公式計算風險分數。只有 (HIGH, MINOR) 與 (HIGH, MAJOR) 兩格是 DO_NOT_ASK；BLOCKER 嚴重度無論信心多高都一定要問，任一軸只要是 UNKNOWN 也一定要問——不確定的嚴重度絕不能被讀成「安全可略過」。

第二，**Next-Best-Question 排序**，`score_question()` 直接套用 `blocking_value * downstream_impact * expected_confidence_gain / user_effort` 公式，四個因子的實際量測方式完全交給呼叫者定義；缺欄位、非數值 (`bool` 會被明確拒絕，因為它是 Python 的 `int` 子類)、負權重、或 `user_effort` 非正數，都誠實回報 `UNVERIFIABLE` 並指名壞欄位，而不是給一個佔位分數。

第三，**Batching**：只有明確宣告 `MINOR` 且沒有 `architecture_defining: true` 標記的問題才會被視為 `LOW` 風險、可批次；`build_question_batches()` 依呼叫者宣告的 `topic` 分組，任何 `HIGH` 風險問題永遠單獨成一批，即使跟其他問題共用主題也絕不合併。

這個模組沒有被接進 `dv-harness` 或 `cli.py` / `gates.py`，前門是：

```
python -m dv_harness.intake_question_priority
```

「先查哪裡」而非「誰說了算」：Intake Source Priority 與 Design Source Inventory

專案裡已經有 `source_authority.py` 的 9 層 `AUTHORITY_ORDER`，回答「兩個已讀到的來源互相矛盾，誰贏」；也有

`tools/verification_flow/evidence_source_priority_gate.py` 的 9 項 `ORDER`，回答「還不知道某個事實時，第一個該查哪裡」。這兩張表恰好都是 9 項，該模組自己的 docstring 就記載了曾因為長度巧合而被誤認一次。`dv_harness/intake_source_priority.py` 是刻意設計為 10 步、與前兩者長度都不同的**第三張表**：repo 現有檔案、既有 UVM 環境、build script/Makefile、RTL/PHY 原始碼、暫存器檔、spec/datasheet、VIP 範例、regression 清單、git 歷史、詢問使用者。`next_sources_to_check(fact_name, available_sources)` 依此順序過濾出呼叫者宣告「目前可用」的來源，永遠把「詢問使用者」排在最後。它連檔案系統掃描都不做——可用來源清單完全由呼叫者宣告，這個模組只負責排序：

```
python -m dv_harness.intake_source_priority order
python -m dv_harness.intake_source_priority next --fact dut_top --available existing_repo_files git_history ask_user
python -m dv_harness.intake_source_priority self-check
```

`design_source_inventory.py` 則是另一張獨立、同樣 10 項的 `DISCOVERY_ORDER`，但服務的是不同粒度的問題——一份實際的**來源登記表**（`{source_id, type, version, hash, authority, status, last_checked}`），狀態值為 `CURRENT/STALE/SUPERSEDED` 再加上 `NOT_AVAILABLE/UNKNOWN` 兩個誠實補充值，權威層級透過 `import`（非重新實作）`source_authority.authority_source()` 解析。這個模組沒有任何 CLI 前門，也沒有落地快照的能力——`recorded_hash / superseded_by` 每次呼叫都要由呼叫者提供。

Intake State：跨來源合併的逐欄位事實表

`dv_harness/intake_state.py` 回答「這個專案的 intake 目前對欄位 X 到底知道什麼、從哪裡來」，每個欄位一筆 `IntakeFieldRecord`（`value / source / confidence / status / last_validated / owner`），狀態為 `AUTO_RESOLVED / USER_CONFIRMED / PARTIAL / CONTRADICTED / MISSING / BLOCKED / UNKNOWN / NOT_APPLICABLE` 八選一。它只做「讀取並合併」三個既有的真實來源：`env_manifest.py` 每層自己的 `status / reason`、`question_queue.QuestionQueueStore.find_decision()`（唯讀，這個模組本身從不寫問題或決策）、以及 `connectivity.py` 的 `BindTier` 詞彙。人類的真實答案永遠壓過系統自己的 Tier-2 猜測；若人類答案與既有計算值不一致，欄位回報 `CONTRADICTED`，絕不被默默覆蓋成「兩者一致」的假象。

它最重要的輸出是 `evaluate_uvm_generation_ready()`——對六個具名阻擋類別（`DUT boundary`、`VIP unresolved`、`active-driver conflict`、`critical bind`、`build env`、`known-PASS test`）做 `worst-wins` 折疊；任何一個類別完全沒有紀錄，一律折成 `MISSING`，絕不會因為「沒資料」就被視為 `ready`。`already_resolved(intake_state, field_name)` 則是給呼叫端在真正呼叫 `question_queue.QuestionQueueStore.add_question()` 之前用的「別再問一次」檢查——這個模組本身從不主動提問。這個模組沒有任何 CLI，純粹作為函式庫被其他呼叫端 `import` 使用。

Intake Baseline：收案十二項事實的凍結

`signoff_export.py` 在簽核時凍結 15 項「驗證後」的專案識別欄位，但沒有任何機制凍結**收案時**（`generation` 開始之前）專案承諾的那組更早的事實。`dv_harness/intake_baseline.py` 針對這十二項：`DUT top/boundary`、`DUT SHA`、`TB SHA`、原始檔案雜湊、`VIP 宣告`、`bind-topology` 雜湊、`reference-UVM` 雜湊、`DE command.txt` 雜湊、已知測試清單，以及未解決 `critical unknowns`、未解決衝突、已記錄使用者決策這三個計數，套用與 `signoff_export.py` 相同的內容雜湊凍結模式（`worst-wins` 的 `VALID/INVALIDATED/UNKNOWN`，「無法檢查」永遠不等於 `VALID`）。多檔案或清單型事實一律透過 `source_identity.aggregate_source_id()` 摺疊成單一雜湊，全專案只有這一套雜湊規則。

```
python -m dv_harness.intake_baseline fields
python -m dv_harness.intake_baseline freeze --facts facts.json --frozen-by <name>
python -m dv_harness.intake_baseline status --project-root .
```

凍結紀錄寫在 `.dv-harness/intake/baselines/<freeze_id>.json`，且是唯一一次寫入；`evaluate_intake_freeze_invalidation()` 每次呼叫都重新從兩個獨立來源推導判定（十二項事實逐一重新捕捉比對、以及凍結紀錄自身的 `freeze_id` 完整性重算），從不信任已儲存的判定字串。它不做任何發現、仲裁或授權——沒有 `stage gate` 依附在它身上，`REVALIDATION`（判定一個 `INVALIDATED` 的 intake baseline 是否仍可接受）是人類的動作，不是這個模組的職責。

Verification Intake Contract：全專案生命週期與 INTAKE_READY

`dv_harness/verification_intake_contract.py` 是這一批收案機制的頂點模組，只做兩件事。第一，一個**十三狀態的生命週期狀態機**——`CREATED` → `DISCOVERING` → `CORRELATING` → `QUESTION_PENDING` → `USER_INPUT_RECEIVED` → `VALIDATING` → `CONFLICT` → `PARTIAL` → `BLOCKED` → `READY_FOR_REVIEW` → `BASELINED` → `STALE` → `REVALIDATING`，由一張封閉的 `TRANSITIONS` 圖強制合法遷移，非法跳轉（例如 `CREATED` 直接跳 `VALIDATING`）會丟出 `IntakeContractError`。`READY_FOR_REVIEW` 明確不是終態——它有五條出邊，`assert_no_absorbing_state()` 在 `import` 時就證明每個狀態（包括 `BASELINED`）都至少有一條真正的出邊，一個已 `baseline` 的收案仍可能因後續 `spec/RTL/config` 變動而滑向 `STALE`。

第二，`evaluate_intake_readiness(conditions)`——對呼叫者宣告的一串 `{"name", "status"}` 條件做**連言 (conjunction)**，狀態為 `MET/UNMET/UNKNOWN/NOT_APPLICABLE`，只要 `blocking` 清單非空（`UNMET` 或 `UNKNOWN` 都算阻擋），`ready` 就是 `False`，並指名是哪一條條件擋住——絕不平均、絕不用百分比稀釋，這與 `golden_flow_readiness.combine_readiness()` 的 `worst-wins` 原則一致。空條件清單回報 `NOT_AVAILABLE`，絕不是零條件下的虛假 `READY`。

```
python -m dv_harness.verification_intake_contract states
python -m dv_harness.verification_intake_contract transitions
python -m dv_harness.verification_intake_contract evaluate --conditions conditions.json
```

這個模組刻意不 `import` 任何其他 `dv_harness` 模組（以 `AST` 測試驗證），每個子領域事實都以一般 `dict/list` 傳入；它不仲裁 `CONFLICT` 代表什麼、不決定哪一方對，也不寫任何 `approval/governance` 紀錄。

支援型證據工具：User Answer Validator 與 File Candidate Ranker

收案過程中人類會打出像「`DUT top = usb_core`」這樣的自由文字答案，`dv_harness/user_answer_validator.py` 專門把這類宣告拿去對真實證據核對——模組名稱存在判斷 `EXISTENCE` 時真的跑 `verible_parser.parse_file()`；`build INCLUSION` 則讀 `env_manifest.py` 已記錄的 `dut_facts.rtl`，從不重新解析一次 `build` 清單。它給出四種誠實狀態：`VALIDATED`、`PARTIALLY_VALIDATED`（真證據支持較弱版本的宣告，例如大小寫不符或路徑不同）、`CONTRADICTED`（證據齊全但宣告為假）、`UNVERIFIABLE`（真的無法核對，例如缺輸入或 `verible` 解析失敗）——這與 `dut_evidence_correlation.py`（驗證 `REQUIREMENTS`，而非原始 `intake` 答案）是明確不同的消費者，兩者不應混淆。這個模組沒有任何 CLI 前門，純函式庫。

`dv_harness/file_candidate_ranker.py` 解決「幾個相似命名的檔案，哪一個才是正本」這種常見場景（`usb_reg.xlsx` VS `usb_reg_v2.xlsx` VS `usb_reg_final.xlsx`）。它蒐集三個獨立證據：真實 `git log --oneline -- <path>` 的引用次數與最近提交、專案內 `build script/Makefile` 對該檔名的真實文字掃描次數、以及 `os.stat()` 的真實 `mtime/size`。凡是真的無法確認的訊號一律回報 `NOT_AVAILABLE` 並附真實原因，絕不猜值；但一個真正查過、結果就是零的 `build-script` 引用，會回報 `AVAILABLE` 且值為 0，因為把「查過是零」混同成 `NOT_AVAILABLE` 本身就是違反 `Evidence Truth Rule`。**這個模組從不挑贏家**——`rank_file_candidates()` 永遠回傳每一個候選人及其完整證據，額外計算的 `display_rank / display_order_reason` 純粹是資訊性排序，測試套件明確斷言它絕不能被讀成決策或建議。它有一個小型 CLI 前門：

```
python -m dv_harness.file_candidate_ranker rank \
usb_reg.xlsx usb_reg_v2.xlsx usb_reg_final.xlsx \
--repo-root . --project-root .
```

這一組機制刻意不做事

整體而言，本章介紹的九個模組刻意分成兩層：一層是真正被接進 `dv-harness` CLI、會寫入持久狀態並影響後續流程的 `question-queue`；另一層是有意保持獨立、以 `python -m` 或純函式庫形式存在、從不寫 `approval/governance` 紀錄、從不觸發 `stage gate` 的收案輔助模組。它們共同的邊界是：沒有一個模組會替人類做仲裁——CONFLICT 是什麼、哪一方對、一個 INVALIDATED baseline 是否仍可接受，全部留給人類決定；它們也都不會憑空猜測——任何無法用真實證據核對的欄位一律誠實回報 UNKNOWN/NOT_AVAILABLE/UNVERIFIABLE，而不是被預設為安全或已解決。這正是收案階段之所以可信的原因：它不試圖幫使用者做決定，只確保在人類做決定之前，每一項事實都經過如實的檢查與排序。

9. 模擬、除錯與 LSF Regression

本章涵蓋 harness 在「送 job、跑模擬、看 log、開波形、決定要不要降級、要不要排隊」這一整條除錯與 regression 鏈路上的真實機制。每一節都對應到 `dv_harness/` 下確實存在、可被呼叫的模組與 CLI 動詞,不包含任何尚未實作的空想功能。

9.1 送出模擬前的強制關卡: `preflight`

`dv_harness/preflight.py` 是「lstat + scheduler preflight gate」。這是 2026-09-03 的使用者需求(「沒過就 BLOCKED,不派 job」),任何 `bsub` / `sbatch` 送出之前都必須先跑一組固定的 BLOCKING 檢查:EDA license 可用性(`check_license()`),解析真實 `lmutil lstat -a -c <server>`)、LSF/Slurm queue 健康度(`check_queue_health()`),解析真實 `bqueues <queue>`)、目標主機可達性(`check_host_reachability()`)、遠端工作目錄的磁碟空間與可寫性(`check_disk_space()` / `check_workdir()`)、以及必要的 EDA 環境變數(`check_env_vars()`)。每個檢查都回傳一個結構化的 `checkOutcome` (name/status/detail/command/evidence),絕不是裸的 boolean——任何 FAIL 都會讓整體結果變成 BLOCKED,而不是降級成警告。

實際呼叫方式有兩種:

```
dv-harness preflight --queue <queue> --workdir <path> --license-server 2900@host-a
```

這是獨立跑一次同樣的閘門,不送任何 job,常用於 Preflight/Resource Guard Agent 事先評估一批 regression 值不值得排隊。另一種是內嵌在送 job 流程裡:

```
dv-harness lsf-submit "<vcs/simv 指令>" --queue <queue> --cores <n> --runlimit-min <minutes>
```

`lsf_client.bsub_submit_with_preflight()` 會在真的呼叫 `bsub` 之前先跑 `run_preflight()`,BLOCKED 就不送。`--skip-preflight` 是唯一的例外逃生門,且是顯式、可稽核的旗標,絕不是預設行為。

值得注意的一個真實環境細節:遠端 DV server 的 login shell 是 tcsh,直接 `echo $VAR` 對未設定變數會是硬錯誤而非空字串,因此環境變數存在性檢查用的是 `$?VAR` (csh/tcsh 語法),而且刻意不使用 `printenv` / `env` / `set` / `export` 這類裸指令——persistent relay 的憑證檢查過濾器會直接拒絕它們 (CREDENTIAL_INSPECTION_DENIED)。傳輸層 (LocalCommandRunner VS RemoteRelayCommandRunner)由 `resolve_transport()` 依真實探測結果(relay 是否 READY、`lmutil` / `bqueues` 是否在 PATH 上)自動選擇,選不到就誠實回報「none available」,而不是亂猜。

9.2 sim.log 的證據式解析: `sim_log_analysis`

`dv_harness/sim_log_analysis.py` 是一套通用(非 USB 專屬)的 sim.log 解析引擎,純函式(str/dict in, dict/list out),沒有任何檔案 I/O、subprocess 或模擬器依賴,因此可套用在任何 UVM 專案的 log 上。

它存在的真實理由是一個真的踩過的坑:某次 `usbrun.sh` 的長跑 LSF job 健康檢查中,一個 PLL model 印出了不帶 `UVM_` 前綴的裸 "ERROR in PLL model",結果只掃 `UVM_ERROR` 的過濾器整整一小時沒發現。所以 `parse_sim_log()` 除了 `UVM_FATAL` / `UVM_ERROR` / `UVM_WARNING` 之外,永遠同時掃描裸 `Error-` / `ERROR` 字樣 (BARE_ERROR marker),再加上 `Fatal`、`assertion`、`timeout`、`mismatch`、`scoreboard`、`protocol`、`license`、`killed`、`memory`、`crash` 等 failure-triage 詞彙。同一行命中多個 marker 只會被記錄一次(依 normalize 過的 signature 分組),絕不重複計數。回傳結構包含 `total_lines`、`signatures` (每個唯一 signature 的 markers/first_line_no/last_line_no/count/example_line)以及 `epilogue` (見下)。`classify_signatures()` 再把每個 signature 分類成 signature/first/last/count/classification/severity/root_cause_hypothesis/proposed_fix/confidence 的完整 schema。

更關鍵的是 `detect_underreporting(job_state, parsed)`:它拿 `lsf_client.JobState` 裡由 agent 自我回報的欄位(`uvm_error_count`、`uvm_fatal_count`、`assertion_failure`)去和這個模組真正解析出來的證據比對,回傳一串人類可讀的落差描述。這是 CLAUDE.md「LSF DONE 不等於 DV PASS」規則在 log 分析層的具體實作——它不取代既有的子字串閘門,而是給那個閘門一個第二意見。

CLI 用法:

```
dv-harness sim-log-analyze --log <sim.log 路徑>
```

或用 `--log-text` 傳入行內文字(方便測試)。

9.3 FSDB 波形報表: `fsdb_report`

`dv_harness/fsdb_report.py` 包裝真實的 Synopsys Verdi/VCS `fsdbreport` 命令列工具,把 FSDB 波形轉成 ASCII 文字報表。這是 CLAUDE.md 執行閘門 PUSH → BUILD → VERIFY → WAVE=1 → `fsdbreport` → REVIEW → SIGNOFF 中「波形/FSDB 檢視」這一段的實際落地。

已確認(CONFIRMED,2026-09-01,從真實姊妹專案的委交 report script 反推)的關鍵事實: `fsdbreport` 永遠把報表寫進檔案,絕不寫 `stdout`。若省略 `-o`,它會悄悄把 `report.txt` 寫進當前工作目錄。這曾是這個模組更早版本的一個真實功能性 bug(讀了永遠是空字串的 `proc.stdout`),現在 `run_fsdbreport()` 一律顯式帶 `-o <tmp_path>` 再把檔案讀回來。真實 flag 語法是:

```
fsdbreport <fsdb> -bt <t0> -et <t1> -s <hier_path> [<hier_path2> ...] [-verilog | -csv | -of h] -o <outfile>
```

`-verilog` / `-csv` / `-of h` 三選一決定輸出格式,省略則是工具自己的預設格式;檔案路徑一律排在所有 flag 之前。另一組已確認的變體是 `-period <T>` `-level 1` `-csv`。尚未獨立確認的是 `-exp` / `-strobe` (條件式報告)與 `-f` (設定檔)——因為這台開發機是 Windows,沒有 Verdi/fsdbreport 安裝,無法對真實 .fsdb 檔驗證,所以這兩個 flag 只列為「未確認」,不寫成保證行為。

CLI:

```
dv-harness fsdb-report --fsdb <path.fsdb> --topic lfps_handshake --job-id <JOB_ID> --pattern <testcase>
```

--topic / --job-id / --pattern 都是選填的證據來源標記,寫進 `normalized_evidence` 表以便和該 job 的其他證據做 join,省略時誠實記為 NULL,而不是杜撰。

9.4 開波形前的人類關卡:Waveform Dump User Gate

CLAUDE.md 的「Simulation Observability Default」規定:正常 regression 預設 **FSDB OFF**,只有需要 signal-level 證據時才升級開波形,而且升級必須通過「使用者已確認 dump scope/level」的關卡。「First-Failure Waveform Rerun」進一步規定:批次的每一輪永遠是全批 FSDB OFF,只有真的產生 UVM_ERROR 或異常終止(fatal/crash/timeout)的子集才選擇性以 `WAVE=1` 重跑,重跑時鎖定第一個相關失敗點終止,絕不因為部分測試失敗就整批開波形重跑。

這個關卡曾經有一個真實的漏洞: `focused_wave_debug_window_gate.py` 只要求 `dump_scope_confirmed.confirmed_by` 是非空字串——這個欄位是寫證據區塊的 agent 自己填的。在互動路徑上這是誠實的(真的問過人),但在無人值守路徑上, `engine.loop()` 派出的是一個 `headless claude -p --dangerously-skip-permissions` 子行程, `prompt` 只透過 `stdin` 灌一次,完全沒有回頭問人類的通道——結果自我填寫的 `confirmed_by` 剛好在「沒有人被問過」的情況下通過了檢查。

`dv_harness/waveform_dump_gate.py` 把這個確認從「證據區塊裡的宣稱」改成「對 question queue 決策儲存區的真實查詢」: `verify_dump_scope_confirmation()` 從宣告的 `scope` 重新推導 `question_key`,查出已持久化的決策,並要求 `current.source == question_queue.HUMAN_DECISION_SOURCE`——這與 T3 bind 及未填 `scoreboard` 欄位共用同一個「真的是人類決定的」來源標記,整個 codebase 裡只有一種「confirmed」的定義。`confirmed_by` 還必須真的等於 `current.decided_by`,引用一個真決策但掛錯署名一樣不算確認。這個問題被歸類為 `blast_radius="unbounded"`,經 `classify_tier()` 判為 Tier 3(CANNOT_ASSUME、blocking、無答案時停在 OPEN)。

實際操作:

```
dv-harness waveform-dump-scope ask --scope top.usb_dev.ctrl --level-or-depth "signal-level, block-scoped" --failure-cone <描述>
dv-harness waveform-dump-scope status --scope top.usb_dev.ctrl
```

`ask` 會透過真實的 `QuestionQueueStore` 建立一個 canonically-keyed 的 Tier-3 blocking 問題,人類用既有的 `dv-harness question-queue answer <Q-ID>` 回答。沒有另外一個「waveform confirm」動詞,因為多一種記錄確認的方式就是多一種閘門要信任的東西。`status` 用同一個檢查回報,未確認時 `exit code` 為 2。

2026-09-04 的同日補強:在無人值守路徑上, `engine._file_waveform_dump_scope_question()` 會在 `run_stage()` 判為 `WAIT_USER` 的當下自動代為送出這個問題(讀取同一個被閘門拒絕的 `evidence block` 裡的 `scope/level`),並把可回答的 `question-queue answer <Q-ID> ...` 指令附進 `blocking_reason`。它只會「送出問題」,絕不會「寫入決策」——只有 `answer_question()` (也就是人類)才能寫決策。**Disclosed residual**:這只關閉了 `WAVEFORM` 這一項決策的漏洞,「headless 無人值守執行中途無法問人類任何問題」這個更廣泛的限制並未改變——修法是讓這種跑法停在 `WAIT_USER` 並留下一個可回答的 `Q-ID`,而不是讓子行程真的獲得了發問能力。整個回合仍是非同步的,需要人類回來執行 `question-queue answer` 後重新 `dv-harness start --loop` 才能恢復。

9.5 背景 Job/Log Monitor: regression_reporter

CLAUDE.md 的「Background Job/Log Monitor Auto-Start」要求:一旦宣告 `REMOTE_EXECUTION_REQUIRED`,在送出或期待看到任何 LSF job 之前,先執行:

```
dv-harness lsf-watch-start --vcuser <account>
```

這會啟動(或確認已在跑)一個 detached 背景行程,發現該帳號下所有存活的 job、對照/分析 `harness` 已註冊的 `JobState`,並維護 `regression.list` 安全網(給那些繞過生成環境自身 `Makefile` 原生 `RECORD=1` 機制的 job)。若已有 `watcher` 在跑則是 no-op(用 `PID` 檔追蹤)。乾淨結束 `session` 或執行模式轉回 `PURE_LOCAL_READ_ANALYSIS` 時,用 `dv-harness lsf-watch-stop` 停止; `dv-harness lsf-watch-status` 查詢是否在跑。

Cadence 是固定週期輪詢,不是事件驅動: `regression_reporter.main()` 跑一輪 reconciliation 後 `sleep --interval-minutes`。預設是 `DEFAULT_INTERVAL_MINUTES = 5` 分鐘; `self_check_list.md` #41 要求「每隔 10 分鐘自動確認」一次,這被讀成一個上限(`SPEC_MAX_INTERVAL_MINUTES = 10`),而非精確值——巡得比 10 分鐘更頻繁都合規。`interval_compliance()` 讓任何比 10 分鐘慢的 `--interval-minutes` 不會被靜默接受: `lsf-watch-start` 會印出 `stderr` 警告, `watcher log` 的第一行也會誠實記下實際週期。這是刻意設計成 `advisory` 而非 `clamp`——真的需要慢速 `watcher`(例如共用 LSF cluster 上的長跑整夜 `soak`)的操作者仍保有這個控制權,只是不會在不知情的狀況下拿到它。

background watcher 內部會呼叫 `sim_log_analysis.parse_sim_log_file()` 與 `detect_underreporting()`,並透過 `uvm_generator.regression_list_manager.apply_verdict_to_file()` 寫回 `regression.list`。

9.6 顯式降級模式: degradation

`dv_harness/degradation.py` 實作使用者需求:「降級路徑: Claude API 不可用、license 全滿、farm 塞車時, `harness` 應降級成『只收集資料、不做判斷』,而不是整個停擺或胡亂重試」。三個真實觸發條件,全部重用既有機制,本模組本身不新增任何檢查:

- TRIGGER_ADAPTER** (Claude API 不可用): 由 `engine.run_stage()` 既有的 `result.ok is False` 分支餵入的連續 `ADAPTER_FAIL` 計數。預設門檻是 `policy.max_stage_retries (2)+1 = 3`,意即既有重試預算已經燒完、`adapter` 仍然失敗,降級才開始。
- TRIGGER_LICENSE** (license 全滿): 直接使用 `preflight.check_license()` 自己的 `CheckOutcome`,不做第二次 `license` 檢查。
- TRIGGER_QUEUE** (farm 塞車): 直接使用 `preflight.check_queue_health()` 的 `CheckOutcome`。

降級狀態下, `mode` 只有 `NORMAL` 與 `DEGRADED` 兩種值,由 `_apply_mode()` 依 `triggers` 是否非空決定,並只在真正的邊界上戳記 `entered_at / cleared_at`。降級實際改變的是: `engine.run_stage()` 拒絕做「需要判斷」的呼叫——不呼叫 `adapter` 提議下一步、不做 `gate` 晉升、不做 `stage` 轉換;但每一輪仍會真正「收集資料」:重新探測三個觸發條件、寫一份真的 `auto-checkpoint session snapshot`、以及在 `events.jsonl` 追加一筆 `DEGRADED_CYCLE` 事件。`Job` 狀態輪詢與

log 收集完全不受影響——原因不是刻意保護,而是它們跑在 detached 的 LSF watcher 行程裡(`regression_reporter.ensure_watcher_running()`),從來沒經過 `run_stage()` ,所以一個 DEGRADED 的 engine 根本擋不住它們。

`degradation.probe_resources` 預設是 **opt-in(False)**;因為 `LocalCommandRunner` 只有在 `dv_harness` 跑在 Linux DV server(`lmutil / bqueues` 真的在 PATH 上)才是正確的傳輸層;在 PC 端 session 上這些指令根本不存在,把「command not found」誤讀成「license 真的滿了」正是 Evidence Truth Rule 禁止的杜撰結論。`degradation.transport` (預設 "auto")透過 `preflight.resolve_transport()` 選擇真正可用的傳輸層(relay-if-READY,否則 local-if-on-PATH,否則 NONE),由 `engine.DVHarness` 注入。

9.7 分層 Regression Cadence: `regression_tiers`

`dv_harness/regression_tiers.py` (2026-09-04)修補了一個真實缺口:過去 regression 是「扁平」的——只有一套送出機制(`Stage.REGRESSION` + `regression_submission_policy_gate`)、一個 escalation 設定、一個門檻(`EscalationConfig.uvm_fatal_burst_threshold = 3`),不管跑的是哪一種批次都套用相同標準。

一個 tier 只擁有,而且僅擁有三個旋鈕:

- 跑哪些測試**——由 `change_impact` 選出的 selection class 決定(`TierPolicy.selection_classes / tests_for_tier()`)。SMOKE 刻意不採納任何 impact-derived class:它是固定的 sanity/critical-path 集合,正是防止 impact model 漏判的 SAFETY 定義,一個工作是「獨立於 impact model」的 tier 不能反過來被 impact model 選擇。
- 時間預算**——`time_budget_minutes` ,只記錄與回報,不負責殺 job(kill/timeout)的實際擁有者仍是 `lsf_client` 的 auto-kill 掃描,避免兩個機制搶同一個決策)。
- escalation 門檻**——`uvm_fatal_burst_threshold` ,依 tier 而不同:SMOKE 是 1(固定 sanity 集合裡一次 UVM_FATAL 就已經是系統性訊號,等到第三次可能整個短跑已經跑完了)、NIGHTLY 維持既有的 3(這個數字原本就是針對 NIGHTLY 批量大小訂的)、WEEKLY 是 5(全宇宙、長尾稀有 pattern 的最寬跑批,3 個孤立 fatal 只是既有文件本就認定的例行雜訊)。

每個預設值都可透過 `.dv-harness/config.json` 覆寫,不會寫死在更深層的程式碼裡。

CLI:

```
dv-harness regression-tier list
dv-harness regression-tier plan NIGHTLY --base-sha <sha>
dv-harness regression-tier start SMOKE --base-sha <sha>
dv-harness regression-tier status
dv-harness regression-tier clear
```

`plan` 是唯一讀的,從 `.dv-harness/regression/computed_selection.json` (harness 算出的 change-impact selection)解出這個 tier 實際會跑的測試清單; `--base-sha` 可以先針對某個 base revision 重算一次 impact selection。`start` 會寫入 `.dv-harness/regression/active_tier.json` ,這個記錄之後被 `lsf-watch` reconciliation loop 讀取,套用該 tier 專屬的 UVM_FATAL escalation 門檻而不是扁平門檻。

注意: `dv_harness/qualification.py` 的 SMOKE_QUALIFIED/REGRESSION_QUALIFIED/PRODUCTION_QUALIFIED 是另一套「協定成熟度」狀態機,回答的是完全不同的問題(這個協定是否已被證明到某個信心等級),與 tier 的「週期性 cadence + 成本額度」概念刻意不共用任何程式碼,名稱撞在一起只是無法避免的領域詞彙重疊。

9.8 RTL-diff 驅動的測試選擇: `change_impact`

`dv_harness/change_impact.py` (2026-09-04)是 CHANGE → DESIGN IMPACT → REQUIREMENT IMPACT → PATTERN IMPACT → REGRESSION SELECTION 這條 verification-change-impact 鏈路真正的產生器,取代過去只存在於「agent 手動照抄 prose、`regression_selection_completeness_gate.py` 只檢查手寫 JSON 的欄位是否非空、`change_impact.csv / regression_selection.csv` 只是 header-only 的空 schema」的狀態。

它從真實資料計算四層:

- 變更檔案**——真實的 `git diff --name-only <base>...<head>` 。
- 設計影響**——變更檔案對照到 evidence DB 裡真實的 `rtl_modules` 表(由 `evidence_db.insert_rtl_parse()` 從真實 `variable --export_json` 輸出灌入)。沒有解析資料的檔案會退回用檔名本身當作受影響區域,並計入 UNRESOLVED,絕不悄悄丟棄。
- 需求/vPlan/pattern/coverage 影響**——對照專案已存在的 `.dv-harness/requirements.csv` 追溯登記表(欄位: REQ_ID, SOURCE, SCOPE, VPLAN_ID, SCENARIO_ID, COMMAND_ID, PATTERN_ID, CHECKER_ID, COVERAGE_ID, RESULT, STATUS, EVIDENCE),只讀取既有的登記,絕不杜撰登記表沒有宣告的關聯。
- Regression 選擇**——分成 TARGETED / DEPENDENCY / SAFETY / MANDATORY_SIGNOFF 四類,全部是算出來的,不是宣稱的。

誠實規則直接寫進程式碼、而非只留在文件裡:任何 impact gap 或 unknown dependency 都會降低 `confidence` ,若該檔案風險為 HIGH,會設定 `expand_to_full_regression` ,把整個已知 pattern universe 都推進 DEPENDENCY 類——這個模組的失敗模式因此永遠是「跑太多」,不會是「悄悄漏掉了本該抓到的測試」。MANDATORY_SIGNOFF 永遠不從 diff 推導,只來自專案自己宣告的清單,任何 impact 計算都不能從中扣減。空類別會附上明確的 `<category>_empty_reason` ,誠實說明原因,不會假裝「什麼都沒被影響」。

這個模組不取代 `Stage.REGRESSION_SELECT` 的 evidence block,而是為它打底: `engine.py` 在該 stage 執行前先算好這份 selection,寫進 `.dv-harness/regression/computed_selection.json` 及兩份 CSV,再交給 agent 看。閘門只強制一條不對稱規則:agent 可以「加測試」(它的判斷力能看到 traceability registry 看不到的東西),但絕不能「刪掉」一個已計算出的測試——永遠只能往上擴張,和上面每一條規則的方向一致。目前沒有獨立的 `dv-harness change-impact` 動詞,實務上是透過 `regression-tier plan/start` 的 `--base-sha` 觸發重算。

9.9 跨 Job/跨專案資源仲裁: `resource_orchestrator`

`dv_harness/resource_orchestrator.py` (2026-09-06)補上完整性稽核中標記為「Global Resource / License Orchestrator NEVER_BUILT」的缺口。稽核前重新做過一次全庫搜尋確認: `preflight.check_license()` / `check_queue_health()` 只回答「這一次送出可不可以」,對「還有誰在問」一無所知; `lsf_client.bsub_submit_with_preflight()` 只針對單一 `bsub` 擋一次; `loop_budget.prioritize_stage()` (LOOP-3)確實是單一 `loop` 的裁決,拿一個 `stage` 和一個壓力讀值,在 `PRESSURE_NONE` 下對每個競爭者都回 `PROCEED`——十個 `job` 兩個空位,一樣是十個 `PROCEED`(這是一個真實的 `negative-control` 測試斷言,不是描述)。

`resource_orchestrator` 補的正是缺的那一塊決策:把「一份測得的容量」轉成「N 個競爭者中一個有界的授權集合」。它不是第二個 `license/queue` 探針(每個資源事實都以真實的 `preflight.CheckOutcome` 傳入,或在明確注入 `transport` 時透過 `degradation.probe_resources()` 取得,自己不呼叫 `imutil/bqueues`);不是第二個壓力分類器(呼叫既有的 `loop_budget.pressure_from_checks()`);不是第二套單一裁決規則(逐一呼叫既有的 `loop_budget.prioritize_stage()` ,`PROCEED/PROCEED_CRITICAL/DEFER` 原封不動帶出來,附帶原因,DEFER 不會被翻成 `GRANT,PROCEED` 也不會被翻成 `DEFER`);也不是送出者或閘門本身——一個 `GRANT` 不授權任何事,只是說「在正在詢問的競爭者裡,輪到這個」, `preflight.run_preflight()` 、`bsub_submit_with_preflight()` 的 `PreflightBlockedError` 、`policy.can_signoff()` 、`ControlPlane.approve()` 、PR-only 的 `main/master` 治理,每一個既有閘門仍站在真正動工之前,一個被 `GRANTED` 的競爭者若自己的 `preflight` 是 `BLOCKED`,依然是 `BLOCKED`。這個模組不送 `job`、不殺 `job`、不持有鎖,也不寫 `state/control/approval/memory` 任何一種記錄。

三種輸出決策——`GRANTED / QUEUED / DEFERRED`——刻意用和 `models.Status` 、`loop_budget.PRIORITY_*` 都不同的詞彙。`QUEUED` 是唯一「只有這一層才能產生」的值,因為它是一句關於「其他競爭者」的陳述,單一 `job` 的檢查永遠答不出來。排序規則本身以資料形式印在每份 `plan` 上(`RANKING_RULE`),讓讀者看見真正套用的規則,而非只能相信文件:(1) `prioritize_stage()` 的 `tier,PROCEED_CRITICAL` 先於 `PROCEED`,且只在真的量測到壓力時才拉開差距;(2) 這個專案自己在 `farm` 上已經持有的槽位數(從真實 `bjobs` 清單與該專案自己登記的 `job id` 取交集)——反壟斷訊號,只有這一層看得到;(3) 最早的 `requested_at` 優先(FIFO,不會餓死),沒宣告到達時間的請求排最後而不是被捏造一個時間;(4) `project_id` 再 `stage`,確保同樣輸入永遠算出同樣的 `plan`。

稀缺性不會被雙向杜撰:沒有測到的容量記為 `slots_available: None`,此時每個符合資格的競爭者都被 `GRANTED`,並在原因裡說明「沒有量測到」——因為沒量測到就延後真正的工作,和「不得捏造可用性」是同一條規則的另一個違反方向。`LSF` 的 `-` (無上限)記為 `None`、絕不是 0;把無上限的 `queue` 讀成滿的會讓整個 `farm` 上的 `job` 全部被延後。一個 `FAIL` 的 `license` 或 `queue` 檢查不會貢獻任何容量數字——資源枯竭的池子直接是 `PRESSURE_CRITICAL`,每個競爭者本來就會透過 `prioritize_stage()` 讀到這個訊號,把它再讀成容量 0 會重複計算同一個事實。

跨專案的競爭者來自這個 `codebase` 既有的 `cross_project_mining.ProjectRegistry`,包含它的 `ProjectIdentityCollisionError` 防護,避免一份 `memory store` 被註冊兩次後偽造出兩個專案的假共識(`contenders_from_registry()`)。目前這個模組還沒有掛進主 `dv-harness CLI`,而是自帶一個獨立入口:

```
python -m dv_harness.resource_orchestrator plan --project-root <root> [--requests <requests.json>] [--queue <queue>]
python -m dv_harness.resource_orchestrator contenders --project-root <root>
python -m dv_harness.resource_orchestrator capacity --project-root <root> --queue <queue>
python -m dv_harness.resource_orchestrator ranking-rule
```

`plan` 省略 `--requests` 時會直接從 `registry` 建構競爭者;有任何競爭者被 `QUEUED` 或 `DEFERRED` 時 `exit code` 為 2(CI 可見的「有人在等容量」訊號,不代表核准與否)。

9.10 章節小結:這條鏈路刻意不做事

本章涵蓋的模組共享同一種紀律:每一層只多做「真正缺的那一塊決策」,絕不重複量測、不重複裁決、不越權觸碰下一層的職責。`preflight` 只探測、`change_impact` 只算選擇範圍、`regression_tiers` 只記錄門檻與預算、`resource_orchestrator` 只排序授權,而波形是否開啟,以及開了之後看到了什麼,永遠停在需要一個真人回答的問題上——這是本專案 `Evidence Truth Rule` 在模擬與除錯鏈路上,一貫且刻意保守的具體實現。

10. Loop Engineering(收斂偵測、預算、斷路器、遙測)

DV Agent Harness L5 內部至少有三個真正在跑的「迴圈」：`engine.DVHarness.loop()` / `run_stage()` 驅動的 Verification Closure Loop、`memory_router.route_and_store()` / `promote_to_organizational()` 驅動的 Project Learning Loop,以及 `capability_evolution.py` 的 11-state 能力演進迴圈。在 2026-09-05 之前,這三個迴圈各自運作,但沒有任何一個能夠回答「這個迴圈自己的預算、收斂條件、plateau/oscillation 政策與終止條件是什麼」,也沒有共同的詞彙描述迴圈目前的狀態(PLATEAU、OSCILLATING、BUDGET_EXHAUSTED 等)。Loop Engineering 這一組模組(`loop_contract.py`、`loop_convergence.py`、`loop_budget.py`、`loop_telemetry.py`)就是為了補上這個缺口而建立的,而且刻意分成「契約與狀態機」、「收斂/plateau/oscillation 偵測」、「預算與斷路器」、「遙測事件」四層,各自對應到 spec 的第 85/86、88-90、91-93、107-108 節。

LoopContract 與 LoopState:一個獨立於 Status 的狀態機

`models.Status` (NOT_STARTED/RUNNING/PASS/FAIL/PARTIAL/BLOCKED/RETRY/WAIT_USER/CLOSED/ACCEPTED_RISK)回答的是「這個 graph node 的 gate 評估結論是什麼」,並且會被寫進每一份 `state.json`。`loop_contract.LoopState` 回答的是完全不同的問題——「這個迴圈作為一個控制流程,現在處於哪個狀態」。`LoopState` 包含 `CREATED` / `READY` / `RUNNING` / `VERIFYING` / `CONVERGING` / `PLATEAU` / `OSCILLATING` / `RETRY_WAIT` / `BLOCKED` / `HUMAN_GATE` / `SUCCESS` / `FAILED` / `BUDGET_EXHAUSTED` / `STOPPED` / `CANCELLED` / `RESUMING` / `STALE` 十七個值,其中好幾個(PLATEAU、OSCILLATING、BUDGET_EXHAUSTED、RESUMING、STALE)在 `Status` 裡完全不存在,若硬塞進 `Status` 只會讓既有的 `if status in (...)` 判斷靜默漏接。

`STATUS_TO_LOOP_STATE` 是唯一連接兩個詞彙的橋樑,並以 `assert_status_mapping_total()` 保證「total」——任何新加入的 `Status` member 若沒有先決定它對應的 `LoopState`,測試會直接失敗。兩個映射特別值得注意: `PASS` -> `CONVERGING` (而非 `SUCCESS`),因為一個 stage 的 `PASS` 只是這個 node 上的進展,迴圈真正的 `SUCCESS` 是 `overall_status == CLOSED`;以及 `ACCEPTED_RISK` -> `STOPPED` (而非 `SUCCESS`),因為人類接受殘餘風險並不等於機器可驗證的完成條件被滿足。

`derive_loop_state()` 是唯一把「後端事實」轉成 `LoopState` 的函式,參數全部來自真實的儲存——`state.json` 的 stage status/attempts、`config.json` 的 `policy.max_stage_retries`、`control.json` 的 `paused/takeover` 旗標,以及 Blackboard 的 `debug_loop_history` topic——絕不接受 agent 自己宣稱的狀態。優先順序是:TAKEOVER 最先(對應 Human Override 永遠優先)、其次是 PAUSED、再來是「重試家族狀態且 attempts 超過 max_attempts」→ `BUDGET_EXHAUSTED`、然後是 `oscillation fingerprint`、`loop_done` → `SUCCESS`,最後才是 `progress_oscillating` / `plateau` 這兩個只會作用在 `CONVERGING` 之上的判斷。

`LoopContract` 這個 dataclass 描述了 spec 第 85 節要求的欄位: `budgets`、`retry`、`convergence`、`plateau`、`oscillation`、`termination`、`escalation`、`human_gate`、`rollback`、`resume`、`audit`。三個真實迴圈的合約(`verification_closure_contract()`、`project_learning_contract()`、`capability_evolution_contract()`)全部是「從程式碼衍生」而非手寫維護——例如 `max_failed_attempts` 直接讀 `config.policy.max_stage_retries`,`convergence` 的門檻直接讀 `loop_convergence.py` 的常數。每一個 `LoopBudgets` 欄位若是 `None` (代表 NOT_ENFORCED), `budget_sources` 就必須說明真正的原因, `validate_contract()` 會拒絕缺少理由的合約——誠實地承認「這個迴圈目前完全沒有 wall-clock deadline、也沒有 run-wide 的 iteration 上限」,而不是假裝有界。

```
dv-harness loop-contract states|list|show <loop_id> [--format yaml]|observe
```

收斂、Plateau 與 Oscillation 偵測(`loop_convergence.py`)

`loop_contract.py` 一開始就誠實聲明自己「不偵測 plateau 或 oscillation」, `observe_*` 只會回報 `PLATEAU_NOT_EVALUATED`。真正的偵測器是 `loop_convergence.py`,它對一段真實的 `coverage_percent` 時間序列(來自 `trend_analysis.daily_rollup()`)分類出七種 verdict: `CONVERGING`、`SLOW_CONVERGENCE`、`NO_PROGRESS`、`PLATEAU`、`REGRESSION`、`OSCILLATING`、`UNKNOWN`。

判斷優先序刻意設計過:先看 `OSCILLATING`(視窗內顯著 delta 的方向反轉次數達到門檻),再看 `REGRESSION`(淨變化跌破 noise floor),再看 `CONVERGING/SLOW_CONVERGENCE`(淨增益是否達標),最後才在「平坦」的情況下區分 `PLATEAU`(連續無增益的樣本數達到 `plateau_window`)與單純的 `NO_PROGRESS`。所有門檻都不是憑空定的魔術數字——`min_gain` 是 noise floor 的兩倍, `plateau_window` 沿用這個專案「兩次獨立觀察才算數」的一貫標準(與 `capability_evolution.REPEAT_FAILURE_MIN_OCCURRENCES`、`memory_router.ORGANIZATIONAL_MIN_CONFIRMATIONS` 相同)。

當序列不足兩個樣本、或專案根本沒有 evidence database、或資料庫裡沒有任何一天有 `coverage` 樣本,回傳的是 `UNKNOWN`,並附上明確不同的理由字串(`INSUFFICIENT_HISTORY` / `NO_EVIDENCE_DATABASE` / `NO_COVERAGE_SAMPLES`)——一個從未執行過的偵測器,和一個執行了但沒找到異狀的偵測器,是兩件不同的事,絕不能混為一談。

`investigate_plateau()` 進一步呼叫既有的 `coverage_analysis.classify_coverage_hole()`,把 `plateau` 拆解成「哪些 bin 尚未充分取樣(應該 ADD_SEEDS)」、「哪些是真正的 stimulus gap(可自動生成 testcase/調整 constraint)」、「哪些是需要人類確認的不可達 stimulus(ESCALATE_TO_HUMAN)」,這個模組只負責分類與命名該用的 `escalator`(`coverage_analysis.escalate_unreachable_holes()`),自己完全不寫入任何東西,也不會代替人類做出升級動作。

```
dv-harness loop-contract convergence
```

統一預算引擎與斷路器(`loop_budget.py`)

在這之前, `policy.max_stage_retries`、`policy.inner_react_max_iterations`、`context_budget.MAX_PACK_BYTES` 各自為政,沒有人能回答「這次 run 在哪个維度上花了多少、對應哪個上限、有沒有耗盡」。 `loop_budget.py` 定義了 spec 第 91 節的十一個預算維度(`max_iterations`、`max_wall_time`、`max_lsf_jobs`、`max_parallel_jobs`、`max_retries`、`max_failed_experiments`、`max_compute`、`max_license_usage`、`max_token_cost`、`max_external_calls`、`max_code_change_scope`), `BudgetEngine` 把每個維度的真實上限、真實花費、以及花費的來源(或「為什麼沒有上限」的誠實理由)都持久化到 `.dv-harness/loop_budget.json`。

值得特別注意的一點: `max_retries` 並不是直接把 `policy.max_stage_retries` 當成 run-wide 的上限——後者是 per-graph-node 的預算,會在 `current_stage` 移動時重置,若直接當成整個 run 的上限,會把「第二個 stage 又用完重試」誤報成「整個 run 的預算耗盡」。這個模組會把每個 node 的花費累加進帳本,讓 run-wide 的總花費是可見的,但真正的 run-wide 上限要靠專案自己在 `loop_budget.limits.max_retries` 明確宣告。

`reset()` 與 `reset_breaker()` 都要求真實的 `reason` 與 `by`,而且會留下一筆不可竄改的紀錄——「預算耗盡不能被靜默重置」在這裡是程式碼強制而非文件承諾。斷路器(circuit breaker)有八種真實觸發原因(`BUDGET_EXHAUSTION`、`REPEATED_IDENTICAL_FAILURE`、`OSCILLATION`、`NO_PROGRESS`、`EVIDENCE_INTEGRITY_FAILURE`、`CRITICAL_ENVIRONMENT_FAILURE`、`UNSAFE_MUTATION`、`DESTRUCTIVE_ACTION`),一旦 `OPEN` 就會在 `engine.loop()` 每個週期最前面(僅次於 `Human Override` 檢查)擋下新的動作,直到有人執行:

```
dv-harness loop-budget breaker-reset --reason ... --by ...
```

十類失敗分類(`FailureType`:`TRANSIENT`、`DETERMINISTIC`、`RESOURCE`、`LICENSE`、`ENVIRONMENT`、`TEST`、`DUT`、`VIP`、`INFRASTRUCTURE`、`UNKNOWN`)決定了哪些失敗值得重試——`classify_failure()` 依序檢查真實的 `preflight FAIL`、`failure_attribution.py` 的 `boundary-trace` 判斷(`DUT_BUG/TB_BUG`、`VIP component` 前綴、一組針對 `license/resource/scheduler/timeout/environment/編譯錯誤`的 `text rule`,最後才落到 `sim_log_analysis` 的 `triage category`。`UNKNOWN` 故意可重試——一個尚未被分類的失敗,不該因為這個模組的無知就縮減既有專案的重試預算。

2026-09-06 追加的 `Compile-Fix Loop` 機制(`is_compile_stage_failure()`、`decide_compile_retry()`、`BudgetEngine.decide_and_trip_compile_retry()`)是這套機制最新且最貼近「收斂偵測」主題的延伸:當同一個編譯/`elaboration` 錯誤 `fingerprint` 反覆出現時,這不只是「重複失敗」,而是 `section 108` 所說的「`artifact churn without verified gain`」的真實 `NO_PROGRESS` 觸發點,會連帶讓斷路器以 `NO_PROGRESS` 名義跳閘。這個判斷完全建立在既有的 `repeated_identical_failure_threshold` 機制上,對非編譯失敗、或還沒達到門檻的編譯失敗,行為與之前完全一致。

```
dv-harness loop-budget dimensions|status|classify|reset|breaker-reset
```

遙測事件與 Loop Engineering Center(`loop_telemetry.py`)

`Spec` 第 108 節列出十九種 `loop telemetry` 事件(`LOOP_CREATED`、`LOOP_STARTED`、`LOOP_ITERATION_STARTED` `LOOP_STOPPED`)。這些事件全部透過既有的 `storage.StateStore.event()` 寫進同一份 `.dv-harness/events.jsonl`——沒有第二份 `audit trail`。`emit()` 會拒絕任何不在這十九個名字之列的事件名稱,避免打字錯誤悄悄變成第二十種沒人讀得到的事件。

每一個 `LoopState` 都對應到零或一個事件(`LOOP_STATE_TO_EVENT`),對於五個沒有對應事件的狀態(`READY`、`VERIFYING`、`CANCELLED`、`STALE` 等),`LOOP_STATE_WITHOUT_EVENT_REASON` 記錄了真實原因,而不是留一個沒人解釋的空白。`read_loop_telemetry()` 把這些事件折疊成 `section 107` 要求的表格——`Loop | State | Iteration | Verified Gain | Budget | Plateau | Oscillation | Next Action`——以及十四個 `drill-down` 欄位。這裡刻意區分兩種進度指標:每次迭代都可測的 `gate_verified_stages` (這次迭代到底有沒有讓某個 `stage` 真正 `gate-verified PASS`),與跨 `run` 才有意義的 `coverage_percent` 曲線,兩者絕不混用。

一個從未跑過 `loop` 的專案,`available` 會是 `false`,並且指名真正的產生者(`dv-harness start --loop "<goal>"`)——一次性的 `dv-harness run-stage` 不是 `loop`,不會產生任何遙測。

```
python -m dv_harness.loop_telemetry names|events|rows|show
```

這套機制刻意沒有做到的事

以上四個模組合起來,誠實地劃出了目前這個 `harness` 的真實邊界:預算引擎「量測」了成本維度,但沒有對它們排序或做效益分析(那是 `section 94/95` 的範圍,尚未建立);`plateau/oscillation` 的「分類」已經完成,但 `spec 89-90` 節要求的「`STOP BLIND RETRY` → `reassess` → 換一個實質不同的策略」這個回應迴路仍未建立——`engine.loop()` 目前遇到重試耗盡的 `stage`,依然是走 `graph` 上原本的 `FAIL edge`,沒有任何機制會因為 `PLATEAU` 或 `OSCILLATING` 的判斷而終止或重新規劃一次 `run`。斷路器的 `trip_on_oscillation` 也因此預設關閉,並附上明確理由:在「反應」機制建成之前,預設開啟這個觸發點只會改變既有的路由行為,而這不是這批 `Loop Engineering` 工作決定要做的事。`Project Learning Loop` 目前只能逐筆記錄觀察(`NOT_OBSERVABLE for a project-wide aggregate`),`Capability Evolution Loop` 也完全沒有任何 `section-108` 事件——這兩個迴圈都是由人類指令(`dv-harness research`、治理狀態轉移)驅動,而非由一個持續迭代的 `driver` 驅動,因此「`per run_id` 的一次 `session`」這個概念在它們身上並不成立。

11. 驗證智慧 (跨專案挖掘、信心校準、驗證策略建議、影子驗證)

DV Agent Harness L5 除了執行單一專案的驗證閉環 (build → verify → coverage → signoff) 之外，還有一組被歸類為「VERIFICATION INTELLIGENCE」的模組，負責回答更高層次、跨越單次 run 甚至跨越專案邊界的問題：這個 root cause 是不是在別的專案也發生過、這個 harness 過去產出的 HIGH/CONFIRMED 結論到底可不可信、面對這個驗證目標該用模擬還是該考慮 formal，以及一個提案中的能力變更能不能在不動到正式程式碼的情況下先跑一次「影子」實驗來驗證。這四組能力都遵守同一條紀律：只讀既有真實證據、絕不臆造閾值或成功率，並且誠實揭露自己目前做不到什麼。

11.1 跨專案模式挖掘 (cross_project_mining.py)

capability_evolution.repeated_unresolved_failure_patterns() 早已能在「單一專案」的 Job Memory 裡，找出同一個失敗特徵在多次「獨立」執行中反覆出現、卻始終沒有 verified_fix 關閉的情形。cross_project_mining.py 把這個邏輯往上提一層：如果你手上不是一個專案，而是好幾個各自獨立、各自有自己 .dv-harness/ 記憶庫的專案，同一個 root cause 有沒有在其中兩個以上的專案裡都出現過——尤其是「A 專案已經有 gate 驗證過的 verified_fix，B 專案的同一個 signature 卻是 OPEN」這種可轉移修復 (transferable_fix) 的情況。

失敗身分的定義完全重用 evidence_db.signature_key()，絕不第二次發明「這算不算同一個失敗」；Job/Engineering 記憶種類則直接 import capability_evolution 的 REPEAT_FAILURE_JOB_MEMORY_KIND / RESOLVING_ENGINEERING_MEMORY_KIND 常數。這裡的「獨立觀察」單位刻意選在專案這一層：一個專案跑了四十次都記錄同一個 signature，那仍然只算「一個」跨專案觀察，因為四十次重跑只是一個環境重複回報同一個事實。CROSS_PROJECT_MIN_PROJECTS = 2 (與 memory_router.ORGANIZATIONAL_MIN_CONFIRMATIONS、REPEAT_FAILURE_MIN_OCCURRENCES 同樣的道理：第二個獨立來源才算得上「不是同一個來源講兩次」)。

為了讓這條門檻不能被作弊，ProjectRegistry.register() 會拒絕註冊一個與已註冊專案共用任何 memory_id 的 root——也就是說，不能把同一個 memory store 用兩個名字登記兩次，硬湊出「兩個專案都同意」的假象；mine_cross_project_patterns() 執行時也會重新檢查一次這件事。

實際操作方式：

```
dv-harness cross-project register <other_project_root> [--id my_project_2]
dv-harness cross-project mine [--min-projects 2]
dv-harness cross-project status
```

mine 的結果一定回報三種狀態之一：STATUS_OK (挖到了什麼，或誠實地什麼都沒挖到)、STATUS_INSUFFICIENT_PROJECTS (樣本數不足兩個專案，這是關於「樣本」的誠實答案，不是「這些專案沒有失敗」的負面結論)。這個模組完全不寫入任何記憶層級：它只讀，promotion_readiness() 只報告「若要晉升到 Organizational Memory 還缺什麼」，真正的晉升路徑仍然只有 memory_router.promote_to_organizational() 的三道關卡。

誠實揭露的邊界：目前 DV Agent Harness L5 這個 repo 本身只有一個專案、一個 memory store，沒有第二個真實專案可以拿來挖掘——這與「Environment Generation Mode」一節揭露「這個 harness repo 本身沒有 RTL tree」是同一種誠實。機制已經用多個真正獨立建立的 memory store 證明可以運作 (見 dv_harness_tests/test_cross_project_mining.py)，但從未在正式作業中產出過一個真正的跨專案發現。另外它目前是 CLI verb (REACHED)，但沒有掛在任何 stage/graph node 上，也不會在 stage boundary 自動觸發——註冊行為刻意保留給人類決定，避免「哪些專案算彼此同意」取決於 harness 恰好在哪裡被呼叫。

11.2 信心校準引擎 (confidence_calibration.py)

inference.score_confidence() 會為一個結論打出 HIGH/MEDIUM/LOW 信心分級，MemoryConsolidator.from_closed_finding() 則會在通過 single_sim + regression + re-audit 三道關卡後，額外授予第四級 CONFIRMED。這四個等級被整個 harness 當作「可信程度」在使用：HIGH 可以被 inference.promote_if_high_confidence() 推進共用的 Knowledge Center，只有 HIGH/CONFIRMED 能進入 Engineering Memory，LOW 甚至不被允許進入 qualified_conclusion.build_qualified_conclusion()。但問題是：從來沒有人回頭檢查過，這些等級的「歷史表現」是否真的對得起它們被賦予的重量。

memory.py 其實一直都在記錄後續發生了什麼——MemoryGC.confirm() 是唯一被授權寫入 confirmation_count 的地方 (代表「一次獨立、稍後的執行，用新證據重新得出同樣結論」)、MemoryGC.retract() (結論被證明是錯的)、MemoryGC.supersede() (被更新、更正確的記錄取代)。confidence_calibration.py 做的就只是把這些既有記錄讀回來，它從不重新評分任何一個結論、不重跑任何 gate、也不寫入任何一筆新紀錄。

刻意不做事，是這個模組最重要的紀律：它不發明任何一個「HIGH 應該代表 90% 準確率」這種數字。這個 harness 裡沒有任何地方宣稱過某個信心等級對應多少百分比的成功率——每個等級的意義是一個「程序性門檻」(需要多少佐證、通過了哪些 gate)，不是一個宣告的成功率。可以在不臆造任何數字的前提下檢查的，是排序：這個 harness 實際採用的次序是 CONFIRMED > HIGH > MEDIUM > LOW，如果歷史紀錄顯示某個「較高」等級的結論實際站得住腳的比例，明顯低於某個「較低」等級，那就是這套排序被自己的歷史紀錄打臉了。每個等級要有絕對門檻，必須由專案自己在 tier_reliability_floor 裡宣告；預設值一律是 None，並且附上為什麼是 None 的真實理由——就像 loop_budget.py 對它十一個預算維度同樣誠實的做法。

判斷樣本數是否足夠的兩個閾值互相推導而來，不是憑感覺選的：MIN_DETERMINATE_OUTCOMES_PER_TIER = 10 是使 $1/N \leq 0.1$ 成立的最小 N (也就是任何低於這個數字的比率，其解析度都比要比較的等級差距還粗)，INVERSION_TOLERANCE 沿用同一個 0.1 作為容忍帶。ACTIVE 但從未被重新驗證過的紀錄不算證據——把它算成「已驗證」，會人為製造出每個等級都接近 100% 可靠的假象；它會被歸類為 INDETERMINATE_NEVER_RECHECKED，跟 DEPRECATED (退役而非被推翻)、NEEDS_REVALIDATION (明確標示還沒被重新檢查過) 都是各自獨立、意義不同的原因。

實測這個 repo 自己的記憶庫：92 筆紀錄裡有 46 筆帶有等級標記，但整個歷史裡只有一筆真正的 determinate outcome (一筆 CONFIRMED 且真的被 confirm 過)，因此報告誠實地回報 INSUFFICIENT_HISTORY，而不是硬算出一個看起來很像結論的百分比。

指令入口目前只有 python -m dv_harness.confidence_calibration tiers|report|show——尚未掛上 dv-harness CLI verb (同一個工作階段裡 cli.py 正被其他並行工作修改，避免衝突)，也沒有任何 run_stage() / advance() 呼叫點，屬於「REACHED，非 WIRED」的能力。讀取行為本身不具備任何寫入副作用：一個沒有 memory store 的專案會直接回報 NOT_AVAILABLE，而不會去建構一個原本不存在的 MemoryStore (那個建構子本身會順手 mkdir 並寫出空的 index.json)。

11.3 驗證策略優化器 (`verification_strategy.py`)

VI-4 要回答的問題是：面對某個驗證目標，該用 simulation、formal、PSS、emulation (ZeBu 級) 還是 FPGA 原型 (HAPS 級) ？在動手之前先做過一次全庫 grep 確認：這個 harness 目前**唯一真正可執行的驗證能力就是模擬**——一條經過 `preflight.run_preflight()` (license/queue/host 檢查) 把關、透過 `lsf_client.bsub_submit_with_preflight()` 送進 LSF 的 VCS regression。沒有任何 formal 工具、PSS 工具、emulator 或 FPGA 原型被整合進這個 repo 的任何地方——沒有 adapter、沒有 client 模組、沒有設定鍵、沒有指令範本。

正因為如此，這個模組的核心設計原則是：「**可執行性**」永遠從程式碼推導，絕不用文字硬寫。每個策略都宣告了自己需要的後端模組與進入點 (`STRATEGY_BACKENDS`)，`derive_executability()` 透過真正的 import 系統去解析它們——SIMULATION 解析得到，回報 `EXECUTABLE_HERE`；其餘四個解析不到，回報 `RECOMMEND_ONLY_NO_BACKEND`。哪天真的有人寫出 `dv_harness/formal_client.py` 並具備該進入點，FORMAL 這一系列會自動翻成可執行，完全不需要改任何狀態字串；`assert_no_unexecutable_strategy_claimed_executable()` 與 `assert_executability_matches_code()` 會在每次產出建議、以及 `capabilities` 這個 verb 被呼叫時，再次確認這件事沒有被悄悄改變。

「可執行性」和「訊號是否建議」被刻意放在**兩個互不影響的軸線**上：一個策略可以同時是「訊號建議採用」又是「這個 harness 目前做不到」——這正是 `protocol_capability.py` 為協定能力欄位做過的同一種拆分，套用在驗證策略上。訊號本身完全重用既有模組的既有輸出，不重新測量任何東西；覆蓋率收斂困難度來自 `loop_convergence.classify_loop_convergence()` (其內部又跑 `coverage_analysis.classify_coverage_hole()` 對每個 bin 的判定)；失敗密度來自 `capability_evolution.repeated_unresolved_failure_patterns()` 與 `evidence_db` 唯讀的 `failure_signatures` 表；每協定的真實能力來自 `protocol_capability.capability_for()`；多子系統範疇來自 `environment_mode_router.read_registered_subsystem_entries()` (只有在真正 gate 驗證過的 SIGNOFF PASS 之後才會寫入的 registry)。

有一條優先順序規則值得特別提：只要**任何一個 coverage bin** 還沒被充分取樣過，FORMAL 就會被標成 `SUPPRESSED` (而不只是「不建議」)——因為隨機化根本還沒被公平地試過，這時候在此基礎上斷言「這是結構性不可達」是最昂貴的一種錯誤答案；同一個 bin 累積到 20 次以上真正獨立的 seed 之後，答案才會反轉為建議 FORMAL。而且，只要建議的策略是這個 harness 目前無法執行的 (`RECOMMEND_ONLY`)，報告裡一定會附上一個 `executable_next_action`——這個 harness 「今天真的能做」的事 (把它送進 `escalate_unreachable_holes()`、把 `capability candidate` 存檔、或把問題送進 `question queue`)，而不是停在一句「請用 formal」就結束，這正是 `loop_convergence.PlateauInvestigation.escalator` 早已建立的同一種契約。

實際操作：

```
dv-harness verification-strategy capabilities
dv-harness verification-strategy recommend --goal "close remaining LTSSM holes" \
--scope SUBSYSTEM --protocol PCIe --holes holes.json [--json]
```

`recommend` 只要建議中出現一個此 harness 無法執行的策略，就會回傳 exit code 2——這是一個「有人得做決定」的 CI 訊號，不是任何方向上的核准訊號。`goal_text` 一律逐字保存並標記 `goal_text_machine_evaluated: false`——目標文字裡出現「formal」這個詞，絕不會替 FORMAL 這一系列多加一分。

誠實揭露的邊界：這個模組**永遠只能建議**，不可能替使用者「跑」formal/PSS/emulation/FPGA 原型中的任何一種，除非哪天真的有人接上真正的後端；吞吐量門檻 (`DEFAULT_THROUGHPUT_BOUND_RUNTIME_HOURS_PER_DAY = 24.0`) 是一個「可被專案覆寫、附帶理由」的經驗法則，不是量測值——這個 harness 讀不到 farm 產能、license pool 大小或排程資訊；它跟前一節的跨專案挖掘一樣，是「REACHED、未 WIRED」的能力，沒有掛上任何 graph node 或 dashboard 卡片。

11.4 影子驗證與能力演化的受控實驗

「影子驗證 (Shadow / Digital-Twin Validation)」在這個 harness 裡並沒有另外開一個叫 `shadow_validation.py` 的新模組——事實上，`capability_evolution.run_controlled_experiment()` 早已經是這個機制的本體：兩份完全隔離的 fixture 副本 (一份保持原樣、代表「目前正式生產行為」；一份帶著候選變更、代表 shadow/twin)，都透過真正的 `DVHarness.run_stage()` 跑過同一組 stage，並透過 `control_plane.describe_stage()` 這個共用讀取路徑量測 gate 結果——把兩者放進一個各自獨立的 gate 通過情況做比較。與其在旁邊另建一套「影子執行器」，這個 harness 選擇承認：這正是同一件事的另一個名字，重複建置只會製造出 Methodology Consolidation Rule 明確禁止的平行機制。

`capability_evolution.py` 的十一狀態治理機 (`DISCOVERED` → `EVIDENCE_GATHERING` → `PROPOSED` → `EXPERIMENT_APPROVED` → `EXPERIMENTING` → `BENCHMARKED` → `PROMOTION_CANDIDATE` → `HUMAN_APPROVED` → `PRODUCTION` → `REJECTED/ROLLED_BACK`) 真正被補上的，是**這個機制外圍的晉升流程**：

- `run_shadow_replication()`：對一個已經到達 `BENCHMARKED` 的候選重複再跑一次 shadow run (絕不改變治理狀態，因為多一次重複量測只是「更多的證據」，不是「往前一步」)；
- `regression_safety()`：因為一個淨值 (`IMPROVED/UNCHANGED/DEGRADED`) 可能藏著「某個 stage 多過了兩個 gate、另一個 stage 少過了一個 gate」這種抵銷，所以另外提供逐 stage 的檢視，任何一個被停止測量的 stage 也視為不安全；
- `assert_stability_window()`： `BENCHMARKED` → `PROMOTION_CANDIDATE` 這條邊新增了一個前提條件——至少要有 `STABILITY_WINDOW_MIN_RUNS = 2` 次可獨立驗證的重複量測 (同一批 stage、結果一致為 `IMPROVED` 或 `UNCHANGED`、無任何 stage 退步、且附上非空的 `rollback_plan`)；
- `shadow_rollback_manifest()`：把「如何回滾」變成真正的資料——從實驗那份未被動過的 baseline arm 反推：mutation 動過的每一個檔案，原本的內容是什麼，回滾動作是 `delete` 還是 `restore_content`，並附上可核對的 digest。

每一次量測 (不論是第一次 benchmark 還是後續的 shadow replication) 都被候選記錄自己的欄位「釘住」——`record_path + record_digest`，每次核可都會重新從磁碟讀取那份記錄、重新雜湊比對，確認它沒有被人手動編輯過、確實屬於這個候選、這次執行——任何一次偽造或竄改都會被拒絕。

沒有任何一道人工核准關卡被動過：`PROMOTION_CANDIDATE` 依然是人類才能推進的狀態，`assert_human_approval()` 底下真正的 `ControlPlane` 檢查沒有被繞過，PR-only 的 main/master 治理也維持原樣——多幾次 shadow replication 買到的只是「更值得信任的證據」，從來不是「更高的授權」。

誠實揭露的邊界 (也是這整套機制目前最重要的天花板)：這裡能比較的只是這個 harness 能量測到、且不需要 ground truth 的東西——gate 是否通過、stage 是否完成、逐 stage 是否退步、gate 結果摘要的 digest；真正的準確率、偽陽性/偽陰性率、覆蓋率增益、執行時間、資源成本、人工審查負擔 (VI-3 原始規格列出的比較項) 需要一份這個 repo 沒有、也不該憑空捏造的「已標記正確答案」語料庫。「Limited Rollout」與「Revalidation」這兩個對正式環境動手的節點沒有被實作——那已經超出 Level C 人工治理的邊界；回滾清單只會被「產出」，從不會被這個模組自動「套用」。同樣地，這是 REACHED

(有真正的呼叫者——`research-architect` 流程與其測試套件) 而非 WIRED 的能力：沒有任何 CLI verb，mutation 本身仍然是由執行實驗的人 (或 agent) 依照候選的 `proposed_action` 手動撰寫，不是自動從文字推導出來的。

11.5 四者如何合在一起用

這四個機制彼此獨立、刻意不互相依賴 (例如 `verification_strategy.py` 只讀取 `cross_project_mining / capability_evolution` 已有的輸出，從不重新推導)，但它們共同構成的問法是遞進的：先問「這個問題別的方案是不是也遇過、有沒有現成的修法」(cross-project mining)，再問「這個 harness 過去給出的信心等級到底可不可信」(confidence calibration)，接著問「面對眼前這個驗證目標，該走哪條路、這個 harness 今天真的能執行哪一條」(verification strategy)，最後，如果答案是「要改動這個 harness 自己的能力」，則透過受控實驗與影子驗證，在正式程式碼被真正碰到之前，先在完全隔離的沙盒裡量出「改了會不會真的變好」。四者共享同一套紀律：從不臆造一個數字或一道門檻，任何「做不到」都寫成一個可檢查、附理由的狀態，而不是沉默地消失。

12. 覆蓋率結案、Waiver、Signoff 與 Golden Scenario

本章涵蓋 DV Agent Harness L5 在專案「結案」前後最重要的一組機制:覆蓋率是否真的關閉、waiver 是否還有效、signoff 當下的證據能否被凍結、以及一個「曾經 PASS」的紀錄多久之後還算數。這些機制彼此獨立成模組,但共享同一個設計原則——**衍生 (derive) 而非儲存 (store)**:任何「現在還算數嗎」的答案都是在每次讀取時,對照真實證據重新計算出來的,從不把一個判斷寫死存進資料庫再照抄。這正是 Evidence Truth Rule 在結案流程上的具體實作。

12.1 覆蓋率關閉公式:Closure = Covered + ApprovedWaiver + ProvenUnreachable

`dv_harness/functional_coverage_signoff.py` 回答「功能覆蓋率是否已經關閉到可以簽核」這個過去只能由人工比對三份不相關文件才能回答的問題:waiver ledger (`waiver_store.py`)、覆蓋率 hole 分類器 (`coverage_analysis.py`),以及 `env_manifest.py` 的 `testplan_correspondence`。它是一個**唯讀 rollup**,不重新量測任何事實,只是把三個既有來源的結果合併成一個 Closure 百分比與一個 `FUNCTIONAL_COVERAGE_SIGNOFF_READY` 判定。

公式本身很直白:

```
Closure % = 100 * (Covered_bins + ApprovedWaiver_bins + ProvenUnreachable_bins)
              / Total_declared_goal_bins
```

- `Covered_bins`:testplan 宣告、且在 `evidence_db.EvidenceStore` 的 `coverage_samples` 表中有真實紀錄數字的類別(不是可能過期的 `summary.json`),取其 `bins_hit`。
- `ApprovedWaiver_bins`:僅當 `waiver_store.status_report()` 中有一筆狀態為 `VALID` 的 waiver,其 `item` 欄位「逐字」對到該覆蓋率類別名稱(不做模糊比對)。
- `ProvenUnreachable_bins`:僅當該類別被 `coverage_analysis.classify_coverage_hole()` 分類為 `UNREACHABLE_STIMULUS`,而且這個判斷已經透過 `question_queue.QuestionQueueStore.answer_question()` 被真人回答 `STRUCTURALLY_UNREACHABLE` ——Tier-2 的自動假設或 Tier-1 的自我解決都不算數;最近一次的 `ANSWERED` 紀錄才是最終依據,人可以事後推翻先前的確認。
- 若一個 hole 是 `INSUFFICIENT_SEED_ATTEMPTS` (seed 數量還沒到門檻),永遠不會算進 `ProvenUnreachable_bins` ——「還沒測夠」與「結構上不可達」在分類器層就已經分開,不會混進同一條路徑。
- 一個類別若同時符合 waiver 與 unreachable,只算一次,不重複計分。

`FUNCTIONAL_COVERAGE_SIGNOFF_READY` 唯有在以下條件同時成立才為真:evidence database 存在且至少一個宣告的 bin 可查;每一個宣告的 bin 都有真實紀錄數字(有任何一個 testplan 宣告卻完全沒被測過的類別,會回報 `INCOMPLETE_EVIDENCE` 並拒絕 `READY`,而不是靜靜地把它排除在分母之外);Closure 達到 100%;而且沒有任何一筆 waiver——不論它是否是被拿來記分的那一筆——狀態為 `EXPIRED`、`REVOKED` 或 `UNKNOWN`。換句話說,即使測量到的 Closure 已經 100%,只要 waiver 清單裡混著一筆壞掉的 waiver,也一樣擋下簽核。

evidence database 不存在時整份報告是 `NOT_AVAILABLE`;waiver ledger 不存在時只有該欄位回報 `NOT_AVAILABLE`,但貢獻真實的零分與零阻擋(尚未採用 waiver ledger 的專案不會被追溯判失敗);`env.manifest.json` 不存在時範圍會擴大到 evidence database 曾經記錄過的所有類別。目前沒有 `dv-harness CLI` 子命令(`cli.py` 在寫作當下正被並行工作修改),前門是:

```
python -m dv_harness.functional_coverage_signoff
```

刻意的邊界:它不核准、不撤銷 waiver、不提問也不回答問題,判定只是給人類簽核決策的輸入,和 `golden_flow_readiness.py` / `platform_health.py` 的角色一樣。`REVALIDATION_REQUIRED` 的 waiver 既不記分也不阻擋簽核。

12.2 覆蓋率結案動作的效用排序

當覆蓋率 hole 分析完之後,通常會有多个候選補救動作(寫新測試、放寬過度限制的 sequence、加 waiver、升級為不可達),但預算不足以全部執行。

`dv_harness/coverage_closure_action_utility.py` 用四個因子的效用公式對候選動作排序:

```
utility = expected_coverage_gain * requirement_priority * risk_coverage / cost
```

四個因子全部是**呼叫端宣告**(`status = DECLARED`),絕不自我量測——因為「這個動作能帶來多少覆蓋率增益」本質上是人或上游規劃 agent 對尚未執行的動作所下的判斷,不是這個 harness 能跑出來的事實。任一因子缺失或不是正的有限數,該動作會被標記 `UNRANKABLE` 並列出缺什麼,絕不補一個假設值(0、1 或同儕中位數)硬讓它參與排名。

真正的核心規則是「便宜但錯誤的動作絕不能靠成本低而贏得正確的動作」,這是一道在**成本介入之前**就先擋下來的閘門,而非把 `correctness/risk` 揉進效用公式裡當懲罰項: `correctness_status = FLAGGED_INCORRECT` 或 `risk_status = HIGH_RISK` 的動作會被整個排除出候選集(`EXCLUDED`),而不是被打低分放到排名底部——因為足夠大的 `gain/priority/risk_coverage` 理論上仍可能把它拉回排名前段。

```
python -m dv_harness.coverage_closure_action_utility --candidates-file <file.json> [--json]
```

輸出分成 `ranked` / `excluded` / `unrankable` 三組,退出碼 0 表示沒有 unrankable 項目、1 表示至少一項無法排名。這個模組只**排序、不執行**:不寫測試、不加 waiver、不跑 regression。

12.3 Waiver Ledger:waiver 閘門的唯一真相來源

Spec 第 237 節要求 waiver 具備過期/重驗證機制與五種狀態 (`VALID` / `REVALIDATION_REQUIRED` / `EXPIRED` / `REVOKED` / `UNKNOWN`)。在 `dv_harness/waiver_store.py` 把這條線接上之前,waiver 是**自我證明**的: `gates.run_gate()` 組裝 waiver gate 的 payload 全部來自 agent 自己寫的 fenced `dv-harness-evidence:<gate_id>` 區塊,同一個 agent 既寫 waiver 也寫「waiver 仍然有效」的證據——一個過期的 waiver 不會被重新標記,只是不再被提起。

現在,三個真正的 gate—— `waiver_scope_consistency_gate`、`waiver_revision_freshness_gate`、`waiver_revalidation_gate`——都會去讀 `.dv-harness/waivers/waivers.json`。只要這個 ledger 存在,它就是權威來源:被評估的 是 ledger 裡的紀錄,agent 引用一個 ledger 裡沒有的 `waiver_id` 會直接 FAIL `WAIVER_NOT_IN_STORE`。ledger 裡的 waiver 在**每一次執行**都會被重新評估——這正是 重點:一旦過期,它所豁免的要求會被重新標記,不管有沒有人提起。

`status` 是**衍生欄位**,從**不儲存**,`derive_status()` 依「最壞優先」順序判定 (`REVOKED` → `UNKNOWN` → `EXPIRED` → `REVALIDATION_REQUIRED` → `VALID`),所以人類的撤銷 永遠蓋過時鐘。缺少第 237 節要求欄位的紀錄,或沒宣告到期時間/重驗證觸發條件的紀錄,一律回報 `UNKNOWN`——「無法檢查」永遠不等於 `VALID`。可觸發重驗證的欄位僅限 `SUPPORTED_TRIGGER_KEYS` (`spec_revision` / `rtl_hash` / `revision`),因為 `GATE_STATUS_CONTEXT` 記錄了哪個 gate 真的量測什麼——宣告一個沒有 gate 在量測的 觸發條件會被直接拒絕。

沒有 ledger 的專案行為與原本自我證明的模式**位元組級相同**——採用 ledger 是專案的 選擇,未遷移的專案不會被追溯判失敗。目前沒有 `dv-harness` CLI 子命令(`cli.py` 同期被並行工作修改),前門是:

```
python -m dv_harness.waiver_store statuses|list|status
```

外加 dashboard 的 Waiver Authoring 表單。**刻意的邊界**:這個模組不仲裁任何事——不撤銷、不重驗證、不批准任何 waiver;撤銷 waiver 必須由 `revoke_waiver()` 指名一個 真人與理由。另外三個消費 waiver 的 gate (`coverage_hole_regeneration_gate`、`coverage_hole_to_test_generation_gate`、`sequence_coverage_closure_gate`)尚未接上這個 ledger。

12.4 Signoff Freeze / Baseline:凍結十五個欄位,偵測凍結後的變動

Spec 第 238 節要求 signoff 時凍結一份可重現的 baseline(十五個欄位:spec 版本、requirement/vPlan 版本、DUT/TB SHA、agent/skill 版本、VIP/工具版本、schema/policy 版本、configuration、測試清單、coverage database、assertion 狀態、waiver、evidence hash、dashboard snapshot、reproducibility capsule),並在凍結後 若有實質變動要能觸發影響分析並使受影響的 signoff 證據失效。`signoff_export.py` 原本只打包了 11 個候選檔案成 bundle,完全沒有這十五個欄位的概念,而且 `compute_bundle_hash()` 只雜湊「artifact 是否存在」,不雜湊內容——一份被打包的 檔案內容可以整個被替換掉而不動到 bundle hash。

`capture_baseline()` 把十五個欄位全部從真實產生者衍生出來,不是任何一個手打的 版本字串:`dut_sha` 來自 `connectivity_check.compute_rtl_fingerprint()` (與 `just connectivity-check` 判斷 RTL 是否變動用的**同一個**內容指紋,而非會隨 README 變動而移動的 git SHA);`waivers` 來自 `waiver_store.status_report()` 的 衍生狀態(這讓一個 waiver 在 signoff 之後過期成為可偵測的凍結後變動);`reproducibility_capsules` 來自 `golden_scenario.load_golden_scenarios()`; `evidence_hashes` 是 `normalized_evidence` 的 `evidence_id` 集合。`spec_version` 與 `dashboard_snapshot` 兩個欄位**誠實地回報 NOT_AVAILABLE**:前者這個 repo 沒有任何真實產生者(只有 agent 自證的 evidence-block 文字,凍結它當作事實正是 Evidence Truth Rule 禁止的行為;人類可以宣告一個,會被記為 `attested: true, machine_verified: false`);後者是因為 dashboard 是即時渲染,沒有一個 snapshot artifact 可凍結。

`evaluate_freeze_invalidation()` 是三個獨立比對、最壞優先:(1) 十五個欄位現在重新 衍生一次並比對 `digest`,凍結時就存在的欄位若變動或再也無法擷取 ⇒ `INVALIDATED`;凍結時 `NOT_AVAILABLE`、現在能擷取到的欄位 ⇒ `INDETERMINATE` (證據事後出現不能證明凍結當時的證據有問題)。(2) 對凍結時記錄的 git HEAD 執行真正的 `change_impact.changed_files()` + `classify_risk()`——與 `golden_scenario.evaluate_freshness()` 用的是同一套判斷。(3) 重新讀取 bundle 的 `manifest.json` 並獨立重算 `compute_bundle_hash()`。

凍結只會發生在 `collect_signoff_bundle()` 判定 bundle 為 `SIGNOFF_GATE_VERIFIED`——也就是 9 個真正的 `STAGE_GATES["SIGNOFF"]` gate 都真的通過的時候; `PRE_SIGNOFF_GATE_INPUT` 的 bundle 刻意不凍結。凍結紀錄寫入 `.dv-harness/signoff/freezes/<freeze_id>.json`,並在 `.dv-harness/events.jsonl` 留下一筆 `SIGNOFF_BASELINE_FROZEN` 事件(`dv-harness audit` 可讀到)。

```
dv-harness signoff-export --out <dir> [--require-signoff-pass]
python -m dv_harness.signoff_export fields|baseline|freeze|list|status
```

`freeze` 動作**強制要求 --frozen-by**——無法歸責的 baseline 不算 signoff baseline。**刻意的邊界**:它只記錄與回報,不仲裁、不授權——沒有 stage gate、沒有「revalidate」動詞,因為判斷一個 `INVALIDATED` 的 signoff 是否可接受是人類判斷,重新 凍結一個變動過的專案只是再叫一次 `freeze`,並在上面署一個人名。這個 repo 自身的 baseline 目前只能誠實捕捉十五個欄位中的 4 個(它沒有 RTL tree、沒有生成的 TB、沒有 VIP 安裝、沒有自己的 waiver ledger 或 coverage database)。

12.5 vPlan-Scoped Baseline

`dv_harness/vplan_baseline.py` 是同一套凍結/失效模式,但範圍縮小到 vPlan/需求/配置這個三元組: `spec_version` (vPlan 自己的 `spec_revision` 欄位或人工宣告)、`requirement_ir_version` (第 184 節 Canonical Requirement Contract IR 內容身分)、`configuration_ir_version` (任意 configuration-variant IR 文件的內容身分——刻意 不驗證其內部合法性,那是 `config_variant_coverage.py` 的職責)、`vplan_items` (vPlan 項目數量加上以 `req_id/item_id` 為鍵的內容雜湊聚合)。凍結詞彙 (`CAPTURED` / `NOT_AVAILABLE` / `FREEZE_VALID` / `FREEZE_UNKNOWN` / `FREEZE_INVALIDATED`) 直接從 `signoff_export.py` 匯入,不重新定義,讓「凍結判定」這三個字在整個 codebase 中意義一致。目前沒有 `dv-harness` CLI 子命令,前門是:

```
python -m dv_harness.vplan_baseline fields|baseline|freeze|list|status
```

12.6 Golden Scenario / Reference Capsule:通過不代表永遠通過

Spec 第 225 節的規則很直白:「Golden 不代表永久有效,相關的 RTL/spec/工具/config 變動都能讓一個 capsule 變成 STALE」。`dv_harness/golden_scenario.py` 就是這個 capsule store,它記錄「測試 T 在 seed S、配置 C 下,對某個特定 commit 驗證過 PASS」這個宣告,並把它存進 `evidence_db.EvidenceStore` 新增的 `golden_scenarios` 表,採用與 `jobs/normalized_evidence` 相同的 natural-key upsert 慣例。

`record_golden_scenario()` 會拒絕任何 `evidence_id` 不是一筆既有 `normalized_evidence` 紀錄、判定不是真正 PASS、或 `test_name` 與該證據列的 `pattern` 不符的 capsule——`verified_sha` 是從該筆證據所屬 job 的真實 `jobs.git_sha` 讀出來的,不是手打的。

新鮮度是即時計算,從不儲存。 `evaluate_freshness()` 執行真正的 `change_impact.changed_files()` (真的 `git diff <verified_sha>..HEAD`)加上真正的 `classify_risk()` HIGH/MEDIUM/LOW 模型——與 regression 選測鏈用的是同一套。若 capsule 宣告的 `watched_paths` 內出現 HIGH(設計 RTL)或 MEDIUM(testbench/ sequence/command.txt/config)變動 $\Rightarrow$ STALE 並列出檔名;LOW(文件、`.dv-harness` / `.claudie` 內部紀錄)不算。錄下的 VIP/工具版本若與呼叫端提供的目前 版本不符,即使沒有任何 git 變動也一樣 STALE。

```
dv-harness golden-scenario record|list|status \
  [--json-file <capsule.json>] [--capsule-id <id>] [--head HEAD] \
  [--db <evidence.duckdb>] [--vip-version TOOL=VERSION] [--json]
```

退出碼:0 表示錄入成功/全部 FRESH,1 表示至少一個 STALE,2 表示 UNKNOWN 或什麼都沒錄到。

刻意的邊界:(1)「無法檢查」永遠是 UNKNOWN,絕不是 FRESH——沒有 git、無法解析的 SHA、diff 失敗或根本沒錄 SHA,都誠實地回報 UNKNOWN 並附上原因。(2) `watched_paths` 為空時範圍會擴大到整個 repo(`scope: WHOLE_REPO_NO_WATCHED_PATHS_DECLARED`),而不是縮小成空——這個模組唯一容許的失敗方向是「把一個仍然良好的 capsule 誤判成 stale」,絕不是反過來。(3) 它不做任何 決策:不跑測試、不送 job,而且刻意沒有 stage gate——一個對「沒人重跑過」的 capsule 放行的 gate,比沒有 gate 更糟。FRESH 只是給人類決定是否重用的輸入。(4) 一份早於這張表存在的 `evidence.duckdb` 會以唯讀方式開啟(略過 DDL),回報 `NOT_AVAILABLE` 而非崩潰。(5) capsule 只透過刻意呼叫 `record` 才會建立——沒有任何 機制會從一次 PASS 自動生成 golden capsule,因為「哪些 PASS 值得留作 golden」是這個 模組不做的判斷。

Qualification Set 完整性檢查。 `golden_scenario.py` 另外還能回答一個相鄰問題:「這個 subsystem 是否對每一種種類的已證實結果都留有至少一筆 golden capsule?」——乾淨 PASS、已知 DUT bug、已知 TB bug、已知 VIP 問題、protocol violation、timeout、coverage hole、waiver、register 測試、perf 測試,共十類(`QUALIFICATION_SET_CATEGORIES`)。GoldenScenario 新增一個可選的 `category` 欄位(接在既有 `watched_paths` 之後,不影響任何既有欄位), `missing_categories()` 回報哪些必要類別目前一筆 capsule 都沒有,一筆未分類或分類 值不在詞彙表內的 capsule,不會被算進任何類別。`qualification_set_report()` 把結果包成 `COMPLETE / INCOMPLETE` 報告。這個檢查的前門是:

```
python -m dv_harness.golden_scenario qualification-set
```

退出碼 0 表示 COMPLETE,1 表示 INCOMPLETE(附上缺少的類別清單)。它只檢查「已經 記錄了什麼」,不會去猜某個 subsystem 應該要有哪些類別,也沒有對應的 stage gate。

12.7 Configuration Variant Explosion Control:成對覆蓋而非笛卡兒展開

Spec 第 232 節談的是另一個結案前的問題:configuration 空間(protocol generation、speed、lane width、data width、compile defines、feature modes、SKU、clock mode、subsystem 組合、VIP configuration)太大,不能盲目做笛卡兒積 regression。`dv_harness/config_variant_coverage.py` 用真正引用文獻的演算法 IPOG(In-Parameter-Order-General,Lei, Kacker, Kuhn, Okun, Lawrence, IEEE ECBS 2007,也是 NIST ACTS 背後的演算法)產生一組縮減過的 t-way(預設 pairwise,即 t=2)覆蓋組合,而不是枚舉全部合法組合。它與 `change_impact.py` / `regression_tiers.py` 是正交的兩個問題——那兩者選「哪些測試」,這個模組選「哪些 configuration」,兩者的組合(測試集 $\times$ 配置集)由呼叫端自己去做,這個模組刻意不做 那個組合。

被宣告為 `critical_combinations` 的關鍵組合會先被 seed 進去,再由真正的 backtracking 搜尋補完成完整合法配置,並且在後續的冗餘列剪枝時被排除——不會為了 省算力而被砍掉。`forbid` 條件在每一次賦值時都被檢查,一個因為被禁止而永遠無法被 任何合法配置涵蓋的 t-tuple,只有在一次獨立的窮舉 backtracking 搜尋證明確實 無解後,才會被標記為 `UNREACHABLE_UNDER_CONSTRAINTS` ——絕不會回報自己沒有達成的 覆蓋。`verify_coverage()` 是一次獨立的、從頭計算的覆蓋率驗證(不是讀回產生器自己的紀錄簿),所以也可以拿去驗證一份手寫的組合清單;`build_plan()` 產生 plan 時會 跑這個驗證,確保 plan artifact 不能宣稱驗證器沒確認過的覆蓋率。

```
dv-harness config-variants plan|verify --space <file> [--strength N]
python -m dv_harness.config_variant_coverage
```

退出碼:0 完全覆蓋,1 有真實的發現(未覆蓋的交互作用、不合法/不完整的 configuration、缺少宣告的關鍵組合),2 宣告本身有錯或用法錯誤。

刻意的邊界:(1) 它只做選擇,不決定其他任何事——沒有 build、job,也刻意沒有 stage gate。(2) 第 232 節列出的其他選測方法(需求驅動、歷史風險、change-impact 組合)只能以呼叫端宣告的 `critical_combinations` 附帶理由進入,這個模組本身不會 去挖掘或發明任何組合。(3) 等價類縮減(例如把 64 個合法 data width 縮成 {8, 32, 512})是宣告端的行為——「這兩個值等價」是一個需要一手證據支持的協定 行為主張。(4) 值必須是 JSON 純量,結構化的值會被拒絕而不是字串化。(5) `legal_cross_product_size` 只在 `MAX_EXACT_ENUMERATION` (20 萬)以內精確枚舉,超過就誠實回報 `UNCOUNTED`。

12.8 Coverage DB Merge Integrity Gate

把多次 regression 的 coverage database 合併成一個計入 signoff 的總覆蓋率,過去 在這個 repo 完全沒有完整性檢查——`coverage_analysis.py` 自己的 docstring 就先 聲明了這個邊界:它相信一份「已經被某個真實覆蓋率工具化簡成 JSON」的 coverage summary,不會去問這個檔案是不是真的、也不會去問被合併的檔案是否來自相容的工具鏈。`dv_harness/coverage_db_integrity.py` 補上這一段,而且完全重用既有元件:內容指紋用 `env_manifest.py` 的 `file_ref()` (與 `compute_rtl_fingerprint()` 相同的 sha256 折疊方案,泛化到任意檔案或目錄);工具/版本相容性比對用 `env_manifest.py` 的 `generator.tool_version` 與 `vip_config.vip_release` 兩個真實欄位——因為 `coverage_analysis.parse_coverage_summary()` 的 schema 根本不含工具/版本欄位,任何從 summary JSON 裡讀出來的「版本」都是憑空發明的。

三種判定,絕不把後兩者混成一種: `MERGE_ALLOWED` (每個指紋聲明都吻合、每筆記錄的 身分都一致)、`MERGE_BLOCKED` (有真實發現——指紋不符或工具/VIP 版本確實衝突)、`MERGE_NOT_VERIFIABLE` (沒有發現問題,但有些東西根本無法檢查——沒有指紋可比對,或某筆記錄沒有對應的 `env.manifest.json`)。只有 `MERGE_ALLOWED` 退出碼 0;一個「無法驗證」的合併絕不會被靜靜地當作合格算進 signoff。

```
python -m dv_harness.coverage_db_integrity fingerprint --db-path <path>
python -m dv_harness.coverage_db_integrity check --merge-request <file.json>
```

其中 `<file.json>` 形如 `{"entries": [{"db_path", "claimed_fingerprint"? , "env_manifest_path"? , "label"?}, ...]}`。刻意的邊界:這個模組從不打開真正的 coverage database(UCIS/urg/vdb),也不 解析覆蓋率 bin——那條界線屬於已經把資料化簡成 JSON 的真實覆蓋率工具,在這裡發明 一個解析器正是 `coverage_analysis.py` 自己聲明過不能做的事。它只比對檔案內容指紋 與既有記錄的事實,不執行合併、不計算總覆蓋率,也刻意沒有 stage gate。

12.9 這一組模組的共通紀律

以上九個模組雖然職責不同,但貫穿著幾條相同的規矩,值得在使用前一併記住:

1. 衍生優於儲存:freshness(golden_scenario)、status(waiver_store)、freeze 判定(signoff_export/vplan_baseline)全部是每次讀取時重新計算,絕不寫死 存一個布林值——因為那個值在下次 commit 之後就可能是錯的。
 2. 「無法檢查」永遠不等於「通過」:UNKNOWN/NOT_AVAILABLE/MERGE_NOT_VERIFIABLE/INDETERMINATE 這些狀態被刻意保留,不會被壓縮成一個簡單的 PASS/FAIL。
 3. 這些模組只報告,不仲裁:沒有一個會自己跑 build、送 job、核准 waiver 或 撤銷凍結——那些都是刻意保留給人類的決定,也是為什麼它們大多刻意不掛 stage gate。
 4. 多個模組因為 `cli.py` (約 4100 行的 `argparse` 樹)在同一天被並行工作修改,而只提供 `python -m dv_harness.<module>` 這個 ad hoc 前門,不是遺漏,而是在 CLAUDE.md 中明確揭露的已知殘留(disclosed residual)。使用前應先確認 `dv-harness --help` 是否已經補上對應子命令,而不是假設每個模組都有 `dv-harness` 一級命令。
-

13. 治理、Git 保護與供應鏈風險

本章涵蓋 DV Agent Harness L5 在「治理層」上的六個實際機制：Git 保護（PR-Only 政策）、變更爆炸半徑（blast-radius）閘門、依賴 / 供應鏈治理、對測試套件本身的突變測試（mutation testing）、平台健康度與錯誤預算，以及 harness 部署到共用 Agent 路徑的機制。這些模組彼此獨立、各自可單獨呼叫，且都遵守本專案的 Evidence Truth Rule：凡是無法真正測量或檢查的項目，一律誠實回報 `NOT_AVAILABLE` / `UNKNOWN`，絕不偽造一個「乾淨」的結果。

13.1 gh CLI + PR-Only 治理政策（`dv_harness/git_governance.py`）

L5 的硬性規則是：agent 可以 `git branch` / `git commit` / 開 PR（`gh pr create`），但絕對不可以直接 `merge` 或 `push` 到受保護分支（`main` / `master`）。人類的 PR review 才是進入這些分支的唯一路徑。

這條規則有兩層防護，彼此獨立、缺一不可：

1. 主要防線（尚屬 **ASPIRATIONAL**）：server-side branch protection。目前本專案的 `origin remote` 雖已存在，但仍是空的，尚無分支可設定保護規則，因此這一層尚未真正生效。
2. 次要防線（已安裝且實際運作）：`tools/git-hooks/pre-push` 與 `pre-merge-commit` 這兩支 git hook，背後呼叫 `dv-harness git-guard --check <hook-name> [--branch <b>]`。這個指令印出 JSON 決策並以 exit code 0（允許）或 1（阻擋）作為 git 是否真正中止操作的依據——這是 githooks(5) 的行為：pre-push/pre-merge-commit 回傳非零值會讓 git 在操作生效前中止。

偵測邏輯的關鍵在於 `detect_ai_agent_markers()`：它比對環境變數中是否存在 `AI_AGENT_ENV_MARKERS`（這組常數是直接從 `tools/remote/remote_relay.py` 的 Layer 2 防護機制匯入，而非重新複製一份，確保兩處判斷永遠不會分歧）。只要偵測到 agent 環境標記，且目的地分支是 `main` 或 `master`，這個閘門就無條件阻擋——這裡刻意沒有設計 **override** 變數（不像 `remote_relay.py` 的 `DV_HARNESS_RELAY_AUTORECONNECT_OK`），因為沒有任何被允許的自動化流程應該讓 agent 直接寫入受保護分支。反過來說，人類在自己的互動式終端機執行同樣的 git 指令（環境中沒有 agent 標記）則完全不受此模組阻擋——這是針對「agent 主動直接寫入受保護分支」的目標式防護，而不是一套通用的 local branch-protection 重新實作。

每一次觸及受保護分支的 `git-guard` 決策都會以 `GIT_GUARD_DECISION` 事件寫入 `.dv-harness/events.jsonl`，可透過 `dv-harness audit` 追溯。`gh`（GitHub CLI）是開啟 / 檢視 PR 的官方工具（`gh pr create`、`gh pr view`、`gh pr checks`），agent 可以自由執行任何唯讀或建立 PR 的 `gh / git` 指令，但絕不可執行 `gh pr merge` 或等效的直接 `push/merge` 到 `main` / `master`——這個動作被保留給人類。

誠實邊界：這是 defense-in-depth，適用於「尚未有 remote 或尚未設定 server-side branch protection」的情況，以及任何在本機執行 git 且完全沒有 server-side 閘門的 Claude Code session。一旦真正的 server-side branch protection 建立起來，那才是權威的第一層防線。

13.2 變更爆炸半徑閘門（`dv_harness/change_blast_radius.py`）

`git_governance.py` 只回答「這個變更要去哪裡」（目的地分支），完全不管「這個變更影響有多廣」——一行 docstring 修正和一次橫跨 47 個檔案的重寫，對它而言毫無區別。`change_blast_radius.py` 補上這個測量，而且刻意只從真實訊號計算，絕不接受自我宣告：

1. 實際改動的檔案——透過 `change_impact.changed_files()` 得到真正的 `git diff --name-only`，繼承其 `REAL_DIFF` / `NO_GIT` / `UNKNOWN_BASE` / `DIFF_FAILED` 狀態語彙，避免出現第二套並行的 git diff 邏輯。
2. 跨子系統的擴散範圍——對每個被改動的 `dv_harness/*.py` 模組，透過 `ast` 解析套件內部的 `import` 陳述式，計算其「遞移 importer 閉包」。 `models.py` 得分 88，是因為真的有 88 個模組會匯入它到； `autonomy_levels.py` 得分 0，是因為完全沒有模組匯入它——沒有任何一個數字是手動填入的。
3. 治理 / 政策自我修改——這個變更是否觸碰到「本身就是一道閘門」的檔案，由 `autonomy_levels.LEVEL_C_ENFORCEMENT` 的 `cites` 表格加上實際的 hook 腳本推導而來，而非人工列表。

門檻（`WIDE_REACH_THRESHOLD = 40`、`WIDE_FILE_COUNT_THRESHOLD = 25`）是根據這個 repo 實際 import graph 測得的 p90 值（134 個模組，中位數 19，p90 為 40）校準出來的，而非隨意調整。

超過門檻的變更（`WIDE` 或任何 `GOVERNANCE` 檔案）不會被永久阻擋，而是需要額外的真實人類確認：透過既有的 `ControlPlane.approve()`，approval-only stage key 為 `CHANGE_BLAST_RADIUS`：

```
dv-harness blast-radius --base <rev> [--head <rev>]
dv-harness approve CHANGE_BLAST_RADIUS --note "<approval note carrying the digest>"
```

approval 是釘死在該次評估結果上的——note 必須帶有這次評估的 `digest`（一個對「排序後的變更檔案集合 + tier」計算的 sha256）。只要多加一個檔案、少一個檔案，或變更規模擴大，digest 就會變動，舊的 approval 隨即失效——這正是防止它退化成「一次性的萬能豁免」的關鍵設計，與 `ControlPlane.add_cosign()` 對「同一位置後續改變的值不受舊 cosign 保護」的推理完全一致。

這一層是**嚴格附加式**的：`cli.py` 先跑既有的分支閘門（`git_governance`）且維持原樣不變，只有在分支閘門已經允許的變更上，這一層才會運作，並且只能把「允許」變成「阻擋」，絕不能把「阻擋」變成「允許」。若無法計算真實 diff（沒有 git、無法解析的 base、全新分支沒有 merge-base），結果是 `NOT_ASSESSABLE` 且允許放行——環境問題不能偽裝成政策判定。這個閘門也無法被用來繞過 `git_governance` 的分支保護：一個 `CHANGE_BLAST_RADIUS` 的 approval 絕不能解鎖對受保護分支的 push。

13.3 依賴 / 供應鏈治理（`dv_harness/dependency_supply_chain.py`）

在此模組出現以前，這個 harness 無法回答一個基本問題：它是建立在哪些第三方元件之上、版本各是多少、其中是否有任何一個是未被約束的。此模組提供三項真實檢查：

- **PINNED_VERSION** (真實, 隨處可用) ——依照每個宣告依賴自己的 specifier set 分類成 `PINNED_EXACT` / `BOUNDED_RANGE` / `LOWER_BOUND_ONLY` / `UNCONSTRAINED` ; 已安裝的 VIP 套件則依安裝目錄是否真的標明版本目錄來判定。
- **DECLARED_VS_INSTALLED** (真實) ——透過 `importlib.metadata` 對照真正在跑的直譯器, 找出「宣告了但沒安裝」或「安裝版本超出宣告範圍」的落差。
- **VULNERABILITY_ADVISORY** ——在目前環境中是 `NOT_AVAILABLE`, 而且如實這樣回報, 而不是回報一個乾淨掃描結果。 `pip-audit` 與 `safety` 皆未安裝, 而 OSV/PyPI 的線上 advisory API 是網路服務, `LOCAL_ANALYSIS` 執行模式下不得聯外。若專案自備離線的 advisory 資料庫 (`.dv-harness/supply_chain/policy.json` 或 `--advisory-db`), 這項檢查才會真正運作並回報該資料庫自己的來源與 as-of 日期。

`NOT_FULLY_CHECKED` 的排名高於 `POLICY_CLEAN` ——這與 `platform_health.py` 把 `UNKNOWN` 排在 `HEALTHY` 之上的道理相同: 一項無法執行的檢查, 絕不能與「執行過且乾淨」混為一談。

```
dv-harness supply-chain inventory
dv-harness supply-chain check
dv-harness supply-chain advisory-status
```

本專案自身目前的真實結果 (並非宣稱, 而是實測) : 4 個宣告的 Python 元件, 沒有一個是精確釘選版本, 其中一個 `HIGH` (未約束)、兩個 `MEDIUM` (僅下限)、一個 `LOW` (範圍約束), 且兩個宣告了但未安裝於當前直譯器; 整體狀態為 `POLICY_FINDINGS`, exit code 為 1——這不是一個乾淨的安全結果。

誠實邊界: 這不是 SBOM——只涵蓋本專案宣告加上真正已安裝的依賴, 不包含遞移依賴圖 (那需要對套件索引做 resolver 呼叫, 是網路行為, 報告中會明確揭露)。這個模組只讀取、不決定: 不寫檔、不核發 approval、不啟動任何 stage, 也刻意沒有 stage gate。它是 `REACHED` (有真實 CLI 呼叫入口) 而非 `WIRED` (沒有任何 `run_stage()` / `advance()` 呼叫它, 也不在 dashboard 上)。

13.4 對測試套件本身的突變測試 (`dv_harness/mutation_testing.py`)

本專案每一道閘門都由測試把關, 而整套 Evidence Truth Rule 的前提, 就是「當被守護的東西壞掉時, 測試真的會失敗」。在此模組出現以前, 從來沒有任何機制測量過這件事。

範圍必須先講清楚, 避免被誤讀: 這是針對「本專案自身 Python 測試套件」的測試基礎設施, 而不是 DUT/RTL 層級的 fault injection (stuck-at / bit-flip / gate-level 的 fault campaign) ——那需要真實的 RTL 目標與模擬器, 這個 repo 並不擁有, 任何結果都不得被引用為關於 DUT 的證據。它把小型的、AST 層級的錯誤注入到某個 `dv_harness/*.py` 模組中, 重新執行該模組現有的 `dv_harness_tests/test_*.py`, 測量測試是否真的能偵測出這個錯誤。

四種突變運算子 (`COMPARISON_SWAP`、`BOUNDARY_SHIFT`、`BOOL_OP_SWAP`、`BOOL_CONST_FLIP`) 都刻意是「小而標準」的一套, 且絕不改動硬碟上的真實檔案: 每個突變都是在一個子行程中、透過 `sys.meta_path` 的 finder 為特定模組名稱服務被突變後的暫存檔內容; 真實檔案永遠以唯讀方式開啟。整個流程永遠先跑一次未突變的 baseline——如果 baseline 就失敗了, 回報 `BASELINE_FAILED`, 不會跑任何突變, 也絕不會捏造分數。 `SURVIVED` (測試在有錯誤的情況下仍全部通過) 是一個發現, 但不必然是漏洞——有些是「等價突變」 (equivalent mutants), 需要人類審視, 此模組從不宣稱一個存活的突變就證明了測試缺失。

```
dv-harness mutation-test --module dv_harness.qualification
dv-harness mutation-test --min-score 0.8
```

本專案的真實測量結果: `dv_harness.qualification` 得分 1.0 (3/3 全殺), `dv_harness.stats_snapshot` 得分 0.333 (12 個突變中殺死 4 個), 存活的突變都是可行動的真實發現 (例如某些缺檔案時的 `return 0` 分支從未被真正測到)。

誠實邊界: 突變分數在此是一項「測量」, 不是一道閘門——沒有 stage gate、也沒有預設門檻 (`--min-score` 是選用項); 它不決定、不核發任何 approval。

13.5 平台健康度與錯誤預算 (`dv_harness/platform_health.py`)

這個模組是 PC-2 平台可觀測性的彙整器, 而且只讀取: 它彙整八個子系統 (`agent_adapter`、`eda_license`、`lsf_queue`、`execution_environment`、`connectivity_gates`、`regression_quality`、`harness_self_reporting`、`slo_compliance`) 的健康狀態, 每一個數字都來自本專案既有的真實機制 (`degradation.describe()`)、`.dv-harness/events.jsonl` 中真實的 `EXECUTION_PREFLIGHT_PASS` / `_BLOCKED` 事件、`connectivity_check` 的閘門狀態與新鮮度、`trend_analysis` 的回歸趨勢與每日 rollout) ——沒有一個指標是由這個檔案自己測量出來的。

`UNKNOWN` 的排名高於 `HEALTHY`: 一個未被測量的子系統會讓整個平台無法被報為健康——這與 `connectivity.py` 對 `NOT_AVAILABLE` 「絕不能與 `FAILED` 混淆」的規則同源, 只是套用到另一側: 缺席的測量絕不能和一個良好的測量混為一談。兩個 SLO (`regression_verdict_pass_rate`、`execution_preflight_pass_rate`) 都只計算真實記錄下來的事件; `UNMEASURABLE_SLIS` 明確列出這個 harness 沒有遙測基礎的 SLI (uptime、request latency、simulator-farm uptime 等), 並且有一個測試斷言若有人不小心把這些加入 `SLO_CATALOG` 就會失敗——這是「不要為沒有遙測能力的東西捏造 SLO」這條規則的結構化落實, 而非僅止於口頭承諾。

```
dv-harness platform-health --json --window-days 14
```

誠實邊界: `assert_authORIZES_nothing()` 直接對這個模組的原始碼做斷言——它不匯入任何 approval 機制, 不寫任何檔案, evidence 資料庫也是唯讀開啟。一個 `BREACHED` 的錯誤預算永遠只是一份「報告」——它不會阻擋任何 stage, 也不會讓任何 stage 通過。本專案自身目前誠實回報 `OVERALL: UNKNOWN` (因為這裡沒有 RTL tree)。

13.6 Harness 部署到共用 Agent 路徑 (`dv_harness/harness_deploy.py`)

`USAGE_MULTI_USER_SAFETY.md` 早已明訂「每次 harness 更新都必須同步到 `/home/svcacct/AI/Agent`」的政策, 但直到此模組出現, 這只是一句沒有機制支撐的政策——`cli.py` 沒有 `deploy / harness-sync` 動詞, 過去每一次同步都是 agent 手動、逐檔複製完成的。

「什麼算是 harness」是資料，而非程式碼——由 `dv_harness/harness_deploy.manifest.json` 宣告 `include / exclude / never_sync` 三個清單。這個模組重用了三個已存在的真實機制而非重新發明：diff 引擎用的是 `tools/remote/source_identity.py` 的 `three_way_diff()` / `aggregate_source_id()`（與 `server_sync_identity_gate.py` 驗證 PC 與伺服器身分一致性所用的同一套原語，這裡改用來計算推送差異）；傳輸機制沿用 `remote_hop.py` 既有的 `tar → --put → tar xzf` 模式，只是把整個目錄改成只傳送差異集合；稽核紀錄則是既有的 `StateStore.event()`，寫入同一份 `dv-harness audit` 已經在讀的 `.dv-harness/events.jsonl`。

三項安全屬性，每一項都在程式碼中強制執行且有對應測試：

1. **絕不盲目覆寫**——推送集合只包含 `local_only | different`，`remote_only` 的檔案一律浮現為「需要人類決策」（exit code 3）且絕不刪除，因為伺服器端的差異可能是有人在期限壓力下做的合法 hotfix。
2. **絕不外洩憑證**——`never_sync` 清單的命中會拋出例外（`SecretPathRefusedError`），而不是靜靜跳過；這不是假設性的風險：`replay.ps1` 這支真實、已加入 `.gitignore`、未進版控的本機憑證腳本就位於這個 checkout 的根目錄，而目的地路徑是多人共用的。
3. **絕不意外連上網路**——`plan()` 依設計零網路呼叫：遠端一側只能是本機目錄（`--target-root`，讓整個工具可單元測試的合成模式）或已擷取的 transcript 檔案；實際套用到遠端時，指令序列只會被印出來，除非明確加上 `--execute`，而且只會提及 `remote_exec.py`——絕不是 `remote_relay.py`，遵守本專案標準規則。

CRLF 與 LF 的差異在這裡是實質而非表面問題：此 checkout 的 `core.autocrlf` 為 `true`，Windows 工作副本是 CRLF，而透過 `git clone` 產生的 `/home/svcacct/AI/Agent` 是 LF——若不處理，一次天真的 md5 diff 會把每個文字檔都判定為「已變動」，永遠重推整個 harness。`classify_line_ending_only_differences()` 會把遠端 md5 拿去和本機位元組的兩種換行版本分別比對，讓純換行差異的檔案不會被誤判為內容變動；真正跨越換行邊界的內容變更則仍會落在 `push` 集合中。

```
dv-harness harness-deploy manifest --print-md5sum-command
dv-harness harness-deploy plan --target-root <dir>
dv-harness harness-deploy apply --execute
```

誠實邊界：這關閉的是「機制」，而不是「已執行的部署」——本工具至今尚未對真正在運作中的 `/home/svcacct/AI/Agent` 執行過同步，真的要這麼做屬於 REMOTE_EXECUTION，需要重新走一次 SSH/Remote Transport Connection Intake 的確認流程；也沒有任何 post-commit hook 或 CI 步驟會自動呼叫它——「每次更新都要同步」目前仍是一個人類主動執行的指令，而非引擎自動觸發的動作。它是一個 REACHED（有真實 CLI 呼叫入口）而非 WIRED 的能力。

14. 共用知識中心與第三方元件治理 (SyoSil / ATB)

DV Agent Harness L5 並不是一個孤立運作的專案。它一方面透過 Knowledge Center 把驗證結論寫進一個跨使用者、跨專案共享的伺服器端知識庫,另一方面又必須引入並管理真實的第三方元件——最典型的例子是 SyoSil 提供的 `uvm_syoscb` scoreboard 函式庫,以及作為「golden」AMBA 參考環境的 ATB(AutoTestBench,原名 L3)。這一章說明這兩件事如何被系統性地治理:一個共用知識庫如何被安全地讀寫,一個第三方原始碼樹如何在「唯讀稽核」與「人工核准後 vendoring」之間被嚴格切分,以及 ATB 這種真實整合環境如何被結構化盤點、決定哪些能力可以重用、哪些必須回報 BLOCKED。

14.1 Knowledge Center:同一組 add/search 動詞,三種固定欄位表

`dv_harness/knowledge_center.py` 本身不是一個新的儲存機制。它的角色是替既有的 Knowledge Center broker(`tools/knowledge_center/broker.py`,執行於伺服器端)定義三組固定的欄位表(field vocabulary)與分類(category),讓「這個專案對子系統 X 知道什麼」「哪些第三方元件已被登記」「這個系統組合(system composition)長什麼樣子」這三個問題,在每一次呼叫時都用同一種形狀去問,而不是每個呼叫端各自發明自己的 record 格式。

三組固定欄位表:

- **subsystem_environment** (SYS-3,KNOWLEDGE CENTER CHECK): `SUBSYSTEM_RECORD_FIELDS` 共 21 個欄位,涵蓋 `SUBSYSTEM_ID`、`PROTOCOL`、`VERSION`、`GIT_SHA`、`BUILD_STATUS`、`REGRESSION_STATUS`、`KNOWN_LIMITATIONS`、`EVIDENCE`、`CONFIDENCE` 等。
- **third_party_component** (SYOSCB-3,KNOWLEDGE CENTER REGISTRATION): `THIRD_PARTY_COMPONENT_FIELDS` 共 11 個欄位——`COMPONENT`、`VERSION`、`SOURCE_REFERENCE`、`ROLE`、`INTEGRATION_POLICY`、`L5_DESTINATION`、`BUILD_STATUS`、`KNOWN_LIMITATIONS`、`PROVENANCE`、`UPSTREAM_DEPENDENCIES`、`EVIDENCE`。這是本章的核心,SyoSil / ATB 的登記都走這條路。
- **system_composition** (SYS-34,KNOWLEDGE CENTER UPDATE): `SYSTEM_COMPOSITION_FIELDS`,並以 `PHASE` 欄位嚴格區分「Phase-1 規劃」與「Phase-2 已整合/已通過 regression」——`record_system_composition()` 會直接拒絕把 `PHASE_1_PLAN_AWAITING_USER_APPROVAL` 寫成可公開發佈的記錄,只有 `PHASE_2_INTEGRATION_COMPLETE` / `PHASE_2_SYSTEM_REGRESSION_PASSED` 才可被發佈。

三組分類共用同一個 `add / search` 動詞、同一個 broker、同一套 provenance(來源)與 staleness(過期)生命週期。這是刻意的設計:CLAUDE.md 反覆強調「Never create a parallel Knowledge Center」——不論是子系統環境、第三方元件、還是系統組合,都不應該各自長出一個獨立的知識儲存機制。

傳輸層: `KnowledgeCenterClient` 完全不會自己開啟到伺服器的連線,也絕不直接呼叫 `remote_relay.py` (那是唯一被授權執行真實登入、帶有密碼的模組)。它一律透過已建立的 persistent relay(`tools/remote/remote_exec.py` 的 `read_relay_info()` / `send_request()`),把 payload 寫成暫存 JSON、put 上去、再對 broker 下一行指令,最後從 stdout 尋找 `DVHMC_RESULT`: 這個標記行來解析回傳的 JSON。這個設計是 2026-09-01 之後的修正:早期版本會直接 spawn 自己的 `remote_hop.py` 子行程並讀取環境變數中的密碼,這正是 CLAUDE.md「Remote Linux Execution」一節明文禁止的風險模式。

容錯與 provenance: `KnowledgeCenterClient` 上每一個方法都是 best-effort、永不因為傳輸/設定問題而拋出例外——沒有設定好共用知識中心的使用者,仍必須能正常使用本地 harness。呼叫端要自行檢查回傳字典的「ok」欄位。每次寫入都會附上 `_provenance():origin_user`、`origin_host`、`origin_project`、`client_written_at`,讓一筆共享記錄永遠可以追溯是誰、從哪個專案寫入的。

實務上,查詢一個子系統的記錄:

```
KnowledgeCenterClient(cfg, project_root).subsystem_record("USB3_DEVICE", protocol="USB")
```

回傳 `{"ok", "found", "record", "candidates"}` ——`found=False` 不是錯誤,而是「知識中心可連線、但沒有這個子系統的記錄」,這與「知識中心連不上」是兩種不同的事實,SYS-2 對待這兩者的方式完全不同。比對邏輯是對 `SUBSYSTEM_ID` 做精確、大小寫不敏感的相等比對,而非自由文字搜尋的字字串命中——否則查 `"USB"` 會意外命中 `"USB3_DEVICE"`。

14.2 SYOSCB-1/3:對真實 SyoSil 原始碼樹的唯讀稽核

`dv_harness/syoscb_source_audit.py` 回答 SYOSCB-1(REQUIRED SOURCE DIRECTORY AUDIT)要求的十六項清單——package 檔案、scoreboard/configuration/queue 類別、producer API、subscriber/TLM 結構、compare 演算法、macro、tests、examples、scripts、docs、compile order、UVM 相依性、license/copyright/provenance、版本 metadata。SYOSCB-1 自己的原文寫得很白:「Do not rely only on this prompt's description」——所以這個模組是用讀取真實檔案來回答這十六個問題,而不是相信一段對話文字。

實際使用方式:

```
python -m dv_harness.syoscb_source_audit /path/to/uvm_syoscb-1.0.2.4 \
--json --registration-payload --l5-destination reference/uvm_syoscb-1.0.2.4
```

`audit_syoscb_source(root)` 會回傳一個 `SyoscbSourceAudit`,其 `to_dict()` 內含 `checklist` (由 `audit_checklist()` 保證十六項清單「一項都不能少答」,少答會直接 raise,而不是靜默漏掉一項)。

重用而非重造是這個模組的第二條主線:

- 類別掃描只用 `vip_symbol_index.index_source_text()` 這一個工具——與 `amba_scoreboard_env` 使用的是同一套邏輯——只保留「宣告與位置」(class 名稱、base class、file:line),永不保留 **method body**,並由 `assert_no_bodies_retained()` 驗證這件事,這正是「可以安全讀一個我們不能結構性吸收的第三方樹」的關鍵前提。
- 類別角色(role)的信心分級沿用 `connectivity.BindTier` 這一套詞彙,不另發明第二套:從真實 extends 鏈得出的角色是 `T2_STRUCTURAL_MATCH` (結構證據),只靠類別「名稱」token 猜出來的角色是 `T3_NAMING_HEURISTIC` (命名啟發式,永遠不能自動被接受),兩者都不成立則是 `T4_UNDECIDABLE` (`ROLE_UNCLASSIFIED`),而非用猜測補上。
- 完全無法決定的欄位(例如「這個元件在 L5 中要放到哪裡」)使用 `connectivity.REQUIRED_HUMAN_INPUT` 這個哨兵值,表示「這是 Phase-2 才由人決定的事,再多讀上游原始碼也生不出答案」。

- Markdown 表格一律走 `connectivity.render_markdown_table()` 。
- 讀取正確性的證明重用 `amba_scoreboard_env.InspectedFile` ——「我們讀了哪些檔案、讀完之後有沒有變動過」在整個 repo 中只有一種形狀,而不是兩種。

這個模組執行的硬邊界:上游樹現在可以被「唯讀地檢視」,但把它 **vendor 進這個 repository** 是 SYOSCB-2/SYOSCB-34 的事,而且必須先通過 SYOSCB-33 的人工審查關卡。兩個斷言把這件事變成可檢查而非口頭承諾: `assert_source_unmodified()` (確保我們的稽核動作沒有寫入上游任何一個位元組)與 `assert_not_vendored(repo_root, audit, approval=...)` (確保沒有任何上游檔案——依目錄名稱或依內容 `digest` 判斷——已經出現在這個 repository 裡)。模組中沒有任何一處會產生可被編譯的 SystemVerilog, `assert_no_emittable_sv()` 對它每一份輸出做這個驗證。

SYOSCB-3(KNOWLEDGE CENTER REGISTRATION)則要求「把這個重用元件登記進既有的 Knowledge Center」。這裡的做法一樣是「只當 payload builder」: `build_component_registration_payload(audit, l5_destination=...)` 用稽核出來的真實事實去填 `knowledge_center.THIRD_PARTY_COMPONENT_FIELDS`, 回傳一個 dict;真正「發佈」出去要呼叫 `KnowledgeCenterClient.record_component()`,走的是同一個 `add` 動詞、同一個 broker、同一套 shard 生命週期——這個模組本身不會開啟任何 socket。

14.3 SYOSCB-2/33:Phase-2 vendoring,從「口頭核准」變成「可執行檢查的核准紀錄」

2026-09-06,專案負責人明確核准了 SyoSil 上游函式庫的 Phase-2 vendoring 決策:把 `D:/DV/Scoreboard/uvvm_syoscb-1.0.2.4` 的真實原始碼(234 個檔案: `src/`、`tb/`、`docs/`、`LICENSE.txt`、`NOTICE.txt`、`VERSION.txt`、`RELEASE_NOTES.txt` 及三個 vendor Makefile)逐位元組複製進 `reference/uvvm_syoscb-1.0.2.4/` 與 `reference/USB_UVM_Handoff` 是同一種「唯讀參考資料」慣例。

關鍵是:這個核准**不是**寫在對話紀錄裡就算數,而是一份結構化的 JSON 核准紀錄 `dv_harness/syoscb_vendoring_approval.json`,內含 `approved: true`、`l5_destination`、核准者、時間、以及真實的上游來源路徑。 `load_vendoring_approval()` 讀取它——若檔案不存在則回傳 `None`,若檔案存在但格式錯誤則必須 **raise**(絕不能把「格式錯誤」讀成「沒有核准」,那會讓檢查在解析失敗時反而變得更寬鬆)。

`assert_not_vendored()` 因此新增了一個**結構性的、而非繞過性的**例外機制:只有當某個檔案命中/內容命中同時滿足「落在核准紀錄的 `l5_destination` 之下」與「該紀錄的 `approved` 為真」,才會被視為已核准而豁免;其餘位置命中、或 `approved` 為 `false`(草稿/已撤銷的紀錄)的命中,一律照舊 `raise`。所有被豁免的命中也會被列入回傳結果的 `approved_vendored` 中,絕不會被靜默吸收進一個看起來乾淨的 CLEAN 結果——讀者永遠能看到「到底核准了什麼」,不只是「檢查有沒有過」。CLI 的 `--assert-not-vendored <root>` 現在也會自動載入該 `root` 的真實核准紀錄再檢查,所以這個檢查在 Phase-2 真正發生之後仍然有意義,而不會從此永遠回報「已被 vendor」而失效。

實測上,重新對 in-repo 的複本執行同一份稽核工具,得到的版本(1.0.2.4)、授權(Apache-2.0)、著作權(SyoSil ApS)與對上游原始樹稽核出來的結果完全一致——這本身就是一種完整性驗證:一份被編輯過或截斷過的複本會稽核出不同結果。

已揭露的邊界:這一節只關閉了 SYOSCB-2 的 vendoring 決策與 SYOSCB-33 的核准強制執行機制。SYOSCB-4(是否要 fork 這個上游函式庫)仍然是 `NOT_AVAILABLE` ——沒有 fork,這裡也沒有建立任何 fork。vendor 進來的複本受「No Golden-Reference Content Mining」規則約束:只能用來檢查結構/組織上的一致性,或未來作為真正整合時的相依元件,**絕不能被挖出協定行為內容貼進生成輸出**。Knowledge Center 的發佈(`record_component()`)本次會期並未執行——那是明確的 `REMOTE_EXECUTION` 動作,留待日後某個已建立連線的 session 執行。

14.4 ATB(AutoTestBench):同樣模式的第二個 vendoring 決策

同一個 session 中,另一棵原本未命名的參考樹(`D:/DV/Task/DV_Agent_Harness_L5/L3`——含 `coretop/` 與 `soc/`,其中 `soc/uvvc/scb/` 底下有一份基於 SyoSil 的 scoreboard)依使用者指示重新命名為 **ATB(AutoTestBench)**,隨後專案負責人明確核准將其 vendor 進本 repository,做法與 SyoSil 的核准模式完全相同:145 個檔案(1.6MB)逐位元組複製進 `reference/ATB/`,經 `diff -rq` 確認 145/145 檔案內容一致、無任何編輯;核准紀錄寫在 `dv_harness/atb_vendoring_approval.json` 與 `syoscb_vendoring_approval.json` 同一種結構形狀,並額外記錄了 `L3 → ATB` 的更名歷程。

ATB 存在的目的必須明白寫出來,以免被誤讀為可以從中挖料:ATB 是一個已經整合完成、真實運作的 golden AMBA M×N SoC 匯流排參考架構——存在的目的是拿來驗證這個 harness 自身的 AMBA 生成與分析能力(結構/組織一致性檢查,以及日後真正整合其 scoreboard 時作為相依元件),適用與 `reference/USB_UVM_Handoff` 完全相同的「No Golden-Reference Content Mining」紀律:絕不能是被挖去複製協定行為內容、貼進另一個專案生成輸出的來源。

與 SyoSil 純上游複本的關係,必須明確區分,以免混淆: `reference/uvvm_syoscb-1.0.2.4/` 是乾淨的上游 SyoSil 函式庫發行版本本身; `reference/ATB/soc/uvvc/scb/` 是不同的東西——一個已整合的 golden testbench 環境,恰好使用了一份基於 SyoSil 的 scoreboard。兩者的內容**預期不會逐位元組相同**(ATB 內的複本可能是較舊的版本,或周圍帶有專案特定設定),任何日後讀取 ATB 中 SyoSil 相關內容的模組都應該明確說明自己是從哪一棵樹讀的,絕不可混為一談。

一個特別記錄下來、與本 harness 一般性效能免責聲明相反的例外:本 harness 一般而言不擁有任何即時模擬器或形式化工具,所有效能領域的模組(`amba_performance_calculator.py` 等)都必須假設數字是呼叫端提供、絕不是 harness 自己量測出來的。ATB 是例外——一旦未來某個 session 真的建置並執行 ATB(一次真實的 VCS/UVVM 呼叫,依循自己的 Execution Mode 宣告,如涉及遠端還需自己的 SSH/Remote Transport Connection Intake 關卡),由此產生的 `fsdb_report.py` 衍生時序/延遲/頻寬數字**就是**該環境的真實量測 ground truth,不需要像本 session 一般性效能模組那樣預設當作「呼叫端聲稱、需要懷疑」的假設。這不放寬 Evidence Truth Rule——數字仍必須來自 ATB 真正產生的 fsdbreport/模擬產出物,絕不能替 ATB 估算或杜撰。本 session 並未執行或建置 ATB;這只是為未來實際執行 ATB 的 session 移除一個過度一般化的假設。

14.5 ATB Reference Inventory:能力盤點與「重用—否則封鎖」規則

如果說 `syoscb_source_audit.py` 回答的是「這一個第三方函式庫裡有什麼」, `dv_harness/atb_reference_inventory.py` 回答的是高一層的問題:「整個參考環境裡有哪些能力,哪些真的被接上使用,哪些存在但沒被用到,哪些與這個專案別處已核准 vendor 的複本產生了漂移(drift),而且——對呼叫端真正需要的某個能力——究竟可以從這裡重用、從別處已核准的 vendor 複本重用,還是必須回報 BLOCKED,因為兩者皆無。」根目錄一律是呼叫端傳入的參數(`discover_atb_capabilities(root, ...)`),從不寫死。

結構掃描全部重用既有工具:類別掃描同樣只用 `vip_symbol_index.index_source_text()` ——只留宣告與 `file:line, assert_no_bodies_retained()` 驗證。模組層級(RTL 形狀)的宣告——`bind connector module`、`testbench top module`——用 `verible_parser.py`,做法與 `env_manifest.py` / `phy_boundary.py` 相同:只看 port 與 instance,絕不看陳述式主體;若機器上沒有真的 `verible-verilog-syntax` 可執行檔,這一層會誠實降級為 `NOT_AVAILABLE` (附真實原因),絕不是靜默回報

零筆而讓人誤以為「不存在」——類別/介面的發現不受影響。唯一沒有任何既有工具索引的構造是 `interface` 宣告(ATB 的 bind interface 都放在 `uvc/bind/` 下): `_index_interfaces()` 是一個小型、局部、只看宣告行的 regex 掃描,遵循與上述兩者相同的紀律(名稱 + file:line,絕不含 signal list 或 body)。

狀態詞彙(逐字引用該模組自身的治理任務): `PROVEN` / `IMPLEMENTED_UNPROVEN` / `PARTIAL` / `PRESENT_UNUSED` / `DUPLICATE` / `STALE` / `MISSING` / `BLOCKED` / `UNKNOWN`。 `PROVEN` 需要一筆呼叫端明確聲明、有引用出處的真實 regression/simulation 證據來指名這個確切的能力——這個模組自己不會生產任何一筆這樣的證據,因為 ATB 是一棵沒有接上任何自身證據庫的唯讀參考樹。 `DUPLICATE` 只在同一個子系統(`coretop` 對 `soc`)內、同名構造真正撞名時才成立——兩個參考環境本身依設計就有的、跨子系統的重複 bind interface 副本不會被標記。 `IMPLEMENTED_UNPROVEN` 要求同一個子系統中,該能力自身 kind 目錄以外的檔案有真正的引用(依 symbol 名稱,或依其宣告檔名被 ``include`)——因為 ATB 真實的 bind interface 是靠檔名被接上,不是靠 interface 的裸名稱。 `UNKNOWN` 保留給工具間真正互相矛盾的情形(verible 判定是 `module` 的名稱,同時又被類別/介面掃描判定為類別或介面)——絕不會用「挑一個」來解決。

「重用—否則封鎖」規則,寫成程式碼: `resolve_capability_reuse(name, manifest, approval_records, project_root)` 依序:(1) 若 ATB 本身就有這個能力,優先重用 ATB 自己的複本—— `REUSE_LOCAL_ATB_CAPABILITY`,附 ATB 自身的 file:line;(2) 否則,若專案裡已存在一份真實的、已核准的 `dv_harness/*vendoring_approval*.json` 形狀紀錄,其 `is_destination` 目錄樹下真的含有這個能力,就重用那一份—— `REUSE_VENDORED_REFERENCE_COPY`,附核准紀錄與 vendor 複本中真實的 file:line;(3) 否則 `BLOCKED_NOTHING_TO_REUSE`,列出這個函式真正檢查過、確認沒有的每一個位置。沒有第四條分支會在什麼都沒有的情況下繼續往下走。 `find_project_vendoring_approval_records()` 透過泛用 glob 搜尋——絕不是寫死單一檔名——讀取磁碟上每一份這種紀錄,遇到格式錯誤的紀錄一律 fail closed(raise,絕不悄悄跳過)。

`evaluate_drift_against_vendored()` 用一個 `component_hint` 把比對範圍限定在「同一個家族(family)」上,避免一份「目的地剛好是整個 ATB 鏡像」的核准紀錄輕易「比對到自己」而掩蓋了對真正上游的漂移發現——在真實資料上, `ATB/soc/uvc/scb/cl_syoscb_queue_std.svh` 確實與本專案已核准的 `reference/uvw_syoscb-1.0.2.4/src/cl_syoscb_queue_std.svh` (著作權年份、base class 均不同)不同,是一筆不需要 fixture 就存在的真實 `STALE` 發現;而 `cl_syoscb_report_catcher.svh` 只存在於已 vendor 的參考複本、不存在於 ATB 本身,得到一筆真實的 `REUSE_VENDORED_REFERENCE_COPY` 解析結果——但這個模組從不宣稱哪一邊比較新,那是人的決定。

`atb_vendoring_approval_status()` 誠實地(依磁碟上真實檔案)回答 ATB 自己是否有一份 vendoring 核准紀錄。值得記錄的一件事:此模組在建置過程中發現, `dv_harness/atb_vendoring_approval.json` 與 `reference/ATB` 的完整鏡像已經由同一個 multi-agent session 中另一個並行 agent 建立——由於 `find_project_vendoring_approval_records()` 是用泛用 glob 而非寫死檔名去發現核准紀錄,它不需要任何程式碼改動就自動檢到了這份真實紀錄,而不是斷言「尚無核准」這個現在已經過時的假設。這是 Evidence Truth Rule 施加在模組自身建置過程上的一個例子:以目前證據為準,而不是相信治理任務裡的舊描述。

已揭露的邊界:這個模組不決定任何事,只回報——不複製檔案、不動 build/job/approval,也沒有 stage gate。 `PROVEN` 需要呼叫端提供的真實引用,這個模組無法自行生產。模組層級(RTL 形狀)的發現是 best-effort,需要真實 verible-verilog-syntax 在 PATH 上;缺少它只會縮小可分類的範圍,絕不會靜默地讀成「不存在」。重複偵測、家族漂移、以及重用—否則封鎖規則全部只運作在**宣告層級的結構事實**上——沒有一項證明行為正確性,也沒有一項決定兩份分歧複本中哪一份才是權威版本。這個模組**沒有** `dv-harness` CLI 動詞,也未接入 `gates.py` / `cli.py`,唯一的入口是它自身的 Python API: `discover_atb_capabilities()` / `resolve_capability_reuse()` / `atb_vendoring_approval_status()`。

15. Human Control Plane、Dashboard 與 GUI

15.1 為什麼需要 Human Control Plane

CLAUDE.md 的 Core Operating Rules 明訂「Human Override is always valid」，但在 `dv_harness/control_plane.py` 出現以前，這句話在很長一段時間內只是文件裡的一句宣示。該模組自己的檔頭註解誠實記錄了這段歷史：在此模組之前，這個 harness 對外宣稱的「12-verb Human Control Plane」——STATUS、WHY、REVIEW、REPLAN、EVIDENCE、APPROVE、REDIRECT、PAUSE、RESUME、TAKEOVER、CORRECT、CONSTRAINT——大部分是空話。STATUS 是真的，WHY/REVIEW 只有較弱的形式存在，REPLAN 雖然存在但從未被呼叫；而 EVIDENCE、APPROVE、REDIRECT、PAUSE、RESUME、TAKEOVER、CORRECT、CONSTRAINT 完全沒有可執行的形式——沒有 CLI subcommand、沒有 `engine.py` 裡的檢查，人類根本沒有任何動作能被執行中的 pipeline 真正注意到。

`control_plane.py` 就是用來補上這個缺口的單一持久化狀態檔：`.dv-harness/control.json`。`dv_harness/cli.py` 的新 subcommands 會寫入這個檔案；`dv_harness/engine.py` 的 `loop()` / `run_stage()` 會在每一次迭代 / 每一次呼叫時讀取並「遵守」這個檔案；`dv_harness/prompts.py` 則會把人類可見的欄位（constraints、correction notes、approvals）折疊進下一個 stage 的 prompt，讓 agent 真正「看得到」人類寫下的東西。這裡沒有一套平行於引擎、agent 自行忽略的「另一套帳本」——這正是這個模組存在的意義。

Human Override 的優先順序在程式碼裡是明確分層的：TAKEOVER 是最強的 override，`engine.py` 會在 `loop()` 的每一次迭代開始（在呼叫 `run_stage()` 之前）以及 `run_stage()` 本身內部都檢查它一次，確保即使有人繞過迴圈直接呼叫 `dv-harness run-stage`，也不能悄悄對一個正被人類 TAKEOVER 的 stage 跑一次 LLM turn。PAUSE 則只在 `loop()` 裡被檢查（不在 `run_stage()` 裡）——因為單次手動 `run-stage` 呼叫是一個明確的人類動作，而不是 PAUSE 本來要阻止的「自動迴圈繼續跑下去」。

15.2 .dv-harness/control.json 的內容與寫入紀律

ControlPlane 類別（`dv_harness/control_plane.py`）提供對 `.dv-harness/control.json` 的讀寫存取。每一個會修改狀態的方法都遵循「load → mutate → atomic save」的模式：沒有長期存活的記憶體內狀態會過期，因為 `engine.py` 在每一次 `loop()` 迭代開始、每一次 `run_stage()` 呼叫開始時都會重新讀取這個檔案。寫入透過 `storage._atomic_replace()` 完成，這是為了處理 Windows 上「並行讀取者 vs. `os.replace()`」的競爭問題——因為 `dashboard.py` 的 `POST /api/control` 處理器是跑在 `ThreadingHTTPServer` 的每請求執行緒上，而每個指令都各自做一次「load → save」的往返，這個競爭是真實可觸發的。

DEFAULT_CONTROL 定義了狀態的形狀，主要欄位包括：

- `paused / paused_reason / paused_at`
- `takeover : {active, stage, message, taken_at, taken_by}`
- `constraints : [{id, text, added_at, added_by}, ...]`
- `corrections : stage -> {note, at, reset_attempts, consumed, corrected_by}`
- `approvals : stage -> {note, reviewer_id, reviewer_confidence, approved_at}`
- `approval_history : stage -> [已封存的 approval 記錄, 由舊到新, 每筆都標註 outcome]`
- `cosigns : stage -> {"<gate_id>/<loc>": {value, reviewer_id, reviewer_confidence, cosigned_at, cosigned_by}}`

值得特別注意的是 APPROVE 的封存機制。`_archive_approval()` 的檔頭註解記錄了一次真實發生過的多人協作審查（multi-persona interaction review, 2026-08-28）發現的 regression：DE Manager 與 DV Manager 各自獨立發現 `approve()` 用 `setdefault(...)[stage]=` 會悄悄覆蓋掉前一位 reviewer 的紀錄，而 `clear_approval()`（當時剛加入，目的是讓 APPROVE 變成單次有效）則會把該紀錄直接刪除——「唯一一筆人類簽署的 SIGNOFF 紀錄就這樣消失，沒有留下任何痕跡」。修復之後，任何一次 approval 被取代或被消費之前，都會先被封存進 `approval_history`，並標註 `outcome`（例如 `OVERWRITTEN_BY_NEW_APPROVAL` 或 `CONSUMED_BY_PASS`，或是 2026-09-05 之後新增的 `WITHDRAWN_BY_HUMAN_HOLD`，用於 `commands.cmd_research_hold()` 主動撤回 `RESEARCH_CAPABILITY_EVOLUTION` 核准的情境）——這兩種「approval 停止生效」的原因必須在稽核紀錄裡保持可區分，因為它們代表完全不同的事實。

15.3 CLI 上的十二個（以上）動詞

`dv-harness` 的這些 subcommand 直接對應到 Human Control Plane 的動詞，全部經由 `ControlPlane` 寫入 `control.json`：

```
dv-harness pause --reason "..."  
dv-harness resume  
dv-harness takeover --message "..."  
dv-harness release-takeover  
dv-harness redirect <STAGE> --reason "..."  
dv-harness approve <STAGE> --note "..." --reviewer-id <id> --reviewer-confidence HIGH|MEDIUM|LOW  
dv-harness evidence [--stage <STAGE>]  
dv-harness checklist [--stage <STAGE>]  
dv-harness correct <STAGE> --note "..." [--reset-attempts]  
dv-harness constraint --add "..." | --list | --remove <ID>  
dv-harness cosign <STAGE> <gate_id>/<loc> --value '<json>' --reviewer-id <id> --reviewer-confidence HIGH
```

其中 `approve` 的 `stage` 選項不是單純用 `[s.value for s in stage]`，而是使用 `commands.approval_stage_choices()`——也就是「圖上的 stage」加上 `commands.APPROVAL_ONLY_STAGES`（真正存在於程式碼中、但不是圖節點的核准點）。這是刻意的修正：如果只用圖上的 stage 列舉，`capability_evolution.py` 的 Human Approval Gate（`RESEARCH_CAPABILITY_EVOLUTION`）就永遠無法被人類實際核准，因為它根本不是一個 graph stage。parser 與 `commands.py` 裡的驗證邏輯共用同一個來源函式，兩者不可能互相矛盾。

`cosign` 是這套 Human Control Plane 的一個延伸機制（GUI/CLI gap-closure, 2026-08-28）。在此之前，滿足 `gates.py` Tier-5 `JUDGMENT_FIELDS` 包裝要求的唯一方法，是讓 agent 自己在下一次 evidence block 裡直接寫出 `{ "value", "reviewer_id", "reviewer_confidence" }` 包裝——完全沒有人類可以直接「共同簽署」某個待審欄位的手段。`add_cosign()` 會用 `(stage, "<gate_id>/<loc>")` 當 key 持久化一筆紀錄，`gates.py` 的 `_check_judgment_fields` 會在下次

`evaluate_stage_evidence()` 呼叫時真正查詢它：一個未包裝的欄位，只要其裸值「精確等於」已儲存的 `cosign` 值，就可以不需要 `inline wrapper` 而通過。比對故意採用「精確值」而非「僅比對位置」——同一位置若之後值改變了，舊的 `cosign` 不再涵蓋它，必須重新簽署或提供新的 `inline wrapper`。這不是一個顯示 / 記錄用的機制，而是真正會影響 `gate` 結果的機制。

`dv-harness explain [--stage <STAGE>]` 是 WHY 動詞的實作，讀取 `control_plane.describe_stage()` ——這是整個系統裡「唯一共用的讀取路徑」，dashboard 上的「Why (current stage)」卡片、CLI 的 `explain / evidence / checklist` 三個指令都經過它，確保這三個介面看到的絕不會互相矛盾。

`dv-harness audit --limit N` 則是稽核追蹤：印出最近 N 筆 `events.jsonl` 事件，加上目前 `control.json` 裡的 `corrections / approvals / approval_history / cosigns`。

15.4 Constraint 與 Correction：不需要重跑 agent 就能修正它

CONSTRAINT 動詞 (`dv-harness constraint --add/--list/--remove`) 用來管理一組會被折疊進「每一次後續 stage prompt」的持久約束。每個 constraint 有自己的 `id` (`CN{seq}`)、`text`、`added_at`、`added_by`，透過 `next_constraint_seq` 保證單調遞增。

CORRECT 動詞 (`dv-harness correct <STAGE> --note "..."` [`--reset-attempts`]) 讓人類在不重新跑一次 agent 的情況下，直接對某個 stage 提供修正意見。`set_correction()` 寫入的紀錄帶有 `consumed: False`；`get_active_correction()` 只回傳尚未被消費的紀錄；`consume_correction()` 則會在下一次相關 stage 執行時把它標記為已消費——這是一次性的訊息傳遞機制，而不是一個永久狀態旗標。

15.5 Dashboard：唯讀觀測 + 受控寫入

`dv_harness/dashboard.py` (約 3600 行) 是這個 harness 唯一的 GUI 表面，實作為一個標準函式庫等級的 `http.server.ThreadingHTTPServer`，沒有任何外部 web framework 依賴。啟動方式是：

```
python -m dv_harness.dashboard --project-root <dir>
```

(或使用 `repo` 根目錄提供的 `START_DV_HARNESS_DASHBOARD.ps1`。)啟動時會印出一段 banner，內容由 `dashboard_auth.startup_banner()` 產生，裡面帶著存取的 URL 與 token。

Dashboard 提供的所有 GET 端點都是唯讀的：目前 stage 狀態、`control_plane.describe_stage()` 的 Why 卡片內容、LSF job 摘要、coverage 狀態、`GET /api/loops` (GUI Loop Engineering Center，見 15.7) 等等。所有真正會產生副作用的動作，一律走 POST (含 PUT / PATCH / DELETE)：`/api/setup`、`/api/start`、`/api/session/save`、`/api/session/restore`、`/api/upload`，以及最重要的 `/api/control` (背後就是 `_dispatch_control()`，轉呼叫 `dv_harness.commands` 裡對應 PAUSE / RESUME / TAKEOVER / RELEASE TAKEOVER / REDIRECT / CONSTRAINT_ADD / CONSTRAINT_REMOVE / APPROVE / COSIGN / CORRECT / RESEARCH_APPROVE / RESEARCH_REJECT 的實作)，再加上 `/api/signoff-export`、`/api/waiver`、`/api/config`。

15.6 GUI-19 Token 關門與 PC-6 三層角色

在 `dashboard_auth.py` 出現以前，`dashboard.serve()` 的 `do_GET / do_POST` 完全沒有身分驗證與權限檢查，而 POST 早已暴露了具有真實後果的動作：control-plane 的 APPROVE / COSIGN / TAKEOVER / constraint 寫入、signoff bundle 匯出、waiver 撰寫、policy 寫入、session save/restore、檔案上傳、啟動 harness。預設綁定 `127.0.0.1` (`config.py` 的 `dashboard.host`) 只限制了「網路層」的暴露面，並不是存取控制——同一台機器上的任何其他 process 或任何其他已登入的使用者，都能對它發 POST，寫入一筆真實的 APPROVAL 事件到這個專案的稽核紀錄裡，而 harness 會把它記錄成一個人類決策。

`dashboard_auth.py` (GUI-19：2026-09-05 GUI completeness audit) 採用的是 Jupyter 對「僅限 localhost 的工具」所使用的同一種模型——因為那是唯一真正適用於這個情境的模型：

- `serve()` 在啟動時產生一個全新的 `secrets.token_urlsafe(32)` token，寫入 `.dv-harness/dashboard_session.json` (在作業系統支援的地方是 owner-only 權限)，並把帶著 token 的 URL 印在終端機上。這個 token 從未被嵌入 GET / 回傳的 HTML 裡——即使有惡意的本機 process 抓取這個頁面，也學不到任何東西。要能真正操作，呼叫者必須讀過終端機輸出，或讀過 session 檔案。
- 每一個 POST 端點都是「deny-by-default」：檢查發生在 `do_POST` 進入任何路徑分派之前，所以之後新增的端點會因為「本來就被擋在外面」而自動被涵蓋，不需要有人記得手動幫它加上檢查。
- 唯讀的 GET 端點維持開放——它們不暴露任何動作，硬要加上關門只會破壞一個狀態頁面的可用性。

比對一律使用 `secrets.compare_digest`，避免透過回應時間差一個字元一個字元地反推出 token。

這個模組明確揭露了兩個尚未關閉的殘留風險：(1) 以同一個使用者身分執行的本機 process 可以直接讀取 `dashboard_session.json`，之後就與真人操作者無法區分——要關閉這個風險需要 OS 層級的 peer-credential 檢查，而一般 TCP socket 並不具備可攜的方式做到這件事；這裡真正關閉的是「完全沒有任何憑證的呼叫者」這種情境。(2) `os.chmod(0o600)` 在 POSIX 上是真的，但在 Windows 上只是切換 read-only bit，並不會透過 ACL 限制其他使用者——`issue_session_token()` 會在回傳紀錄的 `permissions` 欄位裡誠實回報它實際拿到的是哪一種保護，而不是到處宣稱都是 0600。

在此之上，`dashboard_auth.py` 於 2026-09-06 (PC-6) 加上了「角色」這一層。GUI-19 只回答了「這個呼叫者是不是這個 dashboard 的操作者」，卻無法回答「這個呼叫者能不能做這件事」——因為原本只有一個 token，而它對每一個 mutating 端點的授權都相等：能觀看這次執行的人，也能直接寫入一筆真實的 APPROVAL 到 `control.json`、撰寫 waiver、翻轉 policy gate、匯出 signoff bundle。在共用機器上，這正是「一位工程師在旁觀看 regression」與「DV lead 正式簽署放行」之間的全部差異。

三個角色，依權限由低到高排序 (VIEWER < OPERATOR < APPROVER)：

- VIEWER**——只能讀。不授權任何 mutating 動作；因為唯讀 GET 本來就沒有被關門檔住，所以 VIEWER token 唯一的用途，是明確地告訴持有者「這個動作不是你能做的」。
- OPERATOR**——可以操作 harness：setup、start、pause/resume、takeover、redirect、constraint、session save/restore、上傳檔案。

- **APPROVER**——OPERATOR 能做的事全部都能做，再加上會把一筆「人類決策」寫進這個專案稽核紀錄的動作：control 端的 APPROVE / COSIGN / CORRECT 與三個 RESEARCH_* 動詞、撰寫 waiver、寫入 policy、匯出 signoff bundle。

這是 GUI-19 的擴充而不是另立一套機制：`issue_session_token()` 會把三個角色各自的 token 一起寫進同一份 `dashboard_session.json` 紀錄，而該紀錄原本就有的 `token` 欄位就是 APPROVER 的 token——所以啟動時印出的 URL、`read_session_token()` 回傳的值、以及所有單一操作者的既有流程，行為都與加入角色制之前完全一致。新增的能力是：操作者現在可以把 VIEWER 或 OPERATOR 的 token 交給同事，而不必交出能簽署放行的那把。

`required_role()` 在兩個方向上都是 deny-by-default：一個本模組沒有對應到的 POST 路徑、或 `/api/control` 底下一個沒有對應到的子指令，兩者都需要 APPROVER 權限——所以之後新增的端點或指令，會被卡在「最高」的門檻，而不是因為被遺漏而落在「最低」的門檻。`assert_endpoints_mapped()` / `assert_control_commands_mapped()` 這兩個自我檢查函式會去讀 `dashboard.py` 真實的 dispatch 邏輯，一旦兩邊出現落差就會失敗——所以這個 fallback 機制是一張真正的安全網，而不是一個東西被悄悄丟進去、沒人管的地方。

這裡同樣誠實揭露了第三個殘留風險：角色制分開的是「持有不同 token 的人」。一個能讀到 `dashboard_session.json` 的 process，會讀到全部三個 token，因此可以直接以 APPROVER 身分行動——這是殘留風險 1 的延伸，並沒有在這裡被重新關閉。真正被關閉的是原本存在的問題：一個合法拿到 dashboard 存取權的人，原本沒有辦法只被授予「低於完整核准權限」的東西。

`dashboard_authorization_matrix.py`（VIEWER/OPERATOR/APPROVER 角色，套用在每一個 mutating `/api/control` POST 上）由 `dv_harness_tests/test_dashboard_authorization_matrix.py` 驗證。這是「單一 dashboard 專案內」的角色/授權衝突偵測，與同一天實作的 `multi_user_coordination.py`（跨「不同使用者各自獨立的 .dv-harness/」偵測衝突，例如 stale-SHA 衝突、重複的 regression 提交、共用資源的 reservation 衝突）刻意是同一份規格關切的兩種不同範疇，分別在兩個不同的地方關閉，彼此並非重複。

15.7 GUI Loop Engineering Center：從指標到 loop telemetry 的落差

LOOP_ENGINEERING 規格第 108 節列出了十九種 loop telemetry 事件類型（`LOOP_CREATED`、`LOOP_STARTED`、`LOOP_ITERATION_STARTED`、`LOOP_ACTION_SELECTED`、`LOOP_VERIFY_COMPLETED`、`LOOP_PROGRESS_UPDATED`、`LOOP_CONVERGING`、`LOOP_PLATEAU_DETECTED`、`LOOP_OSCILLATION_DETECTED`、`LOOP_NO_PROGRESS`、`LOOP_RETRY_SCHEDULED`、`LOOP_BUDGET_WARNING`、`LOOP_BUDGET_EXHAUSTED`、`LOOP_BLOCKED`、`LOOP_HUMAN_GATE_REQUIRED`、`LOOP_RESUMED`、`LOOP_SUCCESS`、`LOOP_FAILED`、`LOOP_STOPPED`），第 107 節則要求一個「GUI Loop Engineering Center」——一張 Loop | State | Iteration | Verified Gain | Budget | Plateau | Oscillation | Next Action 的表格，附帶十四個欄位的下鑽細節——因為「GUI 必須揭露一個 loop 正在跑的理由」。在此之前，repo 內對這十九個事件名稱做全域搜尋，結果是零命中，而 `dashboard.py` 唯一與「loop」相關的內容，是既有的單次執行 / 連續執行的 Start 切換開關——那是一個控制項，不是可觀測性。

`dv_harness/loop_telemetry.py` 是事件詞彙表與讀取端；`engine.DVHarness.loop()` 是產生者；`GET /api/loops` 加上 dashboard 上的 Loop Engineering Center 卡片則是呈現面。所有事件都寫進同一份既有的稽核檔案，透過同一個寫入函式：`storage.StateStore.event()`，也就是 `_record_debug_loop_round()`、`_record_loop_state_observation()`、`_spend_retry_exhaustion_budget()` 以及 `dv-harness audit` 早已在用的那一個 `.dv-harness/events.jsonl`。`emit()` 會拒絕任何不在這十九個名稱之內的事件名——因為若允許，一個不被辨識的名稱會在讀取端被靜靜丟棄，但發出端卻會誤以為自己已經回報了什麼。

每一個事件都對應到 `loop()` 真正做出的一次狀態轉換，而且都與該決策只相差一行：`LOOP_ITERATION_STARTED` 在 `while True` 迴圈本體的最上方發出；`LOOP_ACTION_SELECTED` 在呼叫 `run_stage()` 之前發出，帶著該 graph node 真正的 `route / skills`；`LOOP_VERIFY_COMPLETED` 在其後發出，帶著 `gates.effective_stage_gates()` 真正的 gate id（是第 87 節「驗證器分離」原則被實際記錄下來，而不只是被假設成立）；`LOOP_RETRY_SCHEDULED` 在唯一那個 `ss["status"] = RETRY` 分支發出；`LOOP_BUDGET_EXHAUSTED` 緊接在 LOOP-3 的 ledger 扣款旁發出。每一次 loop session 恰好只會有一個終端事件（`TERMINAL_LOOP_EVENTS`：SUCCESS / FAILED / BLOCKED / STOPPED）——兩個終端事件會是兩個互相矛盾的答案，回答「這個 loop 是怎麼結束的」。

Dashboard 上呈現的每一個欄位都刻意不重新計算一次已經存在的答案：狀態值來自 `loop_contract.derive_loop_state()`；預算數字來自 `policy.max_stage_retries` 與 LOOP-3 的 ledger payload；plateau/oscillation 的判定來自 `loop_convergence.classify_loop_convergence()`；「Next Action」欄位來自真正的 `inference.next_best_action()`（透過它的 `gap_action_catalog` 參數），這是這個 repo 第 10 節明文禁止重新實作的那個 domain-neutral 引擎。`dashboard_overall_progress()` 現在會委派給 `loop_telemetry.gate_verified_stage_count()`，所以進度條與「Verified Gain」欄位不可能互相矛盾。

一個從未跑過任何一次 loop 事件的專案，`available: false` 會誠實地被回報，並指名它讀取的那個 `events.jsonl`、以及能填入資料的那個真正指令：`dv-harness start --loop "<goal>"`。單次的 `dv-harness run-stage` 或 `--dry-run` 的 loop 完全不開啟 loop session，也不會發出任何事件，因為兩者都不是迭代。

觀測本身不授權任何事情。`LOOP_HUMAN_GATE_REQUIRED` 記錄的是「一個人類決策現在被欠著」，而不是那個決策本身，也不會變成那個決策。`loop_telemetry.py` 完全不引用 `ControlPlane`、`can_signoff`、`assert_human_approval`、`HumanApprovalRequiredError` 或 `ProductionWriteNotAuthorizedError`——這是有測試斷言過的，而不只是文件上的宣稱。`GET /api/loops` 是唯讀的；這裡刻意沒有針對 loop 的寫入端點，因為啟動 / 停止一個 loop 早就有自己的端點了（`POST /api/start`，以及 GUI-19 token 開門後面的 control-plane 動詞）。

Loop Engineering Center 目前只涵蓋「驗證收斂（Verification Closure）」這一個迴圈的 telemetry。Project Learning 迴圈與 Capability Evolution 迴圈完全不會發出任何第 108 節定義的事件——`loop_contract.observe_all()` 已經誠實回報前者是 NOT_OBSERVABLE（因為它是逐筆記錄地被觀察，而非逐專案），後者則是由人類的 `dv-harness research` 或治理層的狀態轉換所驅動，而不是由一個持續迭代的 driver 驅動，所以並不存在一個「以 `run_id` 為範疇」的 session 可供觀察。CANCELLED 與 STALE 兩個狀態沒有任何產生者（`derive_loop_state()` 從不會回傳它們，第 97 節的 stale 偵測也還沒被建置）。這裡也沒有 `dv-harness` CLI 動詞——正式的入口是 `GET /api/loops`（也就是那張卡片本身）與 `python -m dv_harness.loop_telemetry names|events|rows|show`，兩者共用同一個 `execute_verb()`。

15.8 Remote Control Mode：Claude Web/App 的傳輸層地位

CLAUDE.md 的「Remote Control Mode」一節內容簡短但態度明確：

Claude Web/App 可以透過 Remote Control 控制這個本機的 Claude Code session。Remote Control 只是傳輸層；Harness 的治理機制仍然是最終權威。絕對不可以把 SSH 私鑰或憑證暴露給 Web/App 或長期記憶。

換句話說，Remote Control 讓一個從 Claude Web/App 端發起的操作者，能透過網路把指令帶進這個本機 Claude Code session；但這條傳輸線本身不繞過上面第 15.1–15.6 節所描述的任何治理機制——它必須落地成 `control_plane.py` 真正理解的動作，才會對 `engine.py` 的行為產生任何影響。

`dv_harness/remote_control.py` 就是把這個原則落實成程式碼的模組。它的檔頭註解特別記錄了一個關鍵歷史教訓：早期版本的這個模組曾經宣稱「還沒有 PAUSE/RESUME 等 CLI 介面存在」——這是錯的。`dv_harness/cli.py` 早就有真正的、小寫的 `pause / resume / takeover / release-takeover / redirect / approve / status / evidence` 等 subcommand，背後由 `ControlPlane` 驅動、寫入 `.dv-harness/control.json`，而且 `engine.py` 的 `loop() / run_stage()` 確實會讀取那份狀態（PAUSE/TAKEOVER 真的會停止自動迴圈；REDIRECT 真的會改變 `current_stage`）。

如果只把 `remote_control.py` 自己的 `session.json / audit_log.json` 狀態機孤立來看，它其實不會影響引擎讀到的任何東西——一個遠端的呼叫者若只看到這個模組回傳 `{"ok": true, "state": "PAUSED"}`，看到的只是一個沒有真實效果的狀態宣稱，這正是 Evidence Truth Rule / 「LSF DONE 不等於 DV PASS」原則所警告的那種情況。為了關閉這個落差，`validate_and_transition()` 現在會在持久化這個模組自己的 session/audit 狀態之前，先對每一個「真正有對應效果」的指令套用一次真實的 `ControlPlane`（或針對 REDIRECT，`DVHarness.human_redirect`）副作用：

```
PAUSE      -> ControlPlane.pause(reason)
RESUME     -> ControlPlane.resume()
TAKEOVER   -> ControlPlane.takeover(stage=target_stage, message=reason, taken_by=actor)
REDIRECT   -> DVHarness(root).human_redirect(target_stage, reason)
            [可能因為另一個 stage 正處於 TAKEOVER 衝突而丟出 RuntimeError
             —這個錯誤會被視為指令失敗回傳，而不是被吞掉]
APPROVE    -> ControlPlane.approve(stage=target_stage, note=reason, reviewer_id=actor)
            [目前實務上不可達—見下方揭露的殘留缺口]
```

如果 `ControlPlane` 端的副作用失敗或丟出例外（例如 REDIRECT 因為另一個 stage 正被 TAKEOVER 而被拒絕），這個模組自己的狀態也不會被寫入——確保「remote_control 宣稱狀態是 X，但 `ControlPlane` 仍然說不是 X」這個方向的不一致永遠不會發生。

`STATUS / WHY / EVIDENCE / REVIEW / HYPOTHESIS` 這五個動詞沒有對應的 `ControlPlane` 副作用（依照狀態轉換表，它們是「state -> 相同 state」的唯讀動作）；但其中 REVIEW 與 HYPOTHESIS 確實各自帶有真實、獨立的唯讀內容（分別對應 `_current_stage_review_detail / _score_hypothesis`），不同於 PAUSE/RESUME/TAKEOVER/REDIRECT/APPROVE 那樣會真正改變狀態。至於 REJECT 與 STOP，這個模組坦白說明它們完全沒有 `ControlPlane` 對應方法——`ControlPlane` 沒有 `reject() / stop()`，引擎裡也沒有任何地方把「STOPPED」當成真正的中止條件。這兩者僅止於「session 狀態層級」：`remote_control.py` 自己的 `session.json` 會顯示 STOPPED 之類的字樣，但自動迴圈並不會真的被停止。文件明確要求：不要把 REJECT/STOP 呈現成等同於 PAUSE/TAKEOVER——它們不是。

這裡進一步誠實揭露了三個目前刻意沒有掩蓋的缺口：（1）`remote_state_transition_gate.py` 的 ALLOWED 表格，在任何狀態（RUNNING/PAUSED/TAKEOVER/STOPPED）下都完全沒有 APPROVE 或 REJECT 的條目，即使 `remote_control_supervisory_gate.py` 承認這兩者是合法指令並要求它們帶有 `target_stage`——結果是今天沒有任何一串指令序列能讓 APPROVE 或 REJECT 通過狀態轉換閘門，所以上面提到的 `ControlPlane.approve()` 串接其實從未真正透過這個模組被觸發過；它先被接好，只是為了讓「修好那張轉換表」（這屬於該閘門擁有者的決定，不屬於這個模組）成為唯一剩下的步驟。（2）`remote_action_audit_gate.py` 的 MUTATING 集合是 {REDIRECT, APPROVE, REJECT, STOP, TAKEOVER}——不包含 PAUSE 或 RESUME，即使兩者依狀態轉換表確實會改變 session 狀態；這個模組永遠會為每一個指令提供 reason/evidence_snapshot（見 `default_evidence_snapshot`），但依照真實的閘門規則，PAUSE/RESUME 的稽核紀錄即使缺少這些欄位也不會被拒絕。（3）`REMOTE_CONTROL_START.ps1` 會把 "STARTING" 寫進 `session.json`；但 "STARTING" 並不是 `remote_state_transition_gate.py` ALLOWED 表格裡的合法 key，所以沒有任何指令能真正把它接下去——這也是 `dv-harness remote-control bootstrap` 存在的原因之一：它讓一個新 session「直接落地在 RUNNING 狀態」，修正了原本手寫腳本會造成的死路。

實務上，`dv-harness remote-control {bootstrap,status,cmd}` 是這一整套機制唯一的 CLI 入口：

```
dv-harness remote-control bootstrap [--host ...] [--working-directory ...] [--session-id ...]
dv-harness remote-control status
dv-harness remote-control cmd <STATUS|WHY|EVIDENCE|REVIEW|HYPOTHESIS|PAUSE|RESUME|REDIRECT|APPROVE|REJECT|STOP|TAKEOVER>
    [--target-stage <STAGE>] [--reason "..."] [--actor <id>]
    [--env-mode SUBSYSTEM_ENV_MODE|SYSTEM_LEVEL_ENV_MODE]
```

`status` 是唯讀的：印出目前的 `session.json` 加上最近一筆稽核紀錄。`cmd` 則會依序跑過真正的 supervisory gate、state-transition gate、audit gate、replay gate（防止同一個 nonce 被重放），只要有任何一個 gate 沒過，就什麼都不會被持久化。

最後，這個模組同樣明確劃出自己不是什麼：它不會開啟一個 SSH session、一個 socket，或一個 websocket/HTTP 伺服器；它不會讓 Claude Web/Mobile 真的連得到這台機器，也不會讓這台機器連得到某台 Linux DV 伺服器。以網路為基礎的遠端存取，本身仍然是——而且將持續是——一項外部能力（可能是人類自己的 SSH session，也可能是 Claude Code 自己另一套獨立的 Remote Control 功能），如果它真的存在，它會去讀寫的正是這裡的同一份 `session.json / audit_log.json`。這個模組只讓底層的狀態機變得真實而且真正被強制執行——它並不建構那條傳輸線本身。這正好呼應了 CLAUDE.md 那句「Remote Control is transport only; Harness governance remains authoritative」：治理邏輯永遠留在 `control_plane.py / engine.py` 這一側，Remote Control 不論走哪一條傳輸路徑，都必須先通過同一組真實的 gate 才能落地。

16. 完整 CLI 指令參考與端到端使用範例

本章整理 `dv_harness/cli.py` 中 `dv-harness` 這支主指令目前實際掛載的所有 top-level 子指令。這個數字必須以原始碼為準、而非以任何舊文件或記憶為準；直接對 `dv_harness/cli.py` 建構出的 `argparse.ArgumentParser` 做程式化內省（列舉其 `_SubParsersAction.choices`，等同於逐一核對每一個 `sub.add_parser(...)` 呼叫），2026-09-07 校對當下實測共 202 個——不是任何一份舊文件或本文件先前草稿引用過的「約 90 個」或「116 個」；後者兩個數字都已經是過期的快照，本章校對時已同步更正。每個子指令都對應 `dv_harness/` 下一支真實的 Python 模組。

重要的同源更正：2026-09-06 CLAUDE.md 新增的「CLI Wiring: 111 Modules Connected to dv-harness」一節記錄了一次批次接線工作——把 111 個原本「REACHED、未 WIRED」（只有 `python -m dv_harness.<module>` 這種獨立 ad hoc 入口）的分析模組，逐一以 `sub.add_parser(...)` 掛到 `dv-harness` 底下，成為正式子指令。**這次批次接線發生在多數個別模組小節的 CLAUDE.md 說明文字寫成之後**——因此第 2 章至第 14 章裡，凡是某個模組小節聲稱「沒有 dv-harness CLI 動詞」「前門只是 `python -m dv_harness.<module>`」的敘述，都只反映該模組小節自己被寫下當時的快照，其中部分（例如 `arbitration_policy_ir.py`、`security_policy_ir.py`、`amba_functional_coverage_ir.py`、`amba_performance_readiness_gates.py`、`branch_ownership_resolver.py`、`runtime_event_registry.py`、`command_precondition_gate.py`、`de_command_runtime_readiness_gate.py` 等——本次校對已逐一核對並在原地訂正見下）已經在這次批次接線之後改用真正的 `dv-harness <verb>` 子指令；另一部分（例如 `ordering_domain_graph.py`、`qos_policy_ir.py`、`amba_performance_calculator.py` / `_requirement_checker.py` / `_classification.py`、`spec_doc_map.py` 家族的部分模組）**確實**在批次接線之後仍然維持 `python -m` 前門，因為它們不在這批 111 個模組之列。換句話說，兩種情況都是真的，只是分佈在不同模組上——本文件不能再假設「這批新模組一律沒有 dv-harness CLI 動詞」這個在 2026-09-06 之前成立、但現在已經不再全面成立的通則。**任何時候若懷疑某個模組是否已有 `dv-harness` 子指令，本章 §16.1 的分類表與現場執行 `dv-harness --help` 才是最終依據，個別模組小節的敘述文字不是。**

指令的全域參數只有兩個：`--project-root`（預設目前目錄）與 `--degradation-transport`（覆寫 `config.json` 的 DEGRADED 模式探測傳輸方式，可選 `auto` / `local` / `remote_relay` / `off`）。每一次呼叫都會在 `.dv-harness/events.jsonl` 留下一筆 `CLI_ACCESS` 稽核事件（見 `dv-harness audit`），這是 `user-info` 判斷「誰在何時用過這個部署」的依據，讀取類指令（`status/explain`）也不例外。

16.1 指令分類參考表

下表依主題分組，每列給出一行說明（來自各 parser 的 `help=` 文字或對應模組的 CLAUDE.md 段落）。

（一）專案生命週期 / 人機控制平面（Control Plane）

指令	說明
<code>start --goal <G> [--loop] [--dry-run]</code>	啟動/推進自主 stage graph； <code>--loop</code> 讓引擎連續跑到人工節點或錯誤為止
<code>run-stage --goal <G> [--stage <S>]</code>	只跑單一 stage（不進入 loop）
<code>status</code>	印出目前 stage、狀態、DEGRADED 探測傳輸決策等摘要
<code>advance</code>	明確推進到下一個 graph stage（同時觸發 question-queue digest 邊界）
<code>stats</code>	<code>compute_stats()</code> 統計快照
<code>set-stage <STAGE></code>	強制設定目前 stage（不經 gate 驗證）
<code>mark <STATUS> [--message]</code>	手動標記一個 <code>Status</code> 值
<code>explain [--stage]</code>	印出該 stage 的通用說明 + 這次執行真正的 blocking_reason/gate 結果
<code>pause [--reason] / resume</code>	在下個 stage 前乾淨停止自主迴圈 / 解除 PAUSE
<code>takeover [--message] / release-takeover</code>	人工接管目前 stage（ <code>loop()</code> / <code>run_stage()</code> 不再碰它） / 釋放接管
<code>redirect <STAGE> [--reason]</code>	明確、留稽核紀錄地強制指定下一個 stage
<code>approve <STAGE> [--note] [--reviewer-id] [--reviewer-confidence]</code>	記錄人工 sign-off（PROMOTION_READINESS / SIGNOFF 等必需）
<code>evidence [--stage]</code>	顯示該 stage 最後一次回應真正產生的 evidence block / gate 判決
<code>checklist [--stage]</code>	人類可讀的 stage 完成度百分比 + entry/exit checklist
<code>correct</code>	不重跑 agent，直接提供人工修正
<code>constraint</code>	管理會摺入之後每個 stage prompt 的持久化限制
<code>cosign</code>	對某個 DV_JUDGMENT 欄位做具持久性的人工共同簽署
<code>`blackboard write</code>	<code>read`</code>
<code>audit</code>	最近 N 筆 <code>events.jsonl</code> 稽核事件 + 目前狀態
<code>`remote-control bootstrap</code>	<code>status</code>
<code>save-session / restore-session</code>	快照 / 還原目前 run-state 層（state/control/blackboard 等）
<code>stage-profile</code>	每個 stage 的 wall-clock/token/tool-call/retry 報告
<code>stage-report</code>	產生 stage 執行報告
<code>`config set</code>	<code>get`</code>
<code>`context-budget hook</code>	<code>session-start</code>
<code>agent-checkpoint-check / agent-parallelism-policy</code>	agent checkpoint 檢查 / 平行度政策

(二) LSF / 平行任務 / 執行監控

指令	說明
lsf [job_id] [--list]	逐 job 查詢 (等同 GET /api/lsf/jobs[/<id>])
lsf-submit <command> --queue <Q> [--cores][--mem-mb][--pattern][--seed] [--runlimit-min][--skip-preflight]	真正的 bsub · 前面先跑 preflight gate
preflight [--queue][--workdir][--license-server][--remote]	獨立跑 lmsat + scheduler preflight gate · 不送任何 job
lsf-kill <job_id> [--no-verify][--poll-timeout-s]	真正的 bkill · 並驗證 job 真的離開 RUN/PEND
lsf-reconcile [job_id...] [--all]	真正的 bjobs 查詢 · 回寫 job 狀態並回報 WARN/CRITICAL 落差
lsf-watch-start --vcuser <U> [--interval-minutes]	啟動背景 job/log monitor (預設 5 分鐘一輪)
lsf-watch-stop / lsf-watch-status	停止 / 查詢背景 monitor
`pueue add	status
lsf-auto-kill-scan [--dry-run]	依 early_fail_policy.json 掃描 RUNNING job 並視情況 bkill · 人工/排程觸發 · 不會自己跑
`regression-tier list	plan
fsdb-report --fsdb <F> [--out][--topic][--job-id][--pattern]	執行真正的 fsdbreport · 解析輸出並落地為 normalized evidence
`sim-log-analyze --log	--log-text`
trend [--json][--seats-per-job][--rtl-paths-spec][--min-baseline-samples] [--ratio-threshold][--z-threshold]	evidence DB 的跨批次趨勢：pass-rate/coverage/runtime/license-hour 曲線、回歸偵測
platform-health [--json][--window-days]	各子系統 HEALTHY/DEGRADED/CRITICAL/UNKNOWN 聚合 + 2 個可誠實量測的 SLO
connectivity-check [--config][--state][--report][--check-only]	重跑 3 個 machine gate (elaboration / 靜態 zero-time connectivity / transaction activity)

(三) 需求 / vPlan / DUT-VIP intake / 環境生成

指令	說明
`run-profile extract	justfile`
uvm-lint --env-dir <D> [--json][--fail-on-error]	對已生成 UVM 環境做編譯前結構化 lint (factory 註冊、phase 簽章、config_db、TLM、objection)
power-intent --upf <F> [--json][--fail-on-error]	解析真正的 UPF (IEEE 1801) · 抽出結構化模型
requirement-contract --requirements <F> [--json][--fail-on-error]	Spec §184 Canonical Requirement Contract : COMPLETE/PARTIAL/AMBIGUOUS/CONTRADICTIONARY/UNKNOWN
schema-compat	分類一次 JSON Schema 變更為 BACKWARD_COMPATIBLE/...
`golden-scenario record	list
vip-api-check	驗證生成程式碼引用的 VIP API 是否真的存在 (PROVEN/BLOCKED/UNPROVABLE)
`config-variants plan	verify --space [--strength N]`
`supply-chain inventory	check
`benchmark-dataset register	list
system-smoke-proof [--merge-only][--json]	\$206 系統 build & smoke proof 階梯 (merge collision 檢查為真 · 其餘依附既有真實 gate)
env-manifest generate	產生/更新 env.manifest.json (vip_config/dut_facts/env_topology 三層)
vip-user-guide distill	離線萃取 VIP user guide (或任何 protocol PDF) 為受限摘要
`doc-extract categories	plan
vplan-export items.json --out <F.xlsx> --protocol <P> --pattern-dir <D> --dispatcher-file <F> [...]	產出真正可開啟的 vPlan .xlsx · 並對照真實 pattern/dispatcher/task 來源交叉驗證
`exemptions list	add
reference-audit [--escalate]	機械抽取每個 register-write 巨集呼叫 · 比對 host/DUT 側是否對稱

(四) Coverage / Signoff / 治理 (Governance)

指令	說明
signoff-export --out <D> [--require-signoff-pass]	一鍵打包最終 signoff bundle (vPlan/regression/requirements/findings/state/telemetry/pattern registry/生成 UVM 原始碼) · 附 manifest.json
git-guard	main/master 唯 PR 治理閘：擋掉 AI agent 直接 push/merge
blast-radius --base <rev> [--head <rev>]	變更波及範圍評估 (大到需要人工 CHANGE_BLAST_RADIUS 核准)
self-audit	自我稽核通道

指令	說明
<code>mutation-test [--module][--test][--operator][--max-mutants][--min-score]</code>	對本repo自己的測試套件做突變測試，衡量測試真的能抓到錯誤的比例
<code>`authority order</code>	check-doc
<code>`self-tune status</code>	list
<code>`cross-project register</code>	unregister
<code>`research ... [--compare</code>	--impact
<code>`coord detect</code>	reserve
<code>`loop-contract states</code>	list
<code>`loop-budget dimensions</code>	status
<code>golden-flow-readiness [--json]</code>	\$47 二十列黃金流程就緒矩陣
<code>generation-readiness [--json][--no-deep]</code>	\$211 生成能力就緒矩陣 (Flow A 分系統 / Flow B 系統級)
<code>`verification-strategy capabilities</code>	recommend`
<code>`confidence-calibration tiers</code>	report
<code>`consolidated-kpi-benchmark names</code>	report

(五) Memory / Knowledge Center / Question Queue

指令	說明
<code>`knowledge setup</code>	status
<code>`memory status</code>	search
<code>user-info</code>	誰用過這套部署、何時
<code>`question-queue add</code>	list
<code>`waveform-dump-scope ask</code>	status`
<code>`harness-deploy manifest</code>	plan

(六) AMBA fabric / 系統級 design-intelligence (獨立分析模組群)

這一群約 20 幾個窄範圍的分析指令，大多對應 2026-09-06 這幾天新加的 IR/gate 模組，全部「讀多寫少」——通常只讀取一份 caller 提供的 JSON facts 檔並輸出分類/驗證結果，不跑 build、不送 job、不核准任何東西，也刻意不掛 STAGE_GATES。舉例：`amba-functional-coverage-ir`、`amba-readiness-gates`、`amba-performance-readiness-gates`、`arbitration-policy-ir`、`backpressure-model`、`coherency-capability-ir`、`command-precondition-gate`、`command-task-trace`、`command-txt-change-impact`、`connectivity-check`、`coverage-closure-action-utility`、`coverage-closure-hole-correlation`、`coverage-db-integrity fingerprint|check`、`de-command-runtime-readiness-gate`、`dependency-qualification`、`design-architecture-ir`、`design-knowledge-correlation`、`branch-ownership-resolver`、`change-cascade`、`checker-sb-qualification`、`bounded-self-healing`、`artifact-completeness`、`artifact-relationship-discovery`。每個都有自己的 `--help`；需要哪個功能時，先跑 `dv-harness <verb> --help` 看它要求什麼輸入檔格式，再對照 `dv_harness/<module>.py` 開頭的 docstring 確認它「Deliberately bounded」的邊界（幾乎每一支的模組檔案都在自己的文件字串裡明講「這個模組不做什麼」）。

16.2 情境一：從 Intake 到第一次 Test-Proof 的 SUBSYSTEM_MODE UVM 環境建置

依 CLAUDE.md 的「Environment Generation Mode」規則，開工前要先明確宣告 SUBSYSTEM_MODE 或 SYSTEM_LEVEL_MODE；本情境走 SUBSYSTEM_MODE。

```
# 1. 啟動自主流程，宣告目標 (會依 graph 走過 INTAKE → ARCH_DISCOVERY → ...)
dv-harness start --goal "Build USB3 device-side subsystem UVM environment" --loop

# 2. 真正的生成入口 (不是 dv-harness 子指令，是獨立腳本，
#    十個 PROTOCOL_BUILDERS/*/SKILL.md 都是呼叫它)
python tools/generate_protocol_uvm_environment.py --manifest manifest.json --out generated_env/ --root .

# 3. 生成後立即跑 env.manifest.json (3 層事實檔：vip_config/dut_facts/env_topology)
dv-harness env-manifest generate

# 4. 編譯前結構化 Lint (factory 註冊、phase 簽章、config_db、TLM、objection 平衡)
dv-harness uvm-lint --env-dir generated_env/ --fail-on-error

# 5. 若有電源意圖，解析 UPF
dv-harness power-intent --upf design/power_intent.upf --fail-on-error

# 6. 首次成功編譯後，依「Mandatory bind-verification checkpoint」規則跑三機關 gate
dv-harness connectivity-check

# 7. 送出第一個 test-proof job (先過 preflight，再 bsub)
dv-harness lsf-submit "make sim TEST=usb3_dev_link_up_test" \
```

```

--queue normal --cores 4 --pattern usb3_dev_link_up_test --regression-id first_proof

# 8. 啟動背景 job/log monitor，維持 regression.list 安全網
dv-harness lsf-watch-start --vcuser myacct

# 9. 檢視這個 stage 目前完成度與缺什麼證據
dv-harness checklist --stage BUILD_DEBUG

```

依 CLAUDE.md 的 Simulation Observability Default，第一輪一律 FSDB OFF；只有在這個 test-proof 真的失敗且需要 signal-level 證據時，才在確認 dump scope 後 (`dv-harness waveform-dump-scope ask --scope ... --level-or-depth ...`) 針對性重跑。

16.3 情境二：跑一次 LSF Regression 並 Triage 失敗

```

# 1. 獨立跑一次 preflight (license/queue/host/disk)，確認值得排隊再送
dv-harness preflight --queue normal

# 2. 宣告這批屬於哪一級節奏 (決定 UVM_FATAL 升級門檻)
dv-harness regression-tier plan SMOKE
dv-harness regression-tier start SMOKE

# 3. 確保背景 monitor 在跑 (REMOTE_EXECUTION 宣告後必做)
dv-harness lsf-watch-start --vcuser myacct

# 4. 逐一送出 SMOKE 清單裡的每個 pattern (真正 bsub，會先過 preflight gate)
dv-harness lsf-submit "make sim TEST=usb3_dev_suspend_resume" \
--queue normal --cores 2 --pattern usb3_dev_suspend_resume --regression-id smoke_0906

# 5. 觀察 job 狀態
dv-harness lsf --list
dv-harness lsf 128231 # 單一 job 細節

# 6. 與真實 bjobs 對帳，回寫狀態 (LSF DONE 不等於 DV PASS，見 CLAUDE.md)
dv-harness lsf-reconcile --all

# 7. 若某個 pattern 是 FAIL：先看 sim.log 分類 (marker/epilogue)
dv-harness sim-log-analyze --log runs/usb3_dev_suspend_resume/sim.log

# 8. 需要訊號級證據才重跑波形 (First-Failure Waveform Rerun：先問過 dump scope)
dv-harness waveform-dump-scope ask --scope usb3_dev_suspend_resume --level-or-depth "link layer only"
dv-harness question-queue answer <Q-ID> --decided-by "王工程師" --answer "link layer only, depth=2"
# ...重跑並開啟 targeted waveform 後...
dv-harness fsdb-report --fsdb runs/usb3_dev_suspend_resume/wave.fsdb --pattern usb3_dev_suspend_resume

# 9. 需要時中止卡住的 job
dv-harness lsf-kill 128231

# 10. 收工前看這批的跨批次趨勢 (是否是新回歸、是否對到某段 commit)
dv-harness trend --json

```

16.4 情境三：關閉 Coverage、產出 Signoff Bundle

```

# 1. 檢查目前 waiver/exemption 是否有過期或即將過期
dv-harness exemptions check
dv-harness exemptions expire-report

# 2. 對已證實通過的關鍵測試建立 golden scenario capsule (會拒收非 PASS 或證據缺失的紀錄)
dv-harness golden-scenario record --json-file capsules/link_training_pass.json
dv-harness golden-scenario status # 依真實 git diff 判斷是否仍 FRESH

# 3. 檢查需求契約完整性 (§184 五值狀態：COMPLETE/PARTIAL/AMBIGUOUS/CONTRADICTORY/UNKNOWN)
dv-harness requirement-contract --requirements requirements.json --fail-on-error

# 4. 產出真正可開啟的 vPlan .xlsx (會對照真實 pattern/dispatcher/task 來源做交叉驗證)
dv-harness vplan-export vplan_items.json --out deliverables/vplan.xlsx \
--protocol USB --pattern-dir patterns/ --dispatcher-file tb/patterns_registry/dv_uvm_pattern_pool.svh

# 5. 檢視整體就緒矩陣 (20 列黃金流程 + 20 列生成能力)
dv-harness golden-flow-readiness --json
dv-harness generation-readiness --json

# 6. 人工 sign-off (SIGNOFF 這個 stage 硬性要求真實核准紀錄)
dv-harness approve SIGNOFF --note "coverage 100%, waivers reviewed" --reviewer-id "王工程師" --reviewer-confidence HIGH

# 7. 一鍵打包最終 signoff bundle
dv-harness signoff-export --out deliverables/signoff_bundle/ --require-signoff-pass

```

`signoff-export` 打包的內容包含 vPlan、blackboard 上的 signoff/regression/requirements/findings 狀態、stage 執行 telemetry、pattern registry、生成的 UVM testbench 原始碼與 regression manifest，並附上一份重新跑過的 self-audit 結果與 `manifest.json`（誠實記錄哪些項目真的存在、哪些缺失）。`--require-signoff-pass` 預設關閉，因為 `signoff_bundle_completeness_gate`（SIGNOFF 九個 gate 之一）本身就要吃一份 bundle 當輸入，所以 bundle 必須在 SIGNOFF 通過前就能產出；不論開關與否，bundle 都會誠實記錄目前 stage 的真實狀態，並在 `bundle_kind` 欄位標明是 `SIGNOFF_GATE_VERIFIED` 還是 `PRE_SIGNOFF_GATE_INPUT`——這個區分本身，就是本專案「Evidence Truth Rule」在一個具體檔案格式上的體現：一份尚未經 gate 驗證的 bundle，永遠不會被偽裝成已驗證的 bundle。